

UltraFastRng512 V1 - Deconstruction Book (English Edition)

Documentation release v3.3 - English edition; translated from the complete v3.3
Japanese/English source

UltraFastRng512 V1 research documentation

2026-09-30

Contents

UltraFastRng512 V1 - Deconstruction Book	7
0. Purpose of this document	7
0A. Executive Summary - Definitions and current status to read first	7
Part I - The whole architecture at a glance	9
1. In one sentence	9
2. End-to-end pipeline	9
	10
2A. Three points readers should confirm first	10
2B. Two-layer quality evaluation - do not confuse standard RNG tests with MC validation	10
Part II - Design principles that should be understood first	16
3. Why separate Back and Front?	16
4. Why is one-core-complete the baseline?	16
Part III - Seed / Back	17
5. Entropy conditioning	17
6. LPS / PSL2(F29)	17
7. Seed Bank	18
8. CHACHA20-AAA-A1	18
Part IV - VHV / 3 Parents / Front	18
9. V -> H -> V	18
10. 3 Parents	19
11. 16-resident-state budget	19
12. Front recurrence	19
13. 13 auxiliary states	20
14. The 16 Front outputs	20
Part V - Normal v9	21
15. What does Normal do?	21
16. Why an LUT?	21
Part VI - 4-way Fanout	21
17. Fanout	21
Part VII - PaperView / cultivation layer	22
18. What is PaperView?	22
19. epoch -> phase / shift / view	22

20. Why is the cultivation layer necessary?	23
Part VIII - 465 / 555	23
21. Generic final-cache stride = 465	23
22. R2-only stride = 555	23
Part IX - Root-cause discovery and correction	24
23. The old problem	24
Part X - MC Execution Layer	25
25. Direct / L1 / L2	25
26. Final profiles	25
27. Meaning of R2	26
Part XI - Principles of Optimization	26
28. Basic optimization principles adopted in V1	26
Part XII - Current Optimization Option Surface	26
29. V8/V9 optimization profiles	26
30. Finance / Pharma	27
Part XII-A - MC Optimization Techniques - Structure, Mechanism, and Measured Results	27
30A. The overall picture first	27
30B. Five basic optimization techniques	27
30C. R2-type Monte Carlo	28
30D. Measured R2 performance	30
30E. Finance Monte Carlo	30
30F. Pharma Monte Carlo	32
30G. Why does R2 look so much faster?	33
30H. Shared optimization philosophy	33
30I. Rules for optimizing without destroying quality	34
30J. Judge quality correction and speed separately	34
30K. Integrated MC optimization diagram - reader edition	34
30L. Measured-value list - to prevent the impression that only R2 was measured	35
30M. Short explanation for others	36
Part XII-B - Features, Quality Options, and Reproducibility Options Beyond Speed	36
30N. Four option families	36
30O. Normal-output option - Antithetic	36
30P. Final-Cultivation / output format	37
30Q. Cultivation / Transport	37
30R. Chunk / Generation	37
30S. Common Post - result-analysis functions	38
30T. Distribution	38
30U. Convergence / stopping conditions	38
30V. Transform	39
30W. Metadata	39
30X. Advanced analysis	39
30Y. Pairing / Grouped	39
30Z. Seed / Profile exchange	40
30AA. AAA profile - fixed quality-side construction	40
30AB. VHV Seed lineage	40
30AC. Dynamic Seed Exchange - core mechanism for rotating Seed generations without stopping Front	41
30AD. Deterministic Stream / CRN	45
30AE. Checkpoint / Exact Replay	45
30AF. Golden Reference / Verification	46

30AG. Finance / Pharma domain adapters	46
30AH. Explain the whole system on one page	46
30AI. The most important point in the design	47
30AJ. Short explanation for others	48
Part XIII - Reproducibility	48
31. Deterministic Stream	48
32. CRN	49
33. Worker partition / seek	49
34. Checkpoint / Exact Replay	49
Part XIV - Quality Evaluation	50
35. Current V1 aggregate quality	50
35A. Raw Front-state bitstream qualification	50
36. Why sign / dep2 are emphasized	52
Part XV - controlled ablation	52
37. What was removed?	52
38. What does this experiment tell us?	53
Part XVI - inter-stream	53
39. 8-stream audit	53
Part XVII - MC convergence	53
40. Do not assume that a single stream is IID	53
41. Stream-ensemble convergence	54
Part XVII-A - Optimization Quality / Anti-Reuse	54
41A. The two questions separated by this audit	54
41C. Why Full-q256 Uniqueness Alone Is Insufficient	54
41D. Deep-Audit Measurement Results - PaperView	55
41E. Deep-Audit Measurement Results - CHUNK Generation	55
41F. Deep-Audit Measurement Results - FIXED Generation	55
41G. What Does "46 Gsamples/s" Count?	55
41H. Added Fresh-Sample Companion Benchmark	56
41I. Documentation Rule	56
41J. Final Audit Conclusion	56
41K. Measurement Results in Numbers - So Readers Can Judge for Themselves	57
41L. Final Reader Assessment - "Can This Acceleration Be Used for MC?"	60
Part XVIII - Performance	61
42. Current V1 Speed References	61
42A. The Correct Interpretation of the 3-Core Experiment - 12.7485 Gsamples/s Is Not "V1 Getting Slower"	62
43. Why Performance Should Not Be Reduced to a Single Number	65
Part XIX - External RNG Tests	66
44. Current Status	66
Part XX - Positioning Relative to Existing Methods	66
45. Academic Comparison Targets	66
Part XXI - Novelty	67
46. What Must Not Be Presented as Novelty	67
47. System-Level Contribution That May Be Examined for V1	67
Part XXII - How to Use the Project as a POC	67

48. Minimal Build	67
49. Self Verification	67
50. Representative MC Run	68
Part XXIII - What Must Not Be Changed in V1	68
51. Paper Baseline Integrity Rules	68
52. Do Not Assume "Quality Improvement Means It Is Safe"	68
Part XXIV - Current Status of V1	69
53. Areas That Are Currently Strong	69
54. Areas Still Requiring External Evidence	69
Part XXV - Why Stop at V1 for the Time Being	69
55. The Correct Research Cycle	69
Appendix A - V1 Identity	70
Appendix B - V1 Constant Card	70
Appendix C - What the Project Does NOT Claim	70
Part XVII - Evidence Management for Academic Materials	71
40. Why V1 Is Frozen as the Reference Point	71
41. What Has Already Been Demonstrated	71
42. Do Not Hide Important Unfinished Items	71
43. Boundary of External Testing	71
44. Three Levels of Academic Comparison	71
45. Positioning in the Literature	72
46. Recommended POC Reading Order	72
Part XXVI - Exact Implementation Ledger (V1)	72
56. Three-Parent State Placement - Exact Source Mapping	72
57. Exact Parent Recurrence	72
58. Exact Auxiliary Update Schedule	73
59. Exact Output Emission and Fanout	73
60. Exact Normal-v9 Semantics	74
61. Exact PaperView Control Variables	74
62. Exact Final-Cache Rule	75
63. Scientific Interpretation Boundary	75
Part XXVII - Academic Evidence Map	75
64. Evidence Layers	75
65. What Is Already Strong Enough for the Paper	76
66. What Remains Genuinely External	76
Part XXVIII - Related Work: Defensible Positioning	76
67. Random123 / Counter-Based Generators	76
68. PCG	77
69. SFMT and AVX-512 LFSR113	77
70. MRG32k3a / RngStreams	77
71. Ziggurat	77
72. Intel oneMKL	77
Part XXIX - Recommended Paper Claim Set	77
73. Strong Claims That Are Supported	77
74. Claims That Require Qualification Before Use	78

Part XXX - Reader's Execution Path	78
75. How a Third Party Should Approach the Project	78
Final Status	78
Part XXX - Treatment of Publicly Published Benchmarks	79
76. Separate Same-Condition Comparisons from Reference Values	79
77. Public Speed References Confirmed at This Stage	79
78. Public Normal-Generation Reference Landscape	80
79. How to Compare V1 - Separate Raw RNG Speed from End-to-End MC Speed	80
Part XXXI - Three-Core Scaling Experiment	81
80. From One-Core-Complete to Three Cores	81
81. Distinction from Back Placement Experiments	82
82. Wording for the Paper	82
Part XXXII - Current Comparison Strategy	82
83. Recommended Paper Figures	82
84. Current Conclusion	83
Part XXXIII - Comparison Figures for Paper / Deconstruction Book	83
85. Figure Strategy	83
86. V1 Measured Throughput Figure	84
87. Published RNG Reference Landscape	86
87A. How Readers Should Read the Comparison First	86
88. Published Normal Reference Landscape	87
89. Normal Tail Figure	88
90. Raw Front-State p-Value Histogram	89
91. Raw Front-State p-Value Heatmap	90
92. Battery Overview	91
93. Three-Core Scaling	92
94. Same-Condition Comparison Remains the Paper-Grade Target	92
95. Figure Package	93
Part XXXIV - Three-Core Benchmark Baseline Clarification	93
96. Why the Three-Core Experiment Uses 12.7485 Gsamples/s as Its One-Worker Reference	93
97. CPU Microarchitecture Explanation	94
98. The Apparent "Drop" Is Therefore Not a V1 Algorithm Regression	94
99. VM Scheduling and Raw-Run Variability	95
100. Recommended Paper Wording	95
101. Forward-Looking Scaling Statement - Use the Measured Peak as the Envelope	95
102. Latency / Energy / TCO Hypothesis - Explicitly Unmeasured	96
Part XXXV - Mathematical Core / Full Formalization	96
103. Purpose and Position of This Chapter	96
104. Core Mathematical Objects	96
105. Front Parent Recurrence - Formal State Transition	97
106. 13-Phase Auxiliary State Transition	98
107. 16-Vector Emission Map	99
108. Normal-v9 - Mapping from Raw Integer State to the Normal Domain	99
109. Four-Way Fanout - Algebraic Representation	100
110. AVX-512 - V1 as 512-bit Vector Algebra	101
111. PaperView / Final Cultivation - Mathematical Meaning	102
112. Theoretical R2 Distribution - From the Normal Domain to the MC Quantity	103
113. State-Space Size and Period - What the 8192-bit State Implies	104
114. Separate Theoretical Limits from Empirical Results	104
118. Mathematical-Core Audit Result	107

Part XXXVI - Bit-for-Bit Reproducibility and Exact Implementation Semantics	107
119. Purpose of This Chapter	107
120. Binary Encoding Rules	107
121. Light Cleanup - Exact Behavior	108
122. SHA-256 Entropy Conditioning - Exact Payload	108
123. LPS Selector - Exact Bit Consumption	108
124. LPS Lane Seed Derivation - Exact Domain Separation	109
125. canon (X) - Exact Representative Rule	109
126. RootSeed Extraction - Exact Byte Layout	110
127. Seed Bank and Profile - Exact Sequence	110
128. VHV - Exact Front16 Seed Materialization	111
129. Front Recurrence - Closed State Transition	111
130. Exact Resident-State Map	112
131. Normal-v9 - Signedness and Saturation Semantics	112
132. Exact Logical-Sample Accounting	113
133. PaperView - Exact Materialization Boundary	113
134. Final-Cache and R2-Cache - Exact Consumer Contract	114
Part XXXVII - Back / Front Seed-Exchange State Machine	114
135. Slot States	114
136. Preparation and Handoff State Machine	115
137. Exact Switch Condition	115
138. Requested Generation Interval vs. Actual Exchange Interval	116
Part XXXVIII - Benchmark Reproduction Protocol	116
139. Fixed Items for Reproducing Benchmark Results	116
140. Current V1 Peak Reproduction Record	116
141. Boundary for Comparison with Published Raw-RNG References	117
Part XXXIX - Domain Adapter Mathematical Reference	117
142. Finance Common Log-Domain Path	117
143. Pharma Two-Compartment Path	118
144. Piecewise-Constant Infusion / Segment Transition	119
Part XL - Benchmark and Reproducibility Checklist	119
145. Third-Party Reproduction Checklist	119
146. Reproduction Granularity	120
Part XLI - Exactness Audit Revision Record	121
147. V2.5 / V2.6 Correction Set	121
148. Authority Rule	122
Appendix D - Complete Static Implementation Tables	122
Appendix E - Final Implementation-Fidelity Notes	124
E.1 V1 Canonical Normal Scale	124
E.2 V1 Canonical Source Identity	124
E.3 Scope of This Complete Specification	124
Appendix E - Embedded Comparison Figures	124
E.1 Scope and Measurement-Scope Overview	125
E.2 Scope-Aware Throughput Scale	125
E.3 Measurement-Environment Context	126
E.4 Published RNG Reference Landscape	126
E.5 Published Normal-Generation Reference Landscape	127
E.6 V1 Normal Tail	127
E.7 1024-Position p-Value Histogram	128

E.8 1024-Position p-Value Heatmap	129
E.9 Statistical Battery Overview	130
E.10 Three-Core Scaling	131
E.11 Supplemental Data Package	131

Appendix F - External Source Notes Used by the Manuscript	131
--	------------

Appendix G - Documentation License and Intellectual Property Notice	132
Intellectual Property and Use Notice	133
Component License Separation	133

UltraFastRng512 V1 - Deconstruction Book

Documentation release: v3.3 - promotes Dynamic Seed Exchange to a first-class core architecture chapter while preserving the v3.3 scaling rationale, measurement boundaries, and reproducibility rules. This release reconciles the book with the frozen V1 source at bit-for-bit level, corrects the Normal-v9 operand semantics, R2 pair mapping, PaperView shift/tail rules, physical starter representation, and the complete static Normal LUT appendix.

0. Purpose of this document

This document is an integrated guide to the design, implementation, measurement, and use of UltraFastRng512 V1, written so that the system can be understood, reproduced, and evaluated without an oral explanation from its author.

The target is not merely a random-number function. V1 is a multi-layer Monte Carlo execution platform consisting of:

Seed preparation -> Front RNG core -> Normal generation -> data cultivation -> Monte Carlo transport/consumer -> reproducibility/control plane -> domain adapters

The center of this book is **V1**. V2 and later versions are treated only as research history.

0A. Executive Summary - Definitions and current status to read first

When evaluating V1 performance and quality, first separate “what was measured” from “what has not yet been measured.”

Confirmed

- V1 has an **end-to-end Monte Carlo execution path** connecting seed/state prepared by Back to Front, Normal v9, 4-way fanout, PaperView / Final-Cultivation, and the MC consumer.
- The peak observed in a strict one-core complete benchmark is **46.602 G logical MC samples/s**. This is an observed result on a shared VM and does not mean that 46.602 billion independent new Normal values were generated per second.
- For the Finance/Pharma acceleration options audited here, `samples = 32768` and `sample_ratio = 1.0000000000` were maintained, and the maximum relative difference from the intentional BASELINE was no more than $5.594e-07$ within the tested scope.
- Dynamic Seed Exchange is not a quality option or an MC optimization option; it is a **core mechanism that makes the Front/Back separation work**. Back A/B prepares the next-generation Seed, and only a completed Seed that has reached READY is handed off to Front at a safe point. If Back is not ready in time, Front continues using the current Seed.
- The four PaperView views are not byte-identical, but within one chunk they reference the same scalar Normal pool through different deterministic layouts. Therefore, logical sample count must not be equated with independent sample count.

- Under FIXED generation, the same starter pool is reused within the generation interval. This is an intentional cultivation / re-indexing policy and must be distinguished from raw RNG cloning.

Confirmed observations on scaling

- In the 3-core dedicated scaling experiment, independent workers were increased from 1 to 3 on the same AMD EPYC 9V74 environment. On the clean full-active subset, **12.7485 -> 38.656 Gsamples/s, 3.032x aggregate scaling** was observed. This is not “46.602 Gsamples/s multiplied by three”; it is the observed scaling ratio relative to the local baseline used for the dedicated scaling experiment.
- The purpose of the 3-core experiment was to verify experimentally whether the V1 one-core-complete execution model can be expanded into multiple independent workers.
- In the high-core-count illustration, the separately observed **one-core complete peak = 46.602 Gsamples/s** can be used as the starting point for a conditional **8.948 T logical samples/s** envelope under perfect linear scaling to 192 workers. This is neither a 192-core measurement nor a prediction derived from the 3-core median.

Not yet established

- **Final RNG-level qualification by standard named PractRand / TestU01 suites is not complete.** The 512 MiB surrogate battery and MC application-level validation are additional evidence and must not be treated as substitutes for PractRand/TestU01.
- Direct **same-host / same-build / same-MC-work** comparisons against oneMKL, Ziggurat, PCG, xoshiro, and other established methods are not complete. Published values are used only as contextual references.
- The **effective CPU cycle count / frequency mapping required to convert the 46.602 Gsamples/s figure into Cycles/Sample is not fixed in the current record.** Therefore, estimates such as 0.064 cycles/sample obtained by assuming 3.0 GHz are not used in this book.
- The upper performance ceiling on a dedicated bare-metal environment has not been measured.

A sentence for readers

This edition does not attempt to prove how many times faster V1 is than a “standard RNG alternative.” Instead, it measures separately the performance boundary, statistical quality, reuse conditions, and reproducibility of the complete path actually fed into Monte Carlo, and presents the resulting evidence in a traceable form.

Likewise, the 3-core benchmark is not a speed-showcase experiment. It tests whether one-core-complete execution can be expanded as independent workers. The 192-core figure is a conditional peak-envelope thought experiment based on the measured peak and observed scaling context.

Important additions in v3.3

After v3.0 made comparison methodology more conservative and v3.1 restored detailed explanations of MC optimization and domain-specific options, v3.3 promoted the **meaning of the 3-core experiment and the positioning of the 192-core future envelope** into the Executive Summary. This makes it clear, before readers reach deeper sections, that the 3-core measurement is a worker-level scalability test rather than a simple 3-core speed test, and that 8.948 T logical samples/s at 192 cores is a conditional thought experiment rather than a measured result.

Part I - The whole architecture at a glance

1. In one sentence

UltraFastRng512 V1 is:

An AVX-512-oriented Monte Carlo platform that pushes heavy Seed preparation to Back, generates data rapidly in a lightweight Front holding sixteen 64-byte states, and connects that output to integer Normal generation, 4-way fanout, PaperView / Final-Cultivation, and domain-specific MC consumers.

The key is not to mix random-number generation, Normal transformation, data placement, MC computation, and reproducibility into one undifferentiated operation.

2. End-to-end pipeline

Physical / test entropy



Light cleanup



SHA-256 entropy conditioning



LPS / PSL2(F29), 4 lanes



512-bit Root Seed



Seed Bank



CHACHA20-AAA-A1



V -> H -> V Front16 preparation



3 same-profile parent lineages A/B/C



16 resident Front states



Front recurrence



Normal v9 integer transform



4-way fanout



64 KiB logical PaperView / cultivation unit; physical retained starter is 32 KiB int16 plus

```

      |
      +-----> Generic final cache, stride 465
      |
      +-----> R2 pair cache, stride 555
      |
      v
MC consumer / domain adapter
      |
      v
Post / statistics / convergence / results

Orthogonal control plane:
Deterministic Stream / CRN / Checkpoint / Replay / Golden Reference

```

2A. Three points readers should confirm first

When reading V1 performance numbers, first keep the following three points in mind.

(1) The processing scope is different. Published representative numbers for conventional RNGs often describe only the speed of generating raw random output. V1's 46.602 Gsamples/s is **end-to-end logical MC throughput**, including Back/Seed, Front RNG, Normal transformation, 4-way fanout, PaperView/Final-Cultivation, and the MC consumer.

(2) Even so, the reported rate is large in numerical scale. Expressed on the same G/s display scale, the fastest published raw-RNG reference used here, xoshiro256+, is about 1.639 G 64-bit outputs/s, while V1 is 46.602 G logical MC samples/s. The numerical scale is therefore about 28.4x. However, the units and processing scopes differ, so this is **not** a claim that V1 is "28.4x faster" on the same workload.

(3) The measurement is not a bare-metal ceiling. 46.602 Gsamples/s was measured on an AMD EPYC 9V74 virtualized 3-vCPU environment with one logical CPU pinned. A 20-run confirmation showed a 40.166-45.725 Gsamples/s spread, indicating virtualization, scheduling, and frequency variation. Because a dedicated bare-metal value has not yet been measured, any improvement headroom is described only as possible unmeasured room rather than as a fabricated numerical bound.

2B. Two-layer quality evaluation - do not confuse standard RNG tests with MC validation

V1 quality evaluation has two layers answering different questions.

Layer 1: RNG-level statistical qualification

```

raw bitstream / integer stream
-> PractRand
-> TestU01 (SmallCrush / Crush / BigCrush as applicable)
-> supplementary surrogate battery

```

Layer 2: Application-level Monte Carlo validation

```

Normal transformation
-> Var / corr / dep2
-> E[R^2] / Var(R^2) / hit
-> multi-stream correlation
-> convergence
-> acceleration-option BASELINE comparison
-> anti-reuse / sample-accounting audit

```

Layer 2 does not theoretically replace Layer 1. It does, however, directly address the application-level question that Layer 1 alone cannot: whether desirable statistical behavior remains after the data are actually passed through the MC pipeline.

PractRand provides a family of statistical tests for large streams, while TestU01 provides empirical statistical tests and standard batteries for uniform RNGs. V1 explicitly treats these standard suites as future independent qualification tasks and does not hide the fact that they have not yet been run. [PractRand][TestU01] Named suites are themselves empirical validation; passing or failing them does not prove mathematical IID behavior or complete independence.

2C. Document authority rule - separate the Final Specification from development history

This book preserves development history, but fixes the following priority order so that readers do not mistake historical states for the current V1 specification.

1. **Frozen V1 source identity / exact implementation ledger** is the highest-level implementation authority.
2. **Final Architecture / Final Results** describe the current V1 specification and its measurements.
3. **Ablation / development history / failed variants** explain design decisions and do not change the current specification.
4. Measurements are observations obtained under stated conditions; they are not specification constants or universal performance guarantees.
5. If V1 is changed in the future, a new source identity / revision must be assigned rather than overwriting the existing history.

Accordingly, old bias values and experimental trials appearing in this book do not mean that current V1 is in that state. They remain to document how problems were found and why fixes were made.

2D. Meaning of the 3-core experiment - "Why measure 3 cores?"

The 3-core benchmark was not intended to increase V1's best one-core speed. Its purpose was to experimentally determine whether **aggregate MC throughput rises with the number of independent workers when V1 is replicated as an independent worker across multiple cores**.

In the dedicated parallel-scaling experiment on AMD EPYC 9V74, three independent workers were run using the same one-worker condition as a baseline, and the clean full-active subset produced:

```
1 worker = 12.7485 Gsamples/s
3 workers = 38.656 Gsamples/s
scaling = 38.656 / 12.7485 = 3.032x
```

This is empirical evidence of **worker-level scalability** and is the central result of the 3-core benchmark.

The important point is that 12.7485 Gsamples/s is not "V1's current highest one-core performance." It is the **local one-worker baseline used to measure the scaling ratio under the same AMD conditions**. Under a different condition, V1's strict one-core-complete peak was observed at 46.602 Gsamples/s. These numbers must not be interpreted as one time series.

Thus, the 3-core experiment connects the following two pieces of information:

```
46.602 Gsamples/s
|
| <- one-core complete peak: absolute observed peak reference
|
+-----+
|         | separate scaling experiment
|         |
|         | v
|         | 12.7485 G/s x 3 workers
|         |
|         | v
```

38.656 G/s aggregate
 |
 v
 3.032x

The first is an observed one-core peak reference; the second is a scaling measurement showing that multiple workers can be used. They are not the same kind of number.

2E. 192-core expansion is a “peak-envelope thought experiment,” not a prediction

Taking the best measured V1 one-core-complete peak of 46.602 Gsamples/s, perfect linear scaling to 192 workers would give:

46.602 Gsamples/s x 192
 = 8,947.584 Gsamples/s
 = 8.947584 T logical samples/s
 ~ 8.948 T logical samples/s

Here, 192 is a candidate future worker count for a high-core-count CPU class such as the 5th Gen AMD EPYC 9965, whose public specifications list 192 cores / 384 threads. [AMD EPYC 9965 public specifications]

This 8.948 T value simultaneously assumes:

1. the measured 46.602 Gsamples/s one-core peak is the starting point;
2. every worker maintains the same peak;
3. throughput scales perfectly linearly to 192 workers.

Therefore it is an **idealized peak envelope**.

Real systems may fall below this envelope because of NUMA, memory hierarchy, frequency behavior, operating-system scheduling, worker placement, system-wide bandwidth, or other shared infrastructure effects. This book does not present 8.948 T logical samples/s as a prediction, guarantee, or 192-core measurement.

The role of the figure is to show, conditionally, what logical throughput region would correspond to a much larger worker count if the measured one-core peak and observed scaling model continued without loss.

2F. GPU comparison is treated as a future-research hypothesis

CPU-resident and GPU execution cannot be compared solely by samples/s. A future comparison should measure, under the same Monte Carlo work definition, at least:

throughput
 latency / time-to-first-result
 host-device transfer volume
 memory traffic
 energy per logical sample
 energy-to-solution
 software deployment complexity

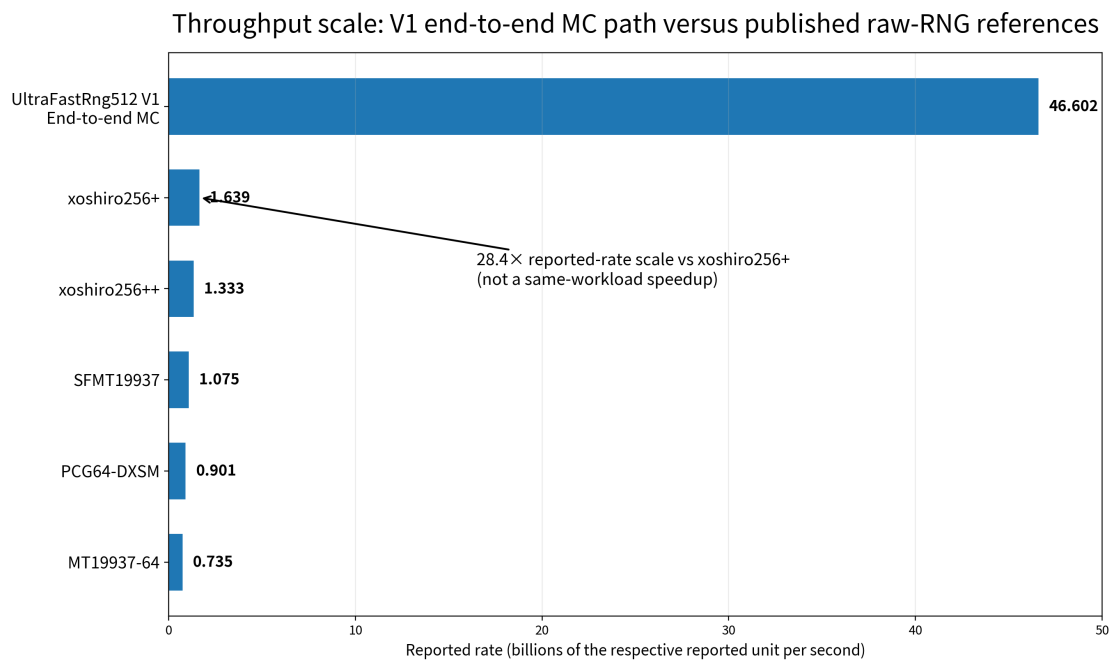
In workloads dominated by host-device movement, kernel launch, PCIe/interconnect, or device-memory management, CPU-resident and GPU execution may show very different performance characteristics. These effects have not been measured here.

Therefore, the possibility that **CPU-only V1 could exhibit lower latency or higher energy efficiency than a particular high-performance GPU configuration** is a future measurement hypothesis, not an established performance result. This book does not claim that V1 is faster, more power-efficient, or lower-TCO than GPUs.

2G. Reproducibility snapshot - what can and cannot currently be fixed

The current headline benchmark was obtained on **AMD EPYC 9V74 / visible 3 vCPUs / one logical CPU pinned / GCC / -O3 -std=c11 -pthread -march=native / C/C++ compiler intrinsics / FIXED generation / shared VM**. The documented core path does not require hand-written inline assembly.

The exact GCC major/minor version is not preserved in the current benchmark record, and the bare-metal result is not measured. No unsupported value is filled in by assumption. For a formal reproduction, preserve `gcc --version`, CPU model, kernel, VM/container image, source hash, exact command line, and cycle-counter information together. Because `-march=native` is host-dependent, the exact ISA target must also be recorded for formal comparisons.



Published references: 64-bit raw-output rates converted from ns/output. V1: logical MC samples/s. Different units and environments; use for scale/context, not as a same-workload benchmark.

Figure 00A. Reported throughput scale. The bar lengths are intentionally shown on a common numerical G/s axis to make the scale visible; the underlying units and workload scopes remain different.

Measurement environment: a cloud/virtualized result, not a bare-metal ceiling

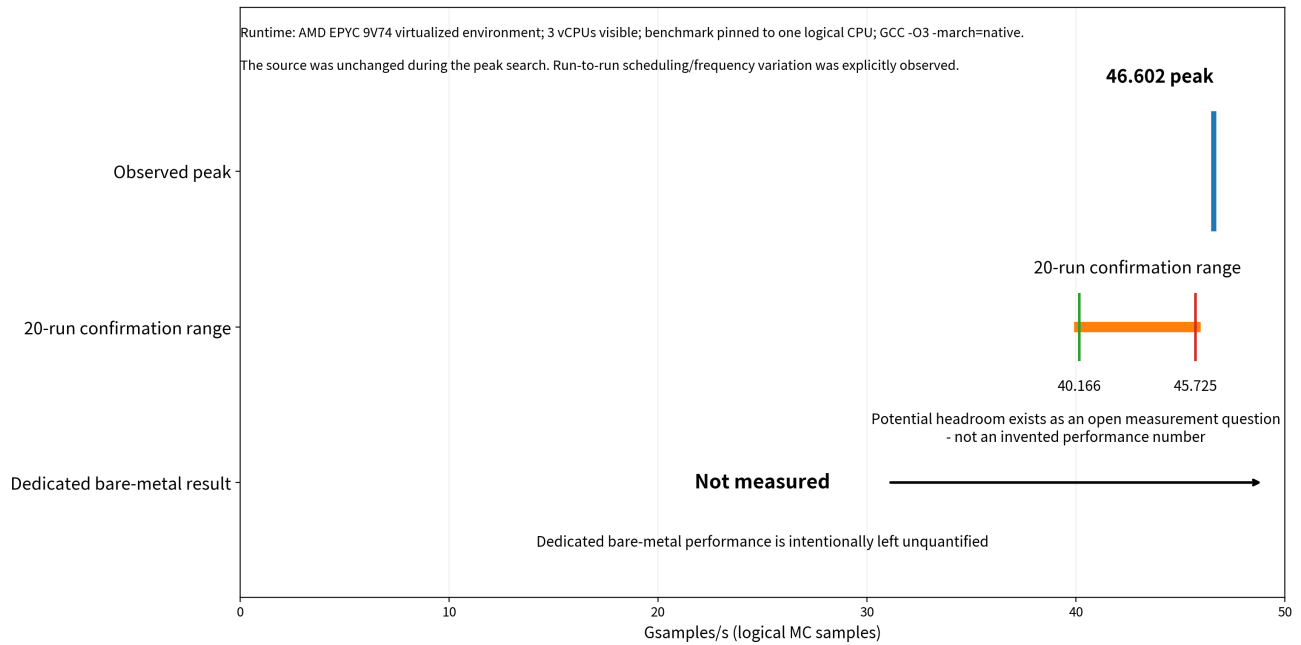


Figure 00B. Measurement-environment context. The observed VM result is a measured point, not a claimed bare-metal ceiling.

Part II - Design principles that should be understood first

3. Why separate Back and Front?

The most important design principle of V1 is to **separate heavy processing for quality from lightweight processing for high-speed execution**.

The Back side contains, among other things:

- entropy conditioning
- LPS/PSL2(F29)
- Seed Bank
- ChaCha20-AAA
- VHV
- preparation of multiple parent seeds
- fingerprint / reproducibility metadata
- A/B seed exchange

The Front side receives completed state and performs the continuous computation.

Therefore the Front hot path must not contain:

- SHA
- LPS
- Seed Bank expansion
- heavy profile generation
- metadata processing
- Checkpoint processing

Architectural meaning

Back is the **“quality construction” department**.

Front is the **“receive the quality and run it at scale” department**.

This separation allows Front performance measurement and Back quality design to be discussed independently.

4. Why is one-core-complete the baseline?

V1’s baseline design keeps Front + Back completable on one CPU core.

Moving Back to an additional CPU core is an optional experiment or extension, not the baseline condition.

This is not a design in which performance is obtained by “escaping Back to another core.” Instead, the first requirement is:

the complete engine must remain viable even when contained within one core.

In measurements comparing one-core-complete execution against Back-split execution, the difference was small; one recorded one-core median was about 0.57% higher than the split case. Therefore V1 must not be described as obtaining its performance by delegating Back to other cores.

Part III - Seed / Back

5. Entropy conditioning

Input is preprocessed in 64-byte blocks.

The current cleanup:

- removes an all-zero block;
- suppresses an immediate repetition of a block that is exactly identical to the previous 64-byte block.

After that, the conditioned value is computed as:

$C = \text{SHA256}(\text{"UFRNG-ENTROPY-CONDITIONER-V2"} \parallel R \parallel \text{NodeID} \parallel \text{RunID} \parallel \text{BE64}(\text{Sequence}))$

The important point is that this is not a mathematical proof of entropy rate.

Its purpose is to turn the input into fixed-length, well-defined, distinguishable conditioned material that can be passed stably into later stages.

6. LPS / PSL2(F29)

V1 performs an LPS walk using four domain-separated lanes.

Fixed parameters:

$p = 5$
 $q = 29$
 $\text{degree} = 6$
 $\text{lanes} = 4$
 $\text{start accepted symbols} = 8$
 $\text{walk accepted symbols} = 16$

Generators:

$G_0 = [[10, 0], [0, 3]]$
 $G_1 = [[8, 13], [16, 8]]$
 $G_2 = [[8, 11], [11, 8]]$
 $G_3 = [[8, 18], [18, 8]]$
 $G_4 = [[8, 16], [13, 8]]$
 $G_5 = [[3, 0], [0, 10]]$

3-bit selector values 0..5 are accepted; 6 and 7 are rejected.

The matrix walk advances as:

$X_{(i+1)} = \text{canon}(X_i * G_s \bmod 29)$

The four lane endpoints and associated fields are finally fed into SHA-512 to obtain a 512-bit Root Seed.

Why four lanes?

Four independently domain-separated streams are used to create fixed-length seed material with the intended diversity before later stages.

The important qualification is that LPS itself is not a claim of complete mathematical assurance for the PRNG. LPS is one part of V1 seed preparation; no formal cryptographic-security or entropy-extraction theorem is claimed here.

7. Seed Bank

From the Root Seed and bank index:

```
Bank_b = SHA512("UFRNG-SEED-BANK-V1" || Root || BE64(b))
```

The Bank entry is 512 bits.

The bank index allows reproducible generation of distinct seed-bank entries from the same root.

8. CHACHA20-AAA-A1

V1 seed profile.

14 double-rounds = 28 total rounds

injection round = 9

injection word = 8

injection = $+0x3B54CDA5 \bmod 2^{32}$

Key:

```
K = SHA256("UFRNG-PROFILE-CHACHA20-KEY-V1" || Bank_b)
```

Nonce material:

```
M = SHA256("UFRNG-PROFILE-CHACHA20-NONCE-V1" || Bank_b || BE64(Sequence))
```

```
Nonce = M[0:12]
```

The first 32 bytes of block 0 and the first 32 bytes of block 1 are concatenated to form the 64-byte profile seed.

The important property of AAA is that it is **not executed on every Front iteration**.

AAA is Back-side quality shaping.

Therefore, the simplistic interpretation that removing AAA automatically shortens the Front hot loop is wrong: AAA is on the seed-generation side.

Part IV - VHV / 3 Parents / Front

9. V -> H -> V

First create 16 lanes of V1:

```
V1_i = SHA512("UFRNG-FRONT16-SHA512-V1" || ProfiledSeed || BE64(bank) ||  
BE64(generation) || BE32(i))
```

Concatenate all 16 values to form the global H:

```
H = SHA512("UFRNG-FRONT16-VHV-HASH-V1" || V1_0 || ... || V1_15 || BE64(bank) ||  
BE64(generation))
```

Then return H to each lane:

```
V2_i = SHA512("UFRNG-FRONT16-VHV-COMPACT-V1" || V1_i || H || BE32(i))
```

This is the VHV structure used by V1.

Why VHV?

The construction does not merely strengthen each lane independently. It first aggregates all lanes, then re-injects the global hash into each lane.

Thus each lane follows:

local identity -> global coupling -> local rematerialization

10. 3 Parents

V1 uses three same-profile VHV lineages: A, B, and C.

For generation g :

$pi(g) = \text{floor}(g/2) \bmod 6$

and the six parent-order permutations are:

0: A B C
1: A C B
2: B A C
3: B C A
4: C A B
5: C B A

Why three parents?

Rather than massively increasing the number of parent states, V1 prepares different seed lineages on the **Back side**, then compresses them into a small number of parents on the Front side.

This makes it possible to bring multiple lineages into Front while keeping the Front state budget fixed at 16.

11. 16-resident-state budget

V1 keeps 16 x 64-byte states resident in Front.

3 parent states
13 auxiliary states

16 states total

Total resident state = 1024 bytes.

This fixed budget is important.

If the number of states were increased without bound, any observed quality change would be confounded by a different problem: L1/register pressure and execution-resource demand.

V1 therefore treats the **16-state budget** as an architectural boundary.

12. Front recurrence

Let the current parent states be a , b , and c .

The mode is:

$\text{mode} = \text{floor}(\text{generation}/2) \bmod 3$

Mode 0

$p0 = a \text{ XOR } \text{ROTL64}(b, 17)$
 $p1 = b + \text{ROTL64}(c, 29)$
 $p2 = c + \text{ROTL64}(a, 41)$

Mode 1

```
p0 = a + ROTL64(b,17)
p1 = b XOR ROTL64(c,29)
p2 = c + ROTL64(a,41)
```

Mode 2

```
p0 = a + ROTL64(b,17)
p1 = b + ROTL64(c,29)
p2 = c XOR ROTL64(a,41)
```

The same three parent relations rotate the roles of ADD/XOR as the generation changes.

13. 13 auxiliary states

The 13 states other than the three parents are not all updated on every step. Instead they are updated sequentially under a 13-phase schedule.

```
phase 0 -> m3
phase 1 -> m4
phase 2 -> m5
phase 3 -> m6
phase 4 -> m7
phase 5 -> m8
phase 6 -> m9
phase 7 -> m10
phase 8 -> m11
phase 9 -> m12
phase 10 -> m13
phase 11 -> m14
phase 12 -> m15
```

For example:

```
m3 <- (m3 + ROTL64(p0,7)) XOR ROTL64(m3,13)
m11 <- (m11 + ROTL64(p2,11)) XOR ROTL64(m11,17)
m15 <- (m15 + ROTL64(p2,43)) XOR ROTL64(m15,47)
```

All 13 exact equations are fixed in Mathematical Specification v0.2.

14. The 16 Front outputs

The output order for one micro-step is fixed:

```
p0
p1
p2
p0 XOR m3
p1 XOR m7
p2 + ROTL64(m11,1)
p0 XOR m4
p1 XOR m8
p2 XOR m12
p0 XOR m5
p1 XOR m9
p2 XOR m13
```

```
p0 XOR m6
p1 XOR m10
p2 XOR m14
p1 XOR m15
```

This produces 16 vectors per micro-step.

Each vector has 16 lanes, so:

16 vectors x 16 lanes = 256 values

Because each Normal vector is then expanded through 4-way fanout, the internal Normal-domain logical geometry is:

16 x 4 x 16 = 1024 logical samples / micro-step

Part V - Normal v9

15. What does Normal do?

V1's Normal stage is not Box-Muller.

It is also not a conventional floating-point inverse-CDF implementation.

It uses a 128-byte integer LUT together with AVX-512 integer dot-product-style instructions.

One ZMM input:

$x = 16 \times \text{uint32}$

Transform:

```
a  = VPERMI2B / VPERMUTEX2VAR_EPI8 style lookup
p16 = VPMADDUBSW(a,x)
q   = VPMADDWD(p16,1)
```

The result q is a quantized Normal representation; it is not yet a literal $N(0,1)$ value.

A fixed downstream scale is applied:

`UF_NORMAL_SCALE = 0.0009317057574691516f * 1.0007781f`

16. Why an LUT?

Classical representative approaches to Normal generation include Box-Muller and Ziggurat.

Ziggurat is long established as a table-based fast Normal-generation method. V1's Normal v9 instead aims to **fold the Normal approximation itself into an existing AVX-512 byte-permute + integer dot-product instruction sequence**.

The defensible V1 claim is that this implementation chooses that execution structure. It must not be described as historically the first Normal-generation algorithm of its kind.

Part VI - 4-way Fanout

17. Fanout

Two shuffles are formed from q:

```
y0 = SHUFFLE(q, _MM_SHUFFLE(2,3,0,1))
y1 = SHUFFLE(q, _MM_SHUFFLE(1,0,3,2))
```

Then four sequences are emitted:

```
q + y0
q - y0
q + y1
q - y1
```

Caution

These four sequences must not be interpreted as “four times as many independent random numbers.”

They are derived from the same *q*, so structural relationships remain inside the construction.

This observation leads directly to the root cause discussed later for the sign/dep2 issue.

Part VII - PaperView / cultivation layer

18. What is PaperView?

The logical representation of standard PaperView is:

1024 rows x 16 lanes = 16,384 logical samples = 64 KiB (int32 view)

On the other hand, the `retained_starter` held at the generation boundary is: 1024 rows x 16 lanes x int16 = 32 KiB, with an additional 1024 x uint16 = 2 KiB `tail_mask`.

Therefore, 64 KiB is the **logical size of the PaperView**, not a statement that an int32 64 KiB starter is permanently retained after Front.

The cultivation layer is not a new random generator.

It takes already-generated Normal-domain data and rearranges it according to:

- view
- row
- lane
- epoch
- phase
- tail rule

so that the MC consumer receives a more suitable data layout.

19. epoch -> phase / shift / view

The current V1 source function `ufr_paper_cultivation_shift()` takes the lower 10 bits of the epoch, applies a 10-bit bit reversal, then extracts the upper 4 bits of that value as `shift`.

Definition:

$$x_0 = epoch \& 1023$$

$$x_1 = BR10(x_0)$$

$$shift = x_1 \gg 6$$

Therefore the actual range of `shift` is 0..15, and **shift = BR10(epoch) itself is not correct**. This is a bit-for-bit reproducibility detail.

Then view/row locations are determined by:

```

phase = (epoch >> 10) & 63
vid    = (view_id + phase + (phase >> 3)) & 31
base   = (A64_OFFSET[vid] + shift) & 1023

```

This re-indexing changes access locations as a function of epoch instead of reading the same starter in the same order.

20. Why is the cultivation layer necessary?

This point is especially important for understanding V1.

Front's job is to create state rapidly.

But if the MC side pairs data in the same order in which the data were generated, relationships that were not intentionally introduced upstream can appear in the MC layer.

The cultivation layer therefore:

controls the physical arrangement relationship delivered to MC while avoiding a large change to the marginal distribution of the generated values themselves.

Part VIII - 465 / 555

21. Generic final-cache stride = 465

The V1 generic final cache uses:

$$P(r) = 465r \bmod 1024$$

Because 465 is coprime with 1024, this is a bijection over all 1024 rows.

The important point is not to make the MC consumer perform this computation repeatedly.

cache build:
 physical row permutation

MC hot loop:
 sequential read

Complexity is therefore paid up front during cache preparation.

22. R2-only stride = 555

The R2 path uses a separate pair geometry:

$$P_{555}(r) = 555r \bmod 1024.$$

Because 555 is coprime with 1024, P_{555} is a bijection over 1024 rows.

For pair index $p = 0, \dots, 511$ and parity $\pi \in \{0, 1\}$, the actual source rows are:

$$r_0 = P_{555}(2p + \pi), \quad r_1 = P_{555}(2p + \pi + 1).$$

Thus, the correct interpretation is not “apply 555 once to the logical pair index”. Instead, **the 555 permutation is applied to each of the two logical rows composing the pair**. At runtime the consumer reads sequentially from the materialized pair bank.

Important

465 and 555 must not be treated as one generic number.

465 = generic final-cache geometry

555 = R2 / pair-geometry repair

Part IX - Root-cause discovery and correction

23. The old problem

In the previous integrated path, adjacent MC pairs could become internally coupled rows derived from the same q, such as:

(q+y, q-y)

As a result, a strong bias appeared:

sign $\approx$ 0.456

dep2 $\approx$ +0.060

In contrast, the upstream AAA direct reference showed:

sign $\approx$ 0.500

dep2 $\approx$ +0.0017

This was around the reported region.

This indicates that the problem was not the standalone Front output, but rather **which physical

* * *

24. Correction

The selected correction was to change the physical row mapping of the final cache to:

``P(r)=555r mod 1024``

The MC side continues to use sequential reads as before.

The meaning of this correction is:

```
```text
coupled row created from the same source q
 |
 v
physical cache permutation
 |
 v
sequential consumer pair
 |
 v
different source rows can be paired
```

As a result, the large integrated audit improved the measured values to approximately:

sign 0.456146... -> 0.498794...

dep2 0.060467... -> 0.001687...

This is one of the most important quality-correction experiments in V1.

---

## Part X - MC Execution Layer

### 25. Direct / L1 / L2

V1 treats data transport into MC as multiple transport paths.

DIRECT  
L1 CACHE  
L2 CACHE

These are caller-selected options rather than silent automatic switching.

For larger data sets, L2 staging can be advantageous in some cases, but this is workload-dependent. The project does not claim that “L2 is always the fastest.”

---

### 26. Final profiles

Current surface:

AUTO  
RAW\_INT16  
INT32  
FLOAT32  
R2\_FLOAT32  
CUSTOM

#### **RAW\_INT16**

An exact baseline format with reduced memory footprint.

#### **INT32**

Handles q directly as 32-bit values.

#### **FLOAT32**

A path designed to exploit cache reuse, particularly for long fixed generations.

#### **R2\_FLOAT32**

A specialized path that moves pair geometry into a dedicated cache and processes it with eight accumulators.

#### **CUSTOM**

Allows a user-defined final-row format to be registered.

---

## 27. Meaning of R2

In a representative MC test, a Normal pair  $(x, y)$  is mapped to:

$$R^2 = x^2 + y^2$$

and the hit condition is:

$$\text{hit} = [R^2 \leq 1]$$

For an independent  $N(0,1)$  pair:

$$P(R^2 \leq 1) = 1 - \exp(-1/2) = 0.393469340287 \dots$$

with reference moments:

$$E[R^2] = 2$$

$$\text{Var}(R^2) = 4$$

This R2 path is not merely a benchmark. It is also a highly sensitive diagnostic for pair-geometry quality.

---

## Part XI - Principles of Optimization

### 28. Basic optimization principles adopted in V1

#### 1. Push heavy work out of the hot loop

Seed hashing, LPS, ChaCha, and VHV are performed on the Back side.

#### 2. Prepay memory transformations

Row permutations and similar transformations are performed during cache construction.

#### 3. Read sequentially on the consumer side

Do not add complicated index formulas to the MC loop.

#### 4. Keep the state budget fixed

16 x 64-byte resident states.

#### 5. Preserve vector width to the end

Process Normal, fanout, and R2 accumulators with AVX-512.

#### 6. Put option selection at setup time

Avoid per-sample dispatch.

---

## Part XII - Current Optimization Option Surface

### 29. V8/V9 optimization profiles

Current option surface:

BASELINE  
VEC8  
PARAM8\_SHARED  
PARAM4\_PRECOMPUTE  
PARAM8\_PRECOMPUTE  
PARAM4\_PRECOMPUTE\_INVARIANT  
PATH8\_BLOCK  
PATH4X2  
PATH4  
R2\_CACHE8

These are not “magic optimizations that are all enabled simultaneously.”

The implementation evaluates workload capabilities and selects one execution plan at setup time.

---

### 30. Finance / Pharma

Finance and Pharma are domain adapters outside the common q256 boundary.

---

## Part XII-A - MC Optimization Techniques - Structure, Mechanism, and Measured Results

This section restores the concrete MC optimization mechanisms and measured values that were overly simplified in the previous edition. Its purpose is not to exaggerate speed, but to let readers trace what was optimized and what performance was actually measured for **R2, FLOAT32, INT32, Finance, and Pharma**.

The numbers in this section include benchmark records from multiple generations and multiple environments. They must therefore not be treated as a single same-host ranking; each measurement condition and scope must be shown alongside the number.

### 30A. The overall picture first

UltraFastRng512 MC acceleration does not simply replace instructions with allegedly faster instructions.

The basic idea is:

```
RNG / Normal
-> PaperView / cultivation
-> prepay computable common work
-> share the same computation across multiple parameters
-> reduce dependencies / branches / conversions in the hot loop
-> process with SIMD + multiple accumulators
-> MC result
```

In other words, the center of optimization is:

**Increase the amount of work that does not need to be performed in the hot loop at all, rather than merely making each sample's arithmetic faster.**

### 30B. Five basic optimization techniques

The current MC acceleration can be organized into five main classes.

Technique	Role
A. Prepayment	Move heavy work into cache build / precompute
B. Sharing	Compute sample- or parameter-common work once
C. SIMD	Process parallelizable work in AVX-512 vectors
D. Dependency distribution	Split accumulation across multiple accumulators to reduce long dependency chains
E. Hot-loop simplification	Move modulo / branch / widen / conversion / complex index generation out of the hot loop

These five techniques appear in different forms in R2, Finance, and Pharma.

### 30C. R2-type Monte Carlo

#### 30C-1. What is being calculated?

In the representative R2 MC, a Normal pair  $(x, y)$  is conceptually processed as:

```

r2 = x2 + y2
hit = (r2 <= 1)
value = hit ? (1 - r2) : 0

```

This is comparatively light domain math.

#### 30C-2. What remained at runtime?

Without precomputation, a typical hot loop must perform:

```

int16 load
-> widen to int32
-> convert to float
-> x2
-> y2
-> x2+y2
-> compare
-> 1-r2
-> accumulate

```

for every sample.

#### 30C-3. R2 Cache - prepay the calculation

During PaperView build, calculate in advance:

```

x
y
-> float conversion
-> x2 + y2
-> r2

```

and store the result in the R2 cache.

The MC hot loop can then conceptually be reduced to:

```

cache load
-> compare
-> 1-r2
-> masked accumulate

```

The essence is therefore not that arithmetic instructions became magically faster, but that **the arithmetic was prepaid outside the consumer loop**.

### 30C-4. R2 Cache structure

One logical PaperView is:

64 KiB = 16,384 samples

The R2 cache is organized as 2 parity banks, 512 pair starts, and 16 float lanes, holding approximately 64 KiB of R2 values per view.

2 parity banks  
x 512 pair-starts  
x 16 float lanes  
= 16,384 float values  
= 64 KiB

The default chunk has four views, so the logical R2-cache capacity is approximately 256 KiB.

The **logical view size** of PaperView is not the same thing as the physical starter representation. The current V1 physical starter is primarily an int16 32-KiB-class representation accompanied by tail-mask metadata. Do not confuse this with the materialized R2 cache values.

### 30C-5. 8-way accumulator

A single accumulator creates a long dependency chain:

acc -> acc -> acc -> acc -> ...

Distribute the work across eight accumulators:

acc0  
acc1  
acc2  
acc3  
acc4  
acc5  
acc6  
acc7

and alternate sample blocks:

block 0 -> acc0  
block 1 -> acc1  
block 2 -> acc2  
...  
block 7 -> acc7

This makes it easier to hide accumulation latency.

### 30C-6. Push wrapping out of the hot loop

When reading 1024 rows as a ring, performing

if end -> wrap

every iteration increases index-generation work and branching. Instead, split the ring into two linear regions:

```
[base ... end]
 +
[start ...]
```

This removes the wrap test from the inner loop and **simplifies the loop while preserving the same amount of work.**

## 30D. Measured R2 performance

### 30D-1. Intel Xeon Platinum 8573C - 1B-class historical reference

A historical 1B-class representative measurement recorded these medians for R2 / FLOAT32 / INT32:

Path	Median throughput
R2 g64	<b>33.065 Gsamples/s</b>
FLOAT32	<b>19.129 Gsamples/s</b>
INT32	<b>16.359 Gsamples/s</b>

The main reason for the R2 result is not that the R2 mathematics itself is intrinsically faster, but that **very little computation remains at runtime.**

This result belongs to a different generation and environment from the current AMD EPYC 9V74 VM head-line peak and is not promoted as the current standard V1 speed value.

### 30D-2. V32 historical specialized path

The historical V32 experiment combined:

```
8 independent accumulators
masked add
wrap split
sample-counter hoisting
distributed hit counters
```

The recorded values were:

Variant	Median	Best
V30 baseline	18.811 Gsamples/s	20.924 Gsamples/s
V32 specialized	29.838 Gsamples/s	31.954 Gsamples/s

This history showed median +58.6% and best-vs-best +52.7%, but it is a **historical specialized optimization experiment** and is not used for the current V1 standard speed claim.

What the experiment does establish is that in MC workloads, **dependencies, loop structure, counter placement, and wrap handling can materially affect throughput**, independently of the domain mathematics itself.

## 30E. Finance Monte Carlo

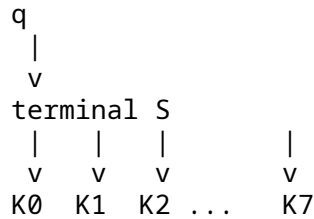
Finance has higher computational density than R2. A representative path conceptually handles:

```
x = q * factor + drift
S = spot * exp_approx(x)
payoff[k] = max(S - strike[k], 0)
```

with eight strikes.

A direct implementation can easily recompute the same terminal value for every strike.

### 30E-1. Compute the common terminal once



Because terminal  $S$  is common to all eight strikes, **compute it once and share it across the eight outputs instead of recomputing it eight times.**

### 30E-2. Finance precompute

Fixed factors such as:

```
q * UF_NORMAL_SCALE
vol * sqrt(T)
```

can be assembled in advance to reduce repeated runtime multiplication. This is the basic idea behind the PARAM8\_PRECOMPUTE family.

### 30E-3. PARAM4\_PRECOMPUTE - 4+4 consumption

Instead of keeping all eight strike-dependent states live, use a two-pass 4+4 structure:

```
terminal S
-> materialize to PaperView
-> strikes 0..3
-> reuse PaperView
-> strikes 4..7
```

The goals are simultaneous reduction of:

```
terminal-computation duplication
+ register pressure
+ live accumulator count
```

while reusing PaperView materialization.

### 30E-4. Finance measurements - historical engineering references

Representative 1B-class measurements include:

Finance option	Median throughput
PARAM8_SHARED	<b>0.937 Gsamples/s</b>
PARAM8_PRECOMPUTE	<b>1.011 Gsamples/s</b>
PARAM4_PRECOMPUTE	<b>1.239 Gsamples/s</b>
PARAM4_PRECOMPUTE_INVARIANT	<b>1.204 Gsamples/s</b>
AUTO	<b>1.239 Gsamples/s</b>

A separate long-run measurement recorded:

PARAM4\_PRECOMPUTE = 1.410 Gsamples/s  
 AUTO = 1.376 Gsamples/s

Thus, across this engineering benchmark family, Finance was observed roughly in the **1.2-1.4 Gsamples/s range**.

A current integrated three-run tour also recorded PARAM4\_PRECOMPUTE = 1.204 Gsamples/s, PARAM4\_PRECOMPUTE\_INVARIANT = 1.237 Gsamples/s, and AUTO = 1.234 Gsamples/s. These are observation-level differences including runtime conditions and should not be collapsed into one fixed number.

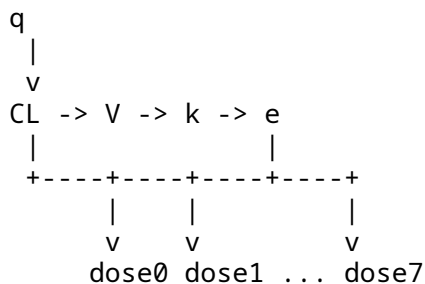
## 30F. Pharma Monte Carlo

Pharma is heavier still. A representative path handles expressions such as:

```
CL = clearance * exp_approx(qcl * factor)
V = volume * exp_approx(qv * factor)
k = CL / V
e = exp_approx(k * (-t)) / V
result += dose[k] * e
```

Depending on the implementation path, roughly three exp\_approx operations and two vector-divide-level operations can remain as common work, so its computational density differs greatly from R2.

### 30F-1. Share exposure



Rather than computing the common exposure  $e$  repeatedly for eight doses, **compute exposure once and share it across the eight doses**.

### 30F-2. PARAM4\_PRECOMPUTE\_INVARIANT

```
common exposure
 -> 4 partial accumulators
 -> later expansion to 8 dose results
```

This avoids duplicate common computation and reduces eight live states and register pressure.

### 30F-3. Pharma measurements - historical engineering references

Representative 1B-class measurements recorded:

Pharma option	Median throughput
PARAM4_PRECOMPUTE_INVARIANT	<b>0.704 Gsamples/s</b>
PARAM4_PRECOMPUTE	<b>0.520 Gsamples/s</b>
AUTO	<b>0.720 Gsamples/s</b>

Thus Pharma was observed at roughly the **0.7 Gsamples/s scale** in this measurement family.

The current integrated three-run tour recorded PARAM4\_PRECOMPUTE = 0.533 Gsamples/s, PARAM8\_PRECOMPUTE = 0.539 Gsamples/s, PARAM4\_PRECOMPUTE\_INVARIANT = 0.916 Gsamples/s, and AUTO = 0.921 Gsamples/s. This does not imply one universal fixed Pharma value; it shows that **the precise option path, generation/consumer condition, and runtime environment affect the observed number.**

### 30G. Why does R2 look so much faster?

This is important when explaining V1 to others.

R2

```
load
compare
1-r2
accumulate
```

Float32

```
load
x2+y2
compare
accumulate
```

INT32

```
load
int->float conversion
x2+y2
compare
accumulate
```

Finance

```
q -> terminal
exp_approx
8 strike payoff
```

Pharma

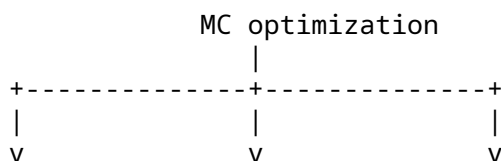
```
3x exp_approx-class work
2x vector-divide-class work
8 dose-related updates
```

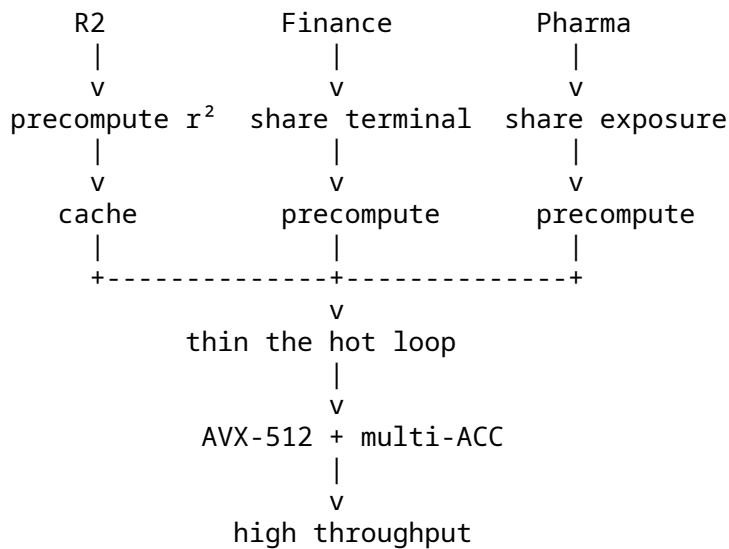
Hence the observed scale difference:

R2	~33 G/s-class (historical Xeon reference)
Float32	~19 G/s-class
INT32	~16 G/s-class
Finance	~1.2-1.4 G/s-class
Pharma	~0.7 G/s-class

This does not mean that one algorithm is intrinsically “better” than another. It means **different amounts of computation remain in the runtime loop per logical sample.**

### 30H. Shared optimization philosophy





Although the concrete optimizations differ by MC, the underlying philosophy is the same:

**Compute shared work once, then run only the repeated part through SIMD at high speed.**

### 30I. Rules for optimizing without destroying quality

Pursuing speed alone can damage pair geometry or sample relations. In V1 development, a localized bias in R2 pair construction was actually observed.

Historically, dep2 deteriorated into the 1e-2 range under one pair-generation route. A dedicated R2 stride-555 permutation was introduced to repair the pair geometry. The principal purpose of that change was **quality recovery**, not a claim of speedup.

The quality correction is therefore pushed into cache construction:

```

R2 pair source
-> stride 555 permutation
-> R2 cache build

```

rather than complicating the MC inner loop.

### 30J. Judge quality correction and speed separately

The development history includes an observation around R2 correction in which:

```

Baseline median = 28.247 Gsamples/s
Candidate median = 30.500 Gsamples/s

```

was observed. However, after considering run-to-run variation, this was not called a speed improvement; it was treated as **"no performance regression was measurable."**

This is a key V1 rule:

quality improvement != speed improvement

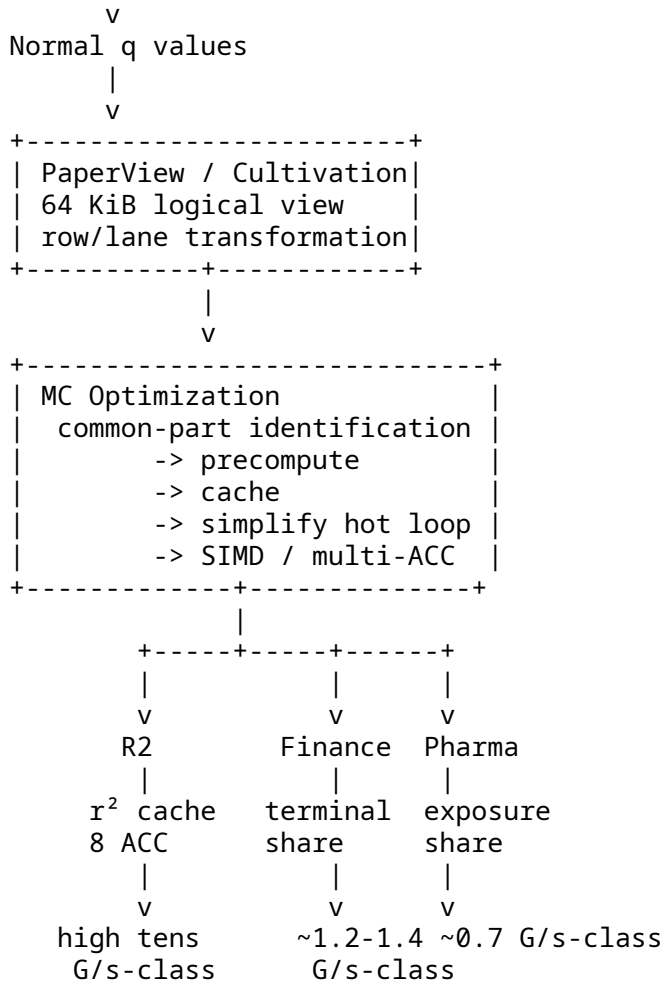
Performance acceptance and quality acceptance are separate experimental axes.

### 30K. Integrated MC optimization diagram - reader edition

```

FRONT / NORMAL
|

```



### 30L. Measured-value list - to prevent the impression that only R2 was measured

Workload / path	Representative measured result	Main optimization target	Notes
R2 g64	33.065 Gsamples/s median	R2 cache / 8-ACC / wrap split	Intel Xeon 8573C historical 1B-class
FLOAT32	19.129 Gsamples/s median	SIMD / direct Normal consumption	Intel Xeon 8573C historical 1B-class
INT32	16.359 Gsamples/s median	conversion / SIMD	Intel Xeon 8573C historical 1B-class
Finance PARAM4_PRECOMPUTE	1.239 Gsamples/s median	terminal sharing / precompute / 4+4	historical engineering reference
Finance long run	1.410 Gsamples/s	same family	historical longer measurement
Pharma PARAM4_PRECOMPUTE_INVARIANT	0.704 Gsamples/s median	exposure sharing / precompute	historical engineering reference
Pharma AUTO	0.720 Gsamples/s median	auto-selected path	historical engineering reference
V1 strict one-core peak	46.602 G logical MC samples/s	complete engine / MC path	AMD EPYC 9V74 shared VM; scope differs

This table does not rank these values. **Its purpose is to show that R2, generic Normal, Finance, Pharma, and end-to-end V1 each have measured targets, while preserving their different scopes.**

### 30M. Short explanation for others

UltraFastRng512 MC acceleration is not just about adding more AVX-512 instructions. First, heavy work that is common across repeated use is prepaid and retained in cache. Next, terminal values or exposure terms that are shared across multiple parameters are computed once. The remaining lightweight work is then driven by SIMD and multiple accumulators, with branches, wrapping, conversions, and complicated index generation moved out of the hot loop. In R2 the center is  $r^2$ ; in Finance it is terminal value; in Pharma it is exposure. The domain-specific details differ, but the optimization philosophy is the same.

---

## Part XII-B - Features, Quality Options, and Reproducibility Options Beyond Speed

Separate from the MC optimization profiles above, V1 contains option surfaces controlling quality, Normal sample relations, transport, reproducibility, analysis, and domain functions. This section organizes them so it is clear that **not all options are speed settings**.

### 30N. Four option families

```
UltraFastRng512
|
+-- Seed / Quality
|
+-- Normal / Sample relation
|
+-- MC function / Common Post
|
+-- Reproducibility / Control Plane
|
+-- Domain adapters
|
+-- MC Optimization Profile <- speed family described above
```

Do not collapse speed, quality, application, and reproducibility options into the same layer.

### 30O. Normal-output option - Antithetic

Normal mode generates a Normal vector from one block.

When Antithetic is enabled, it conceptually creates the sign-opposite pair:

```
z[0] ... z[n-1]
|
+--> -z[0] ... -z[n-1]
```

for use by the estimator. This does not switch to a different random engine; it **controls the sample relation presented to the MC estimator**. The current default is OFF, and only estimators that need it select it explicitly.

### 30P. Final-Cultivation / output format

The current representative Final-Cultivation profiles are:

AUTO  
RAW\_INT16  
INT32  
FLOAT32  
R2\_FLOAT32  
CUSTOM

Profile	Role
RAW_INT16	Baseline format using the quantized 16-bit representation directly
INT32	Intermediate format treating quantized values as int32
FLOAT32	Passes float32 Normal directly to MC
R2_FLOAT32	Functional format for handling R <sup>2</sup> from Normal pairs
CUSTOM	User-defined Final-Cultivation profile

CUSTOM registry supports up to 16 profiles and is designed to be locked after engine creation. Attributes such as row-local builder, READONLY / FLOAT / INT32 / INT16 / EPOCH\_SENSITIVE can describe the profile, making this an extension point that **changes output assembly without modifying the core RNG algorithm**.

### 30Q. Cultivation / Transport

The data transport choices from PaperView into MC are:

AUTO  
DIRECT  
L1\_CACHE  
L2\_CACHE

- DIRECT passes data to MC without normally inserting an intermediate cache path.
- L1\_CACHE is oriented toward L1-resident access.
- L2\_CACHE keeps larger data sets on the L2 side.
- AUTO resolves the path at setup time; in the current source-derived default resolution, this is DIRECT.

This is not an algorithm switch. It is a choice of **how the same generated data are transported and retained for MC**.

### 30R. Chunk / Generation

Chunk size defines the unit of generation, transport, and consumption. Current forms include:

CHUNK\_AUTO  
CHUNK\_CUSTOM

The default is four PaperViews; 1 PaperView = 64 KiB logical view; default chunk = 256 KiB logical unit.

Generation policy provides:

AUTO  
CHUNK  
FIXED

with configurable `min_generation / max_generation`.

This allows selection among:

one-use  
chunk-based  
fixed-interval reuse

according to the use case. This is also important for quality interpretation: FIXED reuse must not be equated with independent fresh-sample generation.

### 30S. Common Post - result-analysis functions

Post-MC analysis is separated into eight Rooms:

Statistics  
Distribution  
Validity  
Convergence  
Transform  
Advanced  
Metadata  
Pairing

Separating MC kernels from analysis allows **how a result is evaluated** to be selected independently of random-number generation.

### 30T. Distribution

Representative distribution-analysis modes include:

HISTOGRAM  
CENTROID

Histogram accepts bins / min / max, while Centroid accepts a centroid count. Current defaults are:

centroids = 128  
histogram bins = 64  
range = [-5, +5]

This does not modify the Normal generator; it is an analysis option determining **how generated results are inspected**.

### 30U. Convergence / stopping conditions

MC termination can combine:

maximum samples  
minimum samples  
TARGET SEM  
maximum elapsed time

Conceptually:

samples >= MAX  
OR  
SEM <= TARGET  
OR  
time >= TIMEOUT

This creates stopping conditions such as the following.

Therefore the engine supports not only fixed-sample-count operation but also **MC runs that stop once the estimation error has become sufficiently small.**

### 30V. Transform

Post-processing of results supports:

```
scale
offset
indicator
derived callback
```

Indicators provide:

```
>= > <= < == !=
```

for threshold events and derived quantities in the common Post layer.

### 30W. Metadata

Metadata may retain fields such as:

```
start wall-clock time
start monotonic time
RNG name
MC name
Run label
```

This is not random-number quality information. It is **operational information for tracing the conditions under which a result was generated.**

Seed identification and reproducibility information are not pushed into this layer; they are managed on the Back-side control/metadata layer.

### 30X. Advanced analysis

Common Post / Advanced provides, among other functions:

```
Run comparison
Richardson extrapolation
Sensitivity
MLMC
```

- Run comparison: handles the difference between A/B runs and difference SEM.
- Richardson: performs extrapolation-style evaluation from coarse / fine results.
- Sensitivity: evaluates derivative-like sensitivity from parameter perturbations.
- MLMC: constructs a multilevel estimate from results at each level.

These are not functions for “making the RNG faster.” They are **functions supporting comparison and estimation of the MC experiment itself.**

### 30Y. Pairing / Grouped

At the batch level:

```
PLAIN
ANTITHETIC_PAIR
GROUPED
```

can be selected. This controls how sample pairs / groups are formed. Antithetic Pair uses the positive/negative relation prepared on the Normal side for the estimator; it is a different concept from R2 internal pair geometry.

### 30Z. Seed / Profile exchange

The Seed side is conceptually organized as:

Physical Entropy

- > Light Cleanup
- > SHA-256 conditioning
- > LPS
- > SHA-512 extraction
- > 512-bit Seed Bank
- > PROFILE
- > Front Seed

PROFILE is the boundary at which the Back-side Seed diffusion method can be exchanged. Reference systems can switch among profiles such as:

CHACHA20  
AES256-CTR  
Variant-A  
Variant-B  
CHACHA20-AAA-A1

The current mainline AAA candidate is a **Seed-side quality profile**; it does not add 28 rounds of ChaCha work to the Front hot loop on every iteration.

### 30AA. AAA profile - fixed quality-side construction

AAA confines ChaCha-style diffusion to the Back side and is a quality-adjustment profile with a fixed injection:

```
rr = 9
word = 8
ADD 0x3B54CDA5
```

The basic ChaCha round budget is 14 double-rounds, i.e. roughly 28 rounds.

The important structure is:

Back

- > AAA
- > strong Seed
- > Front

and **not** adding AAA's heavy work to the normal Front generation loop.

### 30AB. VHV Seed lineage

Seed-quality design prepares multiple VHV lineages on the Back side and supplies them to multiple Front parents.

Back

- |-- VHV lineage A
  - |-- VHV lineage B
  - `-- VHV lineage C
- |

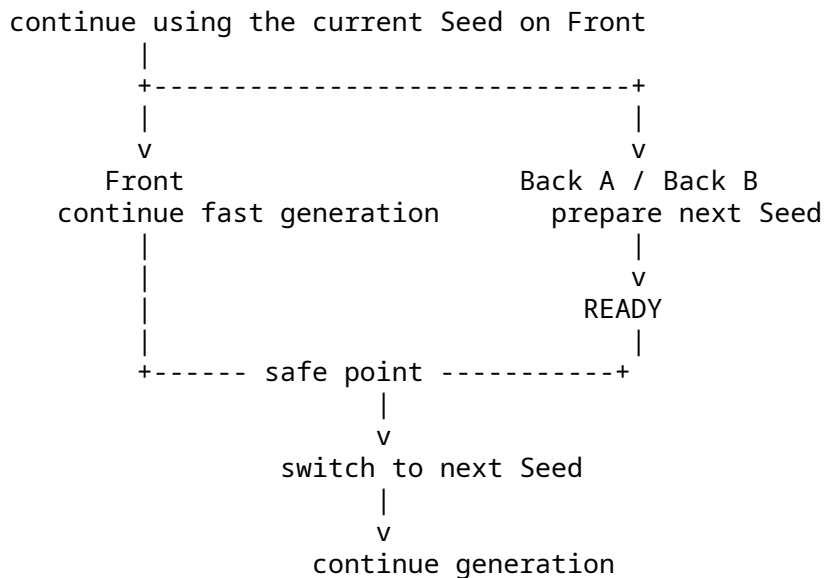
v  
Front16

This is not intended to complicate Front live state. It is intended to **prepare several strongly diffused Seed lineages on Back and then feed them into the existing Front.**

### 30AC. Dynamic Seed Exchange - core mechanism for rotating Seed generations without stopping Front

Dynamic Seed Exchange is not merely an optimization option. It is the **core architecture that separates heavy Seed preparation from high-speed Front execution and safely rotates Seed generations without stopping Front.**

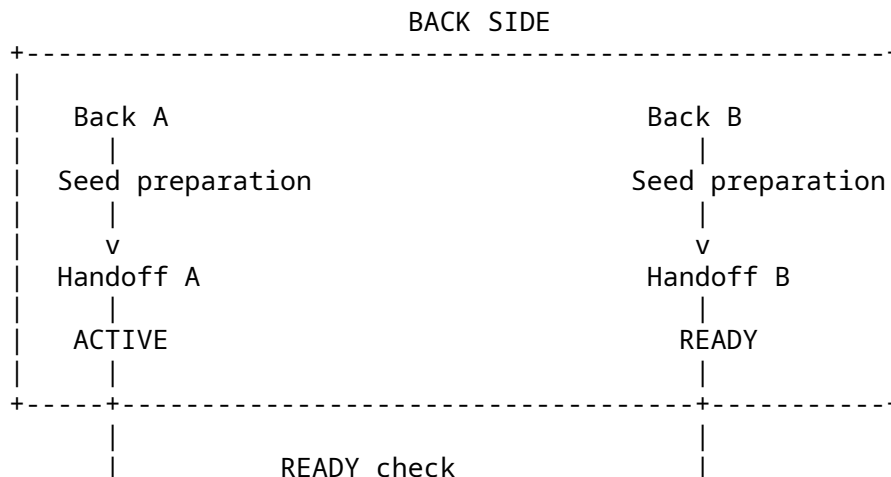
The basic idea is:

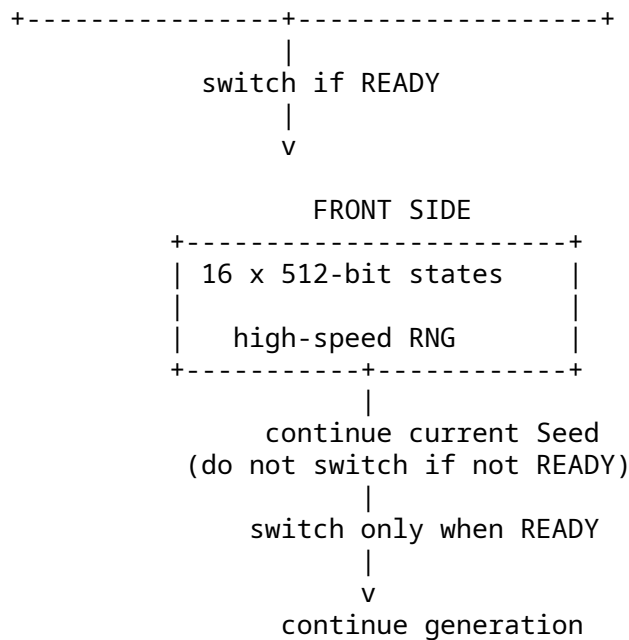


Front does not generate the Seed in place. Back publishes a completed Seed into a handoff slot, and Front accepts the new generation only at a safe point after observing READY.

#### 30AC-1. A/B double buffering

Having two A/B paths allows one to remain ACTIVE for Front while the other is used by Back to prepare the next Seed generation.

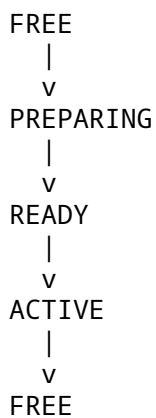




The diagram shows both halves of the mechanism at once: **Back is preparing the next Seed while Front continues running on the current Seed.**

### 30AC-2. Handoff slot state transitions

Each slot uses four states so that an incomplete Seed can never become visible to Front:



- FREE: available for Back to acquire.
- PREPARING: Back is constructing the Seed.
- READY: completed Seed is stored and available to Front.
- ACTIVE: currently used by Front.

Back acquires a FREE slot using an atomic compare-and-exchange and transitions it to PREPARING. When Seed construction completes, it publishes READY with a release store. Front checks READY with an atomic load and performs the READY → ACTIVE handoff at a safe point.

This prevents Front from ever reading an **incomplete Seed while it is still in PREPARING**.

### 30AC-3. Front does not wait for Back

The most important point is that Back preparation speed is not a precondition for Front progress.

Back is fast  
-> READY  
-> switch at next safe point

Back is slow  
-> not READY  
-> continue using current Seed

Therefore:

requested Seed-exchange period  
!=  
actual Seed-exchange period

Dynamic exchange does not impose a fixed contract such as “switch Seed every N samples.” **It switches only when the next-generation Seed is complete and READY.** This prevents Front’s continuous execution from being stopped by Back-side variability.

### 30AC-4. Relationship to one-core-complete

This mechanism does not require Back to be delegated to another CPU. In the current baseline configuration, the engine CPU contains:

same Engine CPU

```
+-----+
| Front |
| Normal|
| PaperView / MC |
| Back A |
| Back B |
+-----+
```

with Back A/B running at low priority. On Linux, SCHED\_IDLE may be used where available; otherwise nice 19 is used.

Thus “asynchronous” here does **not** mean that Back must execute simultaneously on a different CPU. It means that the Front control flow is asynchronous with respect to completion of Back’s Seed preparation: Front does not stop while waiting for the handoff to become ready.

This is compatible with V1’s 1-core-complete execution design rule.

### 30AC-5. Actual switch procedure

For example, if:

handoff A = ACTIVE

handoff B = READY

Front attempts the switch at the next safe point:

1. Atomic-load B and confirm READY
2. Atomic compare-and-exchange READY -> ACTIVE
3. On success, load 16 x 64-byte states into Front
4. Return old A to FREE
5. Resume / re-enable the A-side Back Worker for preparation
6. Reverse A/B roles
7. Increment the switch counter

Only **completed 16 x 64-byte states** are handed to Front. Front does not rerun Seed hashing or heavy Seed expansion.

### 30AC-6. Measured exchange frequency

Under the current V1 one-core condition, seven runs of 1,073,741,824 logical samples / run produced actual Seed-exchange counts of:

3, 3, 4, 4, 3, 4, 4 switches / run

The mean is 3.5714 and the median is 4. The median must not be mechanically reinterpreted as a fixed exchange period.

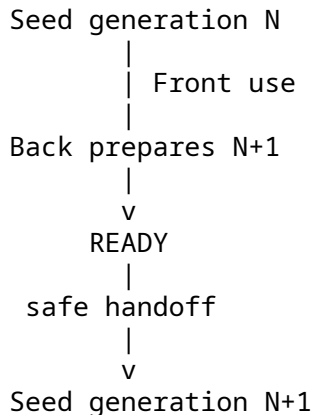
A separate instrumented trace showed generation intervals such as:

Generation 0 : 1 chunk  
Generation 1 : 8,758 chunks  
Generation 2 : 5,586 chunks  
Generation 3 : 2,039 chunks

This concretely illustrates that **the existence of an exchange request and the fact that a next-generation Seed is READY are separate conditions.**

### 30AC-7. "Seed exchange" and "FIXED generation reuse" are different concepts

Dynamic Seed Exchange is:



This is a **Seed-generation switching mechanism.**

FIXED generation, in contrast, is a **sample-generation policy** in which the same starter pool is reused within a generation interval according to the cultivation / re-indexing policy.

Therefore:

safe switching to the next Seed generation

and

how the current starter pool is reused for a specified interval

must be treated as separate design layers.

This distinction is also essential for avoiding confusion between logical sample count and independent sample count.

### 30AC-8. Connection to quality and reproducibility

The new Seed crosses this boundary:

Seed preparation  
-> completed handoff slot  
-> READY

- > safe handoff
- > Front load

It is not mixed into Front mid-computation, so the Seed-generation boundary can be recorded clearly.

Furthermore, managing Seed ID, generation, PROFILE, and similar identifiers on the Back-side control layer while passing only completed Seed material into Front keeps **high-speed execution state separate from reproducibility/control information**.

### 30AC-9. Essence of the mechanism

A one-sentence description is:

**Use A while preparing B; when B is ready, exchange them at a safe point. If B is not ready, keep using A.**

Therefore the essence is:

Make Seed on another CPU	x
Change Seed frequently	x
Mix Seed into Front during execution	x

Separate heavy Seed preparation from continuous Front execution,  
and hand off the completed next-generation Seed at a safe boundary 0

This is a different kind of mechanism from R2 Cache, Finance precompute, and Pharma exposure sharing. It is not merely about reducing Front's work; it is about **separating Front from heavy Seed preparation** as a core architecture.

### 30AD. Deterministic Stream / CRN

The reproducibility control plane can derive a logical stream from a fixed canonical payload. Representative fields include:

```

master seed
experiment ID
run ID
replication ID
scenario ID
CRN group ID
logical stream ID
substream ID
absolute block

```

The canonical payload is SHA-512 hashed to obtain a 64-byte seed.

With CRN, the scenario ID can be excluded so that scenarios share a common random sequence. With CRN OFF, streams are separated by scenario.

Worker ID / worker count are not embedded directly into Seed identity; the design keeps **logical stream identity separate from worker partitioning** where practical.

### 30AE. Checkpoint / Exact Replay

Enterprise control plane features include:

```

Checkpoint
Exact Replay
Golden Reference
CI / Uncertainty Report

```

Checkpointing stores deterministic Front/Normal state, the logical counter, configuration identifiers, and related state so that:

```
checkpoint
-> restore
-> replay
```

can be performed. Archived verification has recorded `exact_match = 1`. Stream/CRN, worker partition, and seek equivalence are additional verification targets.

If the external application maintains its own mutable state, that state also needs an application-side snapshot for complete replay.

### 30AF. Golden Reference / Verification

Golden Reference / Verification is not a production speed path. It is a mode for checking:

**“Are we still producing the same semantic result as before?”**

Representative targets:

```
Deterministic Stream golden vector
CRN scenario invariance
Non-CRN scenario differentiation
Worker partition
Seek equivalence
Checkpoint / Replay
```

### 30AG. Finance / Pharma domain adapters

Finance provides application-specific adapters such as:

```
TIMEVECTOR_ASIAN
TIMEVECTOR_BARRIER
PATH_GEOMETRIC_ASIAN
PATH_LOOKBACK_CALL
PATH_LOOKBACK_PUT
two-factor correlation + Basket
```

Pharma provides structures such as:

```
2C piecewise transition
3C/4C linear piecewise transition
2C biexponential recurrence
2C biexponential closed form
sparse-schedule runners
```

These are not speed profiles. They are **functional choices determining which Monte Carlo problems can be executed**.

### 30AH. Explain the whole system on one page

```
[Seed / Quality]
| - Entropy conditioning
| - LPS
| - Seed Bank
| - PROFILE
| | - CHACHA20
| | - AES256-CTR
| | - Variant-A / Variant-B
```

- |     `- CHACHA20-AAA-A1
- `- VHV lineage

[Normal / Sample relation]

- | - Normal
- `- Antithetic

[PaperView / Transport]

- | - DIRECT
- | - L1\_CACHE
- | - L2\_CACHE
- | - Chunk AUTO/CUSTOM
- | - Generation AUTO/CHUNK/FIXED
- `- Final Profile
  - | - RAW\_INT16
  - | - INT32
  - | - FLOAT32
  - | - R2\_FLOAT32
  - `- CUSTOM

[Common Post]

- | - Statistics
- | - Distribution
- | - Validity
- | - Convergence
- | - Transform
- | - Advanced
- | - Metadata
- `- Pairing

[Reproducibility / Operations]

- | - Deterministic Stream
- | - CRN
- | - Checkpoint
- | - Exact Replay
- `- Golden Reference

[Domain Functions]

- | - Finance
- `- Pharma

[MC Optimization]

- | - R2 cache / 8-ACC / wrap split
- | - terminal sharing / precompute / 4+4
- `- exposure sharing / invariant precompute

### 30AI. The most important point in the design

These options are not all pushed into the Front hot loop.

Heavy quality processing

-> Back / Seed

Output format

-> Final-Cultivation

Transport method  
-> PaperView / Transport

MC statistical processing  
-> Common Post

Reproducibility  
-> Control Plane

Application-specific computation  
-> Finance / Pharma Adapter

Acceleration  
-> MC Optimization Profile

Therefore:

**The fast RNG core and the advanced MC feature set are not packed into the same hot loop; required features can be selected from the outside.**

This is a major characteristic of V1.

### 30AJ. Short explanation for others

UltraFastRng512 is a Monte Carlo execution platform centered on a high-speed AVX-512 RNG, with Normal generation, Antithetic sampling, multiple output formats, PaperView transport, statistical analysis, convergence control, advanced comparisons, deterministic streams, CRN, Checkpoint/Replay, and Finance/Pharma-specific functions arranged as separate layers. A central design feature is that mechanisms for speed are kept separate from mechanisms for quality, application functionality, and reproducibility.

In other words:

```
common RNG / Normal foundation
|
+---- Finance
|
+---- Pharma
|
+---- Generic MC
```

Finance/Pharma domain-specific mathematics must not be brought into Front.

This structure allows the same RNG/Normal/PaperView foundation to be used by different MC models.

---

## Part XIII - Reproducibility

### 31. Deterministic Stream

V1 provides a control plane that addresses a logical stream as a fixed-length seed.

The canonical payload includes:

- master seed
- experiment ID
- run ID
- replication ID

- scenario ID
- CRN group ID
- logical stream ID
- substream ID
- absolute block

A 192-byte canonical payload is hashed with SHA-512 to obtain a 64-byte seed.

Worker ID and worker count are not included in the seed itself.

This makes it easier to preserve logical stream identity when the worker count changes.

---

## 32. CRN

With Common Random Numbers, the scenario ID is excluded from seed derivation.

Thus:

```
Scenario A ----\
Scenario B ----- same logical random stream
Scenario C ----/
```

can be used for comparison.

This makes it less likely that noise from different random sequences is introduced into the scenario difference itself.

---

## 33. Worker partition / seek

Logical blocks can be partitioned among workers.

Even when the same blocks are split across:

```
worker 0
worker 1
worker 2
...
```

worker identity does not enter seed derivation, so the logical stream can be reconstructed from its logical address.

Seek is likewise expressed by absolute block address.

---

## 34. Checkpoint / Exact Replay

A checkpoint stores, among other things:

- stream ID
- stream plan
- logical block
- generation
- Front 16 states
- phase
- parent mode
- Normal starter
- tail mask

- config hash
- state hash

On restore, the canonical hash is verified before state is restored.

This is not a function that simply restores an entire file from disk. It is a function for **logically fixing the deterministic execution state**.

The customer's own mutable MC state must still be snapshotted on the application side.

## Part XIV - Quality Evaluation

### 35. Current V1 aggregate quality

8 seed families x 512 groups, 134,217,728 x/y pairs:

mean x/y	-0.00005304 / +0.00002500
variance x/y	1.000016879 / 0.999839084
skew x/y	+0.0002867 / +0.0005465
kurtosis x/y	2.9993048 / 2.9987968
corr	+0.000047075
dep2	-0.000001154
$E[R^2]$	1.999855966
$\text{Var}(R^2)$	3.997521318
$P(R^2 \leq 1)$	0.393400602
sign	0.499963343

tails:

1 $\sigma$	0.317391440
2 $\sigma$	0.0454207063
3 $\sigma$	0.00270991027
4 $\sigma$	0.0000622868538

Relative difference from the ideal Normal tail:

1 $\sigma$	+0.026%
2 $\sigma$	-0.175%
3 $\sigma$	+0.375%
4 $\sigma$	-1.667%

The 4 $\sigma$  region is more sensitive to statistical noise than 3 $\sigma$  because the event count is lower.

### 35A. Raw Front-state bitstream qualification

Separately from the normal/MC quality audit, an independent statistical battery was run directly on the V1 raw Front-state bitstream.

Because formal PractRand / TestU01 / SmallCrush / dieharder executables were not available in this runtime, the checks were run as an **external-suite-equivalent surrogate battery** covering the major classes. This is not a PASS from the formal suites.

The target stream is the concatenation of the production Front16 -> Normal group update output:

16 resident states x 64 bytes = 1024 bytes/group

It is not the Normal value stream or PaperView/MC output.

Fixed test artifact:

```

groups 524,288
bytes 536,870,912
bits 4,294,967,296
stream SHA-256 c080b9b70484e30a6438ac8d653081ad96274dc0558c2876b793ac554f388cfb
regeneration byte-identical

```

Main results:

Test	Result
Byte frequency $\chi^2$	0.91699
Hamming weight	0.21359
Bit frequency	0.11790
Bit transitions	0.17195
Runs	0.81608
32-bit adjacent-word correlation	0.40494
32-bit collision / birthday-like	0.24091
5-word permutation / OPERM5-like	0.39848
Geometric gap	0.42248
Max-of-5	0.34795
32x32 binary matrix rank	0.29666
Linear complexity M=500	0.09337
Spectral LSB	0.20128
Spectral MSB	0.86762
16 MiB compression screen	1.0003046x

Additional checks produced 32-bit word byte-position  $\chi^2$  values of 0.6031 / 0.8087 / 0.5074 / 0.9597, 16-bit uniformity  $p=0.7150$ , uniform KS  $p=0.2929$ , and lagged-sum KS  $p=0.1102$ .

When all 1024 state-byte positions were examined separately, the minimum was  $p=0.0016696$ , with  $p<0.01$  at 8/1024 positions and  $p<0.001$  at 0/1024 positions. These results include the natural variation expected from multiple testing and were not, by themselves, classified as a defect.

## lag-2 recheck

A short observation window once produced a temporarily low lag-2 p-value. Recalculation over the full 4,294,967,296 bits gave:

```

lag 2 correlation = -7.773e-06
lag 2 p = 0.610472

```

Therefore the short-window low p-value is not treated as a stable structural anomaly.

## Meaning of this test

This battery is not redundant with the existing Normal/MC quality audit.

```

raw Front-state stream
|
bit / word / matrix / run / spectral screen

```

```

Normal / PaperView / MC output
|
marginal / pairwise / radial / tail / convergence screen

```

The first asks whether the base bitstream shows obvious pathology; the second asks whether values delivered to MC show appropriate application-level behavior.

Therefore, **both layers should be included in the Deconstruction Book**, but they should not be fused into one composite score.

## Positioning

The defensible statement at the current stage is:

A 512 MiB-scale independent bit/word-level statistical qualification was performed on the V1 raw Front-state stream, and no obvious anomaly was observed in the principal surrogate tests.

Formal suites remain explicitly:

Surrogate qualification: complete

PractRand executable: not run

TestU01 / SmallCrush: not run

dieharder: not run

---

## 36. Why sign / dep2 are emphasized

Marginal mean/variance alone can miss a pair-structure problem.

For example, the older version could retain a reasonably stable single-row distribution while the pair-formation geometry was damaged.

Therefore V1 considers together:

marginal

+

pairwise

+

radial

+

sign

+

dep2

The purpose is not only to ask "Is it Normal?" but also **"How is it combined inside MC?"**

---

## Part XV - controlled ablation

### 37. What was removed?

For V1, one element is removed at a time.

BASE

NO-AAA

NO-M11R

NO-M15P1

#### NO-AAA

Remove the seed-side AAA mutation.

## NO-M11R

Remove ROTL64(m11, 1) and return to the same ADD form.

## NO-M15P1

Change the parent used for m15 emission from p1 to p0.

V1 itself is not modified by this comparison.

---

## 38. What does this experiment tell us?

In the full-chain integrated audit, all four variants retained broadly stable marginal Normal quality.

However, comparing the numeric changes caused by removing individual V1 components allows us to distinguish:

**“Is this element for speed, for quality, or simply redundant?”**

Because run-to-run noise on the VM can be substantial, a small speed difference of roughly 1-3% must not be interpreted directly as a causal effect.

---

## Part XVI - inter-stream

### 39. 8-stream audit

Eight logical streams were generated independently, with 16,777,216 observations per stream.

All 28 stream pairs were examined.

$\max |corr| = 0.00647255561$   
 $RMS\ corr = 0.002673720513$

In a longer remeasurement, the maximum correlation of the first-run pair declined to roughly 0.0023-0.0027.

The appropriate statement at this stage is therefore:

**No large cross-stream coupling was observed in the measured 8-stream ensemble.**

This must not be stated as a proof of mathematical independence.

---

## Part XVII - MC convergence

### 40. Do not assume that a single stream is IID

It can be misleading to treat V1 single-stream convergence as if it were a simple IID Bernoulli process.

The reason is the presence of PaperView / cultivation / generation-reuse structure.

Therefore, do not directly impose the ideal IID expression

$RMSE \approx C / \sqrt{N}$

on V1 as though it were a complete model.

---

## 41. Stream-ensemble convergence

Example 64-stream ensemble:

```
M=1 RMSE = 8.166995e-3
M=2 RMSE = 6.049314e-3
M=4 RMSE = 3.272206e-3
M=8 RMSE = 2.321948e-3
```

RMSE falls as the number of streams increases.

However, it remains larger than the ideal IID binomial standard error, so V1 convergence should not be described as perfectly matching an ideal IID model.

This is not automatically a defect. It should be recorded as the **actual sampling model induced by V1's deterministic-stream and cultivation structure**.

---

## Part XVII-A - Optimization Quality / Anti-Reuse

### 41A. The two questions separated by this audit

In MC acceleration, the following two questions must not be collapsed into a single "quality" metric.

- A. Optimization preservation
- B. Sample-information reuse

The first asks whether optimization preserves the benchmarked estimator and quality characteristics. The second asks whether an apparent increase in logical sample count actually corresponds to newly generated sample information. ## 41B. Existing bench\_quality\_options\_\* Results

For both V1 and V2, all tested Finance / Pharma options maintained:

```
samples = 32768
sample_ratio = 1.0000000000
```

The maximum relative difference from BASELINE in this rerun was:

```
V1 Finance = 2.299e-07
V1 Pharma = 1.851e-07
V2 Finance = 4.381e-07
V2 Pharma = 5.594e-07
```

Therefore, **there is no evidence in the tested options that the sample counter is being inflated to create an apparent performance increase**.

This is not a proof of independence. Shared or precomputed parameter transforms are intentional reuse and are a different concept from duplication of MC samples.

### 41C. Why Full-q256 Uniqueness Alone Is Insufficient

The existing guard checks:

```
unique full q256 blocks = 256 / 256
adjacent identical full blocks = 0
```

This is useful for detecting complete byte-for-byte copy/replay.

However, an implementation such as PaperView can reorder the same values. In that case the output byte strings can differ while the underlying scalar set is identical. Therefore:

raw bytes differ  
!=  
underlying sample pool is new

#### 41D. Deep-Audit Measurement Results - PaperView

For one chunk, each view contains 64 q256 blocks = 16,384 scalar Normal values. V1 and V2 produced the same structural result:

```
VIEW0 vs VIEW1 : scalar_multiset_equal = 1, exact_q256_replay = 0/64
VIEW0 vs VIEW2 : scalar_multiset_equal = 1, exact_q256_replay = 0/64
VIEW0 vs VIEW3 : scalar_multiset_equal = 1, exact_q256_replay = 0/64
```

Thus the four views are not four disjoint Normal value pools. They present the **same 16,384 logical Normal values as four different deterministic layouts**.

The full q256 blocks are not in the same order, so the original byte-level uniqueness guard does not reveal this fact.

#### 41E. Deep-Audit Measurement Results - CHUNK Generation

Under UFR\_GENERATION\_CHUNK, comparison of two consecutive chunks gave:

```
raw_chunk_equal = 0
exact_q256_replay = 0/256
scalar_multiset_equal = 0
```

This is consistent with the conservative behavior of regenerating the Normal starter at each generation boundary.

Accordingly, CHUNK mode can be treated as **fresh-starter-per-chunk**.

#### 41F. Deep-Audit Measurement Results - FIXED Generation

For UFR\_GENERATION\_FIXED, the same two-chunk comparison was repeated with generation intervals of 2 / 64 / 1,048,576. V1 and V2 both produced:

```
raw_chunk_equal = 0
exact_q256_replay = 256/256
scalar_multiset_equal = 1
```

This is **not duplication of the chunk byte strings themselves**. PaperView's epoch-dependent re-indexing changes the output byte strings. However, the underlying Normal starter value pool is retained within the generation interval, and later chunks re-index that pool.

On the implementation side, `ufr_mc_run()` calls `ufr_paper_make_starter()` only at the generation boundary (`chunks_in_generation == 0`), while FIXED mode continues using the same retained\_starter within the generation.

#### 41G. What Does "46 Gsamples/s" Count?

Accordingly, the V1 value of

46.602 Gsamples/s

measured under a long FIXED generation is **logical MC sample throughput under the long fixed-generation cultivation policy**.

It must not be described as:

46.602 billion newly generated independent Normal samples/s

The deep audit measured reuse at the underlying sample-pool level, so that interpretation is not supported.

## 41H. Added Fresh-Sample Companion Benchmark

Using the same DIRECT / R2\_FLOAT32 / representative specialized path, the companion benchmark used UFR\_GENERATION\_CHUNK and processed 268,435,456 logical samples in one run (4,096 chunks × 65,536 logical samples/chunk).

The 3-run medians in the recovered environment were:

V1 CHUNK = 12.678 Gsamples/s  
V1 FIXED = 40.613 Gsamples/s (same 4,096-chunk harness; generation=1,048,576)

V2 CHUNK = 12.503 Gsamples/s  
V2 FIXED = 44.101 Gsamples/s (same 4,096-chunk harness; generation=1,048,576)

The strict one-core complete benchmark peak for V1 was separately observed at:

46.602 Gsamples/s

These numbers make clear that a path that regenerates the starter every chunk and a path that amortizes starter/cultivation work over a long generation have different throughput semantics. The difference is **not interpreted as a pure causal speed effect of reuse alone**: CHUNK additionally pays the cost of regenerating the starter and loses long-generation cache/cultivation amortization.

## 41I. Documentation Rule

For future V1 benchmarks, at minimum the following should be reported together with Gsamples/s:

logical sample count  
generation policy: CHUNK / FIXED  
generation interval  
PaperView/view count  
final profile  
MC pair geometry

Furthermore,

logical throughput  
independent/fresh-sample throughput

must remain separate concepts. The latter should be reported only when independence or fresh-starter production has been separately verified.

## 41J. Final Audit Conclusion

The audit can be summarized in two layers.

**Optimization quality:** the tested Finance/Pharma options preserved sample count, and estimator differences relative to BASELINE remained at sub-ppm scale.

**Anti-reuse:** the four PaperView views reuse the same underlying Normal starter value pool as different layouts, and FIXED generation reuses that pool within the generation interval.

Therefore, the precise V1 description is:

**Estimator-preserving acceleration with deterministic cultivation and explicitly qualified sample-pool reuse.**

This does not by itself indicate an MC defect. Amortizing starter generation and cultivation cost over a long generation is an intentional throughput strategy. The resulting logical throughput must nevertheless not be reinterpreted as independent fresh-sample throughput.

## 41K. Measurement Results in Numbers - So Readers Can Judge for Themselves

Rather than asking the reader to accept the chapter conclusion alone, this section collects the principal numerical results obtained in the audit so that the scale of the observations can be inspected directly.

### 41K-1. MC Quality Preservation With Acceleration ON/OFF - Common-Core Screen

All cases use N=262, 144 Normal pairs. The theoretical reference for independent standard Normal pairs is approximately:

$\text{Var}(x) / \text{Var}(y) = 1$   
 $\text{corr}(x, y) = 0$   
 $\text{Dep2} = 0$   
 $E[R^2] = 2$   
 $\text{Var}(R^2) = 4$   
 $P(R^2 \leq 1) = 0.393469340287\dots$

Observed values:

Version	Path	Var(x)=Var(y)	corr	dep2	E[R <sup>2</sup> ]	Var(R <sup>2</sup> )	hit=P(R <sup>2</sup> ≤1)
V1	DIRECT / R2_FLOAT32	1.005379	+9.820e-04	-1.535e-03	2.010783	4.035997	0.394981
V1	DIRECT / FLOAT32	0.997287	+4.255e-03	+3.202e-03	1.994574	3.972522	0.392143
V1	DIRECT / INT32	1.004413	+4.060e-03	+8.260e-04	2.008829	4.082587	0.393253
V1	DIRECT / RAW_INT16	1.007821	+2.620e-03	+2.704e-04	2.015642	4.023270	0.390392
V1	L1_CACHE / FLOAT32	1.002832	+8.273e-04	+2.882e-03	2.005686	4.030508	0.391727
V1	L2_CACHE / FLOAT32	1.011196	+6.192e-04	+6.930e-03	2.022515	4.133512	0.391823
V2	DIRECT / R2_FLOAT32	1.005502	+9.604e-04	-1.523e-03	2.011027	4.035535	0.395275
V2	DIRECT / FLOAT32	0.997443	+4.232e-03	+3.230e-03	1.994887	3.971129	0.391983
V2	DIRECT / INT32	1.004515	+4.023e-03	+7.356e-04	2.009033	4.077967	0.393154
V2	DIRECT / RAW_INT16	1.007851	+2.610e-03	+5.927e-05	2.015703	4.021305	0.390110
V2	L1_CACHE / FLOAT32	1.002456	+9.532e-04	+2.431e-03	2.004934	4.024806	0.391949
V2	L2_CACHE / FLOAT32	1.010987	+6.114e-04	+6.817e-03	2.022095	4.128087	0.391766

For direct comparison with the theoretical reference, the representative absolute deviations are:

```

maximum |Var-1| = 0.011196
maximum |corr| = 0.004255
maximum |dep2| = 0.006930
maximum |E[R2]-2| = 0.022515
maximum |Var(R2)-4| = 0.133512
maximum |hit-0.3934693| = 0.0033593

```

These values do not mean that the theoretical targets were matched exactly. They mean that **first- and second-order MC statistics and R<sup>2</sup>-related metrics were measured for each tested acceleration option, without an obvious failure mode appearing in this screen.** The original audit record reports that the V1/V2 official option-quality executables completed with rc=0. ### 41K-2. Finance / Pharma Acceleration Options - Sample Count and Estimator

For the tested acceleration options, each option was compared with BASELINE, including a check that **the sample count itself did not increase or decrease.**

```

samples = 32,768
sample_ratio = 1.000000000

```

The maximum relative difference from BASELINE was:

```

V1 Finance = 2.299e-07
V1 Pharma = 1.851e-07
V2 Finance = 4.381e-07
V2 Pharma = 5.594e-07

```

Thus, within the tested scope, **there was no observed behavior in which enabling an acceleration option inflated the program-level sample counter to create an apparent performance increase.** At the same time, estimator differences relative to BASELINE remained at sub-ppm scale in this measurement scope.

### 41K-3. Two-Stage Test for “Is It Just Copying?”

Simple byte comparison is insufficient, so two types of checks were performed.

First, the existing full-q256 guard reported:

```

unique full q256 blocks = 256 / 256
adjacent identical full blocks = 0

```

This detects **complete byte-for-byte duplication.**

However, an implementation such as PaperView can reorder the same values. Therefore the value set itself was also compared.

### 41K-4. Measured Results for the Four PaperView Views

For one chunk, each view contains 64 q256 blocks = 16,384 scalar Normal values.

```

VIEW0 vs VIEW1 : scalar_multiset_equal = 1, exact_q256_replay = 0/64
VIEW0 vs VIEW2 : scalar_multiset_equal = 1, exact_q256_replay = 0/64
VIEW0 vs VIEW3 : scalar_multiset_equal = 1, exact_q256_replay = 0/64

```

Therefore, **the four views are not four independent 16,384-value pools.** They expose the same Normal starter value pool through different layouts.

This does not mean that the same byte string is copied four times. The order and placement differ. It does mean that the **underlying Normal scalar pool is shared**, which is confirmed numerically.

#### 41K-5. CHUNK Generation - Rebuilding the Starter Every Chunk

Under UFR\_GENERATION\_CHUNK, comparison of two consecutive chunks produced:

```
raw_chunk_equal = 0
exact_q256_replay = 0/256
scalar_multiset_equal = 0
```

Thus, CHUNK mode uses the conservative structure of **generating a new Normal starter at each chunk boundary**.

#### 41K-6. FIXED Generation - The Substance of Long-Run Throughput

In FIXED generation, the same structure was observed even when the generation interval was varied:

```
GEN=2:
 raw_chunk_equal = 0
 exact_q256_replay = 256/256
 scalar_multiset_equal = 1
```

```
GEN=64:
 raw_chunk_equal = 0
 exact_q256_replay = 256/256
 scalar_multiset_equal = 1
```

```
GEN=1,048,576:
 raw_chunk_equal = 0
 exact_q256_replay = 256/256
 scalar_multiset_equal = 1
```

Thus, in FIXED mode, **the chunk byte strings change, but the same Normal starter value pool is retained within a generation and re-indexed**.

This is intentional cultivation/reuse and is different from accidental byte-level copying. It is nevertheless not correct to describe FIXED mode as generating a completely new independent Normal pool for every chunk.

#### 41K-7. Throughput Side by Side: Fresh Starter vs Long FIXED

In the same recovered environment, the same representative DIRECT / R2\_FLOAT32 path was used to compare the fresh-starter and FIXED paths.

3-run median

	CHUNK / fresh starter	FIXED / long generation
V1	12.678 Gsamples/s	40.613 Gsamples/s
V2	12.503 Gsamples/s	44.101 Gsamples/s

The strict one-core complete V1 peak was separately measured at:

46.602 Gsamples/s

These figures show that a path that regenerates the starter every chunk and a path that amortizes starter/cultivation work over a long generation have different throughput semantics. The difference is **not interpreted as a pure causal speed difference caused by reuse alone**. CHUNK additionally incurs starter regeneration and loses long-generation cache/cultivation amortization.

#### 41K-8. Inter-Stream and Convergence Numbers

Rather than examining the accelerated MC path through only a single stream, combinations of eight logical streams were also measured.

8 streams  
16,777,216 observations / stream  
28 stream pairs

max |corr| = 0.00647255561  
RMS corr = 0.002673720513

In a longer remeasurement, the correlation of the pair with the highest initial value fell to approximately 0.0023–0.0027. What can be said from this is that **no large cross-stream coupling was observed in the measured eight-stream ensemble**; it is not a proof of mathematical independence.

An example of 64-stream convergence was:

M=1 RMSE = 8.166995e-3  
M=2 RMSE = 6.049314e-3  
M=4 RMSE = 3.272206e-3  
M=8 RMSE = 2.321948e-3

The RMSE decreased as the number of streams increased. However, because the result does not exactly match the ideal IID binomial standard error, the V1 sampling model should not be described simply as IID. [internal audit record]

#### 41K-9. Assessment After Reading the Numbers

For clarity, the practical interpretation of the measurements is:

[MC quality broken by acceleration]  
Not observed in the tests performed.

[Sample-counter inflation by acceleration options]  
Not observed in the tested Finance/Pharma option set.

[Sample-pool reuse through PaperView / FIXED]  
Confirmed.

[46 Gsamples/s = 46 G independent fresh samples/s]  
That is not what the number means.

[46 Gsamples/s = usable logical MC processing per second under the measured policy]  
This is the accurate description.

[Mathematical IID / complete-independence proof]  
Not established.

Therefore, the appropriate interpretation of the measurements is:

**Within the tested acceleration configurations, V1 maintains MC-usable quality. The 46 Gsamples/s headline throughput is not an independent fresh-sample generation rate; it is logical MC throughput that includes deterministic PaperView cultivation, starter reuse under FIXED generation, and cache/cultivation amortization.**

#### 41L. Final Reader Assessment - “Can This Acceleration Be Used for MC?”

The conclusion of this chapter is stated first.

**Within the scope of the current audit, no quality problem was observed when the tested acceleration options were used for Monte Carlo calculations.**

This conclusion is not based merely on increased speed. In the acceleration ON/OFF comparisons, the tested options preserved the MC sample count, and estimator differences from BASELINE remained at sub-ppm scale in the rerun. The eight-stream cross-stream audit did not observe large coupling, and the multi-stream convergence experiment showed the expected decrease in MC estimation error as the number of streams increased. However, this is not a mathematical proof of independence or strict IID behavior.

The reader should therefore distinguish:

How much MC processing using numerically acceptable random inputs can be performed per second

↓

logical MC throughput

How much new independent sample information is produced per second

↓

fresh / independent sample rate

V1's 46.602 Gsamples/s is the former: **logical MC sample throughput under long FIXED-generation conditions**. PaperView and FIXED generation reuse the starter value pool and amortize cultivation/cache work, so the value must not be interpreted as **46.602 billion completely new and independent Normal samples generated per second**.

Conversely, this does not mean that the path is unsuitable for MC. The observed reuse is not simple duplication of the same byte string for every chunk; it is implemented as deterministic cultivation with re-layout and re-indexing. The acceleration comparison also did not reveal a failure of the MC estimator in the tested scope.

Therefore, the practical V1 conclusion is:

**Within the scope of the current audit, no quality problem was observed in the tested accelerated MC path. The reported 46.602 Gsamples/s value should be treated as logical MC throughput that includes sample-pool reuse and cultivation amortization, not as a fresh independent sample rate.**

The phrase "usable" here refers to the engineering audit scope described in this book. It does not constitute final external qualification by named suites such as TestU01, PractRand, or NIST.

## Part XVIII - Performance

### 42. Current V1 Speed References

#### FLOAT32

268,435,456 logical samples/run:

23.374

23.485

22.962

10.311 <- VM/load outlier

22.335

median = 22.962 Gsamples/s

#### R2

41.558

34.491 <- outlier

43.618  
43.514  
43.741

median = 43.514 Gsamples/s

## Peak

strict one-core complete-engine benchmark:

46.602 Gsamples/s

Peak observed in an 8.59-billion-logical-sample timed run. This is a long FIXED-generation logical-throughput figure and does not represent freshly generated independent Normal samples/s.

The measurement was obtained under:

- one core
- complete engine
- AMD EPYC 9V74 virtualized environment

and is not a CPU-independent constant.

---

## 42A. The Correct Interpretation of the 3-Core Experiment - 12.7485 Gsamples/s Is Not “V1 Getting Slower”

The one-worker value 12.7485 Gsamples/s shown in the 3-core experiment does not represent V1’s current highest one-core performance. It is the **one-worker baseline measured under the same CPU and compiler conditions in order to evaluate how aggregate throughput changes when independent workers are added to three cores.**

Without this distinction, comparing it with the ordinary V1 throughput reference 22.962 Gsamples/s could create the false impression that V1 suddenly became slower.

The measurement conditions are different.

Ordinary V1 throughput reference

CPU = Intel Xeon Platinum 8573C  
compiler = gcc -O3 -std=c11 -pthread -march=icelake-server  
FLOAT32 median ≈ 22.962 Gsamples/s

3-core parallel-scaling experiment

CPU = AMD EPYC 9V74 80-Core Processor  
runtime = 3 vCPUs (CPU0-2), 1 thread/core exposed  
compiler = gcc -O3 -std=c11 -pthread -march=znver4  
1 worker = 12.7485 Gsamples/s median  
3 workers = 38.656 Gsamples/s clean full-active median

## Why the 1-worker value is lower on AMD EPYC 9V74

AMD Zen 4 provides 512-bit AVX-512 registers, while the internal datapath for these vector operations is 256-bit. AMD describes 512-bit AVX-512 execution as **double-pumped**, processing a 512-bit operation as two 256-bit vector operations over successive clock cycles. [AMD Zen 4 / EPYC architecture documentation]

This matters for V1 because its hot path makes extensive use of 512-bit AVX-512 operations:

512-bit ZMM  
↓  
AVX-512

↓  
GFNI / byte permutation / integer MADD / other 512-bit vector operations

Intel Ice Lake systems also support 512-bit vector operations and 512-bit loads/stores, according to Intel's optimization documentation. [Intel optimization documentation]

Therefore, the observed one-worker difference between the Intel and AMD environments can be influenced substantially by microarchitecture and AVX-512 execution characteristics.

Intel Xeon 8573C     $\approx 22.96$  Gsamples/s  
AMD EPYC 9V74      $\approx 12.75$  Gsamples/s

This difference should be interpreted as **the combined result of different CPU microarchitectures, AVX-512 execution characteristics, compiler targets, and VM conditions, rather than as evidence of degradation in the V1 algorithm itself.**

However, the ratio  $12.75 / 22.96 \approx 0.555$  must not be attributed entirely to the 256-bit datapath of Zen 4. Actual throughput is also affected by instruction mix, scheduler behavior, cache state, clock behavior, compiler code generation, VM scheduling, and other factors. This benchmark alone cannot completely separate those individual causes.

### The Actual Result of Moving to Three Cores

The correct way to read this experiment is:

AMD EPYC 9V74 / same experimental condition

1 worker  
  12.7485 Gsamples/s  
  ↓  
3 independent workers  
  38.656 Gsamples/s (clean full-active median)  
  ↓  
 $38.656 / 12.7485 = 3.032x$

The raw 10-run median for three workers was 37.016 Gsamples/s, but it included runs clearly disturbed by the VM scheduler, such as:

20.348  
14.709  
14.918 Gsamples/s

The six runs in which all three workers broadly maintained normal throughput were:

38.393  
38.956  
35.668  
39.319  
38.364  
38.918

giving a clean full-active median of 38.656 Gsamples/s.

Therefore, these six runs should not be described as merely selecting convenient numbers. The disturbed runs remain in the raw record, while the condition in which all three workers were operating normally is separately shown as the **clean subset for evaluating parallel scaling.**

### What This Experiment Does and Does Not Show

The three-core test shows that:

**On the same AMD EPYC 9V74 environment, increasing the number of independent Front/MC workers from one to three produced approximately 3.03× aggregate throughput for FLOAT32.**

It does not mean that:

- the one-worker reference of 12.7485 Gsamples/s is V1's universal or current highest one-core performance;
- three complete Back+Front engines were measured on three cores;
- 3.03× scaling will occur on every CPU.

In the three-worker benchmark, the internal Back worker thread of each worker was not started. The purpose was to measure **application-level independent Front/MC parallelism**. This is a separate experiment from the complete Back+Front one-core engine benchmark.

### **Relation to One-Core-Complete Execution**

V1's basic design remains that **Front + Back can complete on one core**. In a separate experiment, Back A/B was moved to CPU1/CPU2 while Front remained on CPU0:

```
strict one-core complete = 11.763084 Gsamples/s
split-Back = 11.696659 Gsamples/s
```

The median difference was only approximately 0.568%.

Thus the demonstrated 3.032× is not explained by moving Back work to other cores. It should be described as **parallel scaling obtained by adding independent workers**.

### **Future Expansion to High Core Counts - 192-Core Peak-Envelope Thought Experiment**

AMD offers processors with up to 192 physical cores in the 5th Gen EPYC 9965. [AMD EPYC 9965 public specifications]

Here, the 3-core scaling result of 38.656 Gsamples/s is not multiplied by 192. That number is a **clean aggregate measurement from a separate experiment whose purpose was to validate three-core parallelism**.

For the 192-core outlook, the baseline is the **highest one-core complete peak actually observed for V1 = 46.602 Gsamples/s**.

Measured V1 one-core complete peak:  
46.602 Gsamples/s

Idealized 192-core linear envelope:  
 $46.602 \times 192$   
 $= 8,947.584 \text{ Gsamples/s}$   
 $= 8.947584 \text{ T logical samples/s}$   
 $\approx 8.948 \text{ T samples/s}$

This number was **not measured on 192 cores**. It is also not an estimate derived from the three-core clean median.

It is a conditional **peak-envelope thought experiment** based on three assumptions:

1. use the measured one-core V1 peak as the baseline;
2. assume each worker can sustain the same peak;
3. assume perfectly linear scaling to 192 workers.

On real hardware, NUMA, memory hierarchy, frequency behavior, OS scheduling, worker placement, system-wide bandwidth, shared infrastructure, and other system effects may reduce throughput below this ideal value. Therefore, this value must not be treated as a prediction, guarantee, or measured result.

Nevertheless, combining the observed 46.602 Gsamples/s/core with a 192-core-class CPU shows that a throughput region of **approximately 8.95 T logical samples/s is theoretically within view** under the stated idealized assumptions. It is meaningful as a figure illustrating an upper-envelope thought experiment.

### Hypothesis Concerning GPU Comparison

As future research, it may also be useful to compare a configuration that distributes the same Monte Carlo work across multiple GPUs. In workloads where host-device data movement, kernel launch, PCIe/interconnect paths, or device-memory management become dominant, CPU-resident execution may exhibit different characteristics in one or more of **latency, time-to-first-result, software complexity, energy per sample, and energy-to-solution**.

However, no same-condition GPU comparison, measured latency, energy/sample, or TCO results have been obtained at this stage. Therefore:

**The possibility that CPU-only V1 may offer lower latency or higher power efficiency than a high-end GPU configuration**

should be treated as a **testable future hypothesis**, not as an established result. This book does not conclude that V1 is faster or more energy-efficient than GPUs.

---

## 43. Why Performance Should Not Be Reduced to a Single Number

V1 separates:

- Front
- Normal
- PaperView
- final-cache build
- MC
- R2
- seed exchange
- transport

Accordingly, a single statement such as:

“UltraFastRng512 runs at XX Gsamples/s”

is insufficient.

At minimum, a paper should report these together:

profile  
transport  
chunk size  
view count  
generation policy  
sample count  
CPU / compiler

---

## Part XIX - External RNG Tests

### 44. Current Status

The actual executable implementations of the formal external suites have not yet been run. In contrast, an independent 512 MiB surrogate battery covering the principal test areas has been completed.

Surrogate bitstream qualification	COMPLETE
PractRand executable	not run
TestU01 / SmallCrush	not run
dieharder	not run

Accordingly, the paper may describe the current result as **independent empirical bitstream qualification**, but must not claim PASS PractRand, PASS SmallCrush, or PASS dieharder.

NIST SP 800-22 likewise treats statistical testing as an initial stage and does not present statistical testing alone as an absolute certification of a generator.

---

## Part XX - Positioning Relative to Existing Methods

### 45. Academic Comparison Targets

#### Random123

A representative counter-based PRNG design emphasizing stateless keyed transformations, parallel streams, and vectorization.

#### PCG

A representative example combining a compact state with a permutation-based output transformation.

#### SFMT

A representative design in which the generator structure itself is tailored to SIMD architecture.

#### Ziggurat

A representative table-based method for fast Normal generation.

#### Box-Muller

A classical reference for Normal generation. Unlike V1, it uses floating-point transcendental operations such as log, sqrt, sin, and cos.

V1 should not claim that it is categorically “better” than these alternatives.

The defensible position at this stage is:

**V1 should be positioned as a system-level architecture that builds on established SIMD PRNG, parallel-stream, and table-based Normal-generation approaches while co-designing them for the complete Monte Carlo data path.**

---

## Part XXI - Novelty

### 46. What Must Not Be Presented as Novelty

The following items alone should not be presented as novel contributions:

- using AVX-512;
- using 16-lane SIMD;
- using a LUT;
- using ChaCha;
- using SHA-512;
- having multiple streams.

These elements are broadly represented in existing work.

---

### 47. System-Level Contribution That May Be Examined for V1

The central candidate is the combination of:

1. separating Back-side seed preparation from the Front hot path;
2. compressing three same-profile VHV parents into a 16-state resident budget;
3. implementing integer Normal-v9 with a 128-byte LUT and AVX-512;
4. separating four-way fanout as an explicit supply and placement layer;
5. controlling physical layout and reuse through PaperView/cultivation;
6. using different pair geometries, namely generic 465 and R2 555, according to the consumer;
7. connecting deterministic streams, CRN, checkpointing, and domain adapters to the same system.

This is a **co-designed Monte Carlo execution architecture** at the system level.

The claim should ultimately be checked against a broader literature review.

---

## Part XXII - How to Use the Project as a POC

### 48. Minimal Build

The V1 POC package includes the main source and required headers and uses OpenSSL/libm.

Typical build:

```
gcc -O3 -std=c11 -march=icelake-server -pthread \
-Iinclude src/UltraFastRng512_MC_NORMALTAIL_OPTIMIZED_V1.c \
-lcrypto -lm -o ufr_v1
```

The target CPU must support the required AVX-512 ISA set.

---

### 49. Self Verification

```
./ufr_v1 verify
```

On success:

Enterprise selftest: PASS

This is not external statistical certification. It verifies the V1 built-in golden reference and control-plane consistency.

---

## 50. Representative MC Run

The CLI exposes:

```
[auto|direct|l1|l2]
[chunks]
[auto|raw16|int32|float|r2]
[generation_chunks]
```

For example,

```
./ufr_v1 direct 4096 float 8192
```

runs a representative Direct / FLOAT32 / fixed-generation 8192 configuration.

```
./ufr_v1 direct 4096 float 8192
```

This runs a representative Direct / FLOAT32 / fixed-generation-8192 configuration.

---

## Part XXIII - What Must Not Be Changed in V1

### 51. Paper Baseline Integrity Rules

During the measurement period for the paper, do not change the following without creating a new version:

- AAA profile
- 16-state budget
- 3-parent architecture
- VHV
- Normal LUT
- Normal scale
- 465 generic stride
- 555 R2 stride
- PaperView phase/shift rules
- stream derivation
- checkpoint semantics

Any change must be handled as a separate version / branch.

---

### 52. Do Not Assume “Quality Improvement Means It Is Safe”

Especially in MC, marginal quality can improve while pair geometry becomes worse.

Conversely, single-row moments can remain unchanged while pair quality improves dramatically.

Therefore, V1 evaluation must examine all of the following together:

marginal + pair + radial + tail + reproducibility + speed

---

## Part XXIV - Current Status of V1

### 53. Areas That Are Currently Strong

Architecture	strong
Front / Normal	strong
PaperView	strong
MC acceleration	strong
Speed measurements	strong
Statistical quality	strong
Root-cause experiment	strong
Reproducibility	strong

The V1 core mathematical specification is substantially complete, and a full-chain controlled-ablation version is available.

---

### 54. Areas Still Requiring External Evidence

Formal external RNG suite execution  
Fair same-host competitor benchmarks  
Exhaustive literature review  
Customer-domain numerical error models

These items do not mean that the code is incomplete.

They remain open because the required evidence must come from external environments; V1 itself should not be modified merely to obtain it.

---

## Part XXV - Why Stop at V1 for the Time Being

### 55. The Correct Research Cycle

In a large integrated system such as V1, changing the algorithm while measurements are in progress causes five things to diverge:

paper numbers  
code  
mathematical specification  
reproduction experiments  
POC

Therefore, the current structure should remain:

```
V1 freeze
|
+-- paper
+-- mathematical specification
+-- deconstruction book
+-- POC
+-- benchmark archive
|
+-- V2 research history
```

---

## Appendix A - V1 Identity

Source filename:

UltraFastRng512\_MC\_NORMALTAIL\_OPTIMIZED\_V1.c

Frozen submission SHA-256:

40a6b4311b4cb0e4348f08990b5380ff7607444a92534e17e7014b997af393ba

Measured benchmark snapshot SHA-256:

cd8dae646a9865f9867095df5da4903a6e03ce3f806a9dbbce135c8ec2a4ee8b

---

## Appendix B - V1 Constant Card

Front states	16
Parent states	3
Auxiliary states	13
Fanout	4
PaperView rows	1024
PaperView lanes	16
PaperView samples	16384
PaperView bytes	65536
Generic final-cache stride	465
R2 pair stride	555
AAA rounds	28
AAA injection rr	9
AAA injection word	8
AAA kick	0x3B54CDA5
LPS p	5
LPS q	29
LPS lanes	4
Normal LUT	128 bytes
Normal scale factor	0.0009317057574691516 * 1.0007781

---

## Appendix C - What the Project Does NOT Claim

This project does not, on the basis of the current V1 evidence alone, claim:

- cryptographic unpredictability;
- formal entropy-extraction security;
- a formal independence theorem for streams;
- ideal IID convergence for one deterministic stream;
- formal PractRand/TestU01/SmallCrush/dieharder certification before those exact suites are actually run;
- despite the surrogate battery, that it is identical to any named external suite;
- portable hardware-independent throughput;
- universal superiority over published PRNGs or Normal generators.

The scientifically defensible position is narrower and stronger: V1 is a documented AVX-512 Monte Carlo execution architecture with reproducible internal controls, substantial engineering-scale statistical evidence, and explicitly identified remaining external qualification tasks.

---

## Part XVII - Evidence Management for Academic Materials

### 40. Why V1 Is Frozen as the Reference Point

The primary lineage of this material is UltraFastRng512\_MC\_NORMALTAIL\_OPTIMIZED\_V1.c. V2 is retained as Normal-tail research history, but is separated from the reference implementation, paper numbers, and POC baseline of this book.

For V1, the measurement-time source snapshot and the submission freeze copy are tracked with separate IDs. This prevents an administrative SHA change caused by source organization from being confused with an algorithm change.

### 41. What Has Already Been Demonstrated

The current V1 materials independently track the complete architecture, Front/Normal, PaperView/cultivation, MC acceleration, performance, statistical quality, root-cause experiments and fixes, deterministic stream / CRN, checkpoint/replay, and controlled ablation.

For quality in particular, 134,217,728 x/y pairs have been evaluated for variance, kurtosis, correlation, dep2,  $E[R^2]$ ,  $\text{Var}(R^2)$ , hit, sign, and 1/2/3/4 $\sigma$  tail behavior.

### 42. Do Not Hide Important Unfinished Items

If the external PractRand/TestU01/dieharder executables are not available or have not been run, they remain marked as **not run**.

Likewise, no unconditional  $N^{-1/2}$  RMSE law of ideal IID sampling is claimed for a single deterministic stream. The current PaperView/cultivation structure deliberately reuses finite structures, so MC convergence is presented through measured stream ensembles.

For comparisons with existing methods, literature performance figures are not promoted into same-condition V1 values. Only results measured under same-host / same-build / same-output-target conditions should be promoted into a direct comparison table.

### 43. Boundary of External Testing

External RNG batteries should be run through the published raw Front-state stream adapter. This stream is an independent diagnostic of Front-state bit quality and is distinct from the quality of the Normal distribution, PaperView, or MC layers.

Therefore, even after external batteries are run, report:

raw Front-state statistical screen

and

Normal/MC application-quality audit

in separate fields.

### 44. Three Levels of Academic Comparison

Separate comparisons into three layers:

1. Normal transform: V1 LUT-v9 versus Box-Muller / Ziggurat
2. RNG core: V1 Front versus published SIMD-oriented RNGs
3. End-to-end: RNG -> Normal -> cultivation -> MC

These answer different questions and should not be collapsed into one "RNG speed" number.

## 45. Positioning in the Literature

AVX-512 vectorization of PRNGs is itself an established area, with examples including AVX-512 vectorization of LFSR113, Random123 counter-based generators, SFMT, PCG, and MRG32k3a/RngStreams.

Therefore, the central V1 claim should not be “we used AVX-512.” It should be the co-design of Back-side seed conditioning, same-profile VHV multi-parent Front, the 16-state resident budget, integer Normal-v9, fanout, PaperView/cultivation, use-specific pair geometry, and deterministic streams/control-plane as one Monte Carlo execution architecture.

That claim is presented as a system-level contribution and remains separate from any claim of universal superiority or a proof of statistical independence.

## 46. Recommended POC Reading Order

The distribution package should make it possible to follow the project without oral explanation in this order:

README -> build/verify -> example -> architecture -> exact mathematical specification -> benchmark results -> limitations

---

## Part XXVI - Exact Implementation Ledger (V1)

### 56. Three-Parent State Placement - Exact Source Mapping

For generation  $g$ , the permutation index is:

$\text{pi} = \text{floor}(g/2) \bmod 6.$

The selected parent order is one of the six permutations of  $(0, 1, 2)$ .

The 16 resident states are populated exactly as follows:

```
state 0 = selected A[0]
state 1 = selected B[0]
state 2 = selected C[0]
state 3 = selected A[2]
state 4 = selected B[2]
state 5 = selected C[2]
state 6 = selected A[3]
state 7 = selected B[3]
state 8 = selected C[3]
state 9 = selected A[4]
state 10 = selected B[4]
state 11 = selected C[4]
state 12 = selected A[5]
state 13 = selected B[5]
state 14 = selected C[5]
state 15 = A[1] XOR B[1] XOR C[1]
```

This is not a generic three-parent abstraction; it is the exact V1 placement implemented by `ufrx_prepare_next_multi`.

### 57. Exact Parent Recurrence

The V1 source selects one of three recurrence modes through `parent_xor_omit`:

```

mode 0:
 p0 = a XOR ROTL64(b,17)
 p1 = b + ROTL64(c,29)
 p2 = c + ROTL64(a,41)

```

```

mode 1:
 p0 = a + ROTL64(b,17)
 p1 = b XOR ROTL64(c,29)
 p2 = c + ROTL64(a,41)

```

```

mode 2:
 p0 = a + ROTL64(b,17)
 p1 = b + ROTL64(c,29)
 p2 = c XOR ROTL64(a,41)

```

The generation-level selection and the runtime state field that carries the active mode must remain distinct in the documentation. The public mathematical description is the three-mode recurrence above; the executable source carries the selected mode in the Front state context.

## 58. Exact Auxiliary Update Schedule

The 13 auxiliary states are updated one at a time in a 13-phase cycle. The exact source equations are:

```

phase 0: m3 = (m3 + ROTL64(p0, 7)) XOR ROTL64(m3, 13)
phase 1: m4 = (m4 + ROTL64(p0, 13)) XOR ROTL64(m4, 17)
phase 2: m5 = (m5 + ROTL64(p0, 17)) XOR ROTL64(m5, 21)
phase 3: m6 = (m6 + ROTL64(p0, 23)) XOR ROTL64(m6, 29)
phase 4: m7 = (m7 + ROTL64(p1, 29)) XOR ROTL64(m7, 33)
phase 5: m8 = (m8 + ROTL64(p1, 31)) XOR ROTL64(m8, 37)
phase 6: m9 = (m9 + ROTL64(p1, 37)) XOR ROTL64(m9, 41)
phase 7: m10 = (m10 + ROTL64(p1, 42)) XOR ROTL64(m10, 47)
phase 8: m11 = (m11 + ROTL64(p2, 11)) XOR ROTL64(m11, 17)
phase 9: m12 = (m12 + ROTL64(p2, 19)) XOR ROTL64(m12, 23)
phase 10: m13 = (m13 + ROTL64(p2, 27)) XOR ROTL64(m13, 31)
phase 11: m14 = (m14 + ROTL64(p2, 37)) XOR ROTL64(m14, 41)
phase 12: m15 = (m15 + ROTL64(p2, 43)) XOR ROTL64(m15, 47)

```

The phase advances modulo 13 after each micro-step.

## 59. Exact Output Emission and Fanout

The 16 emitted source vectors are:

```

1 p0
2 p1
3 p2
4 p0 XOR m3
5 p1 XOR m7
6 p2 + ROTL64(m11,1)
7 p0 XOR m4
8 p1 XOR m8
9 p2 XOR m12
10 p0 XOR m5
11 p1 XOR m9
12 p2 XOR m13
13 p0 XOR m6
14 p1 XOR m10

```

```

15 p2 XOR m14
16 p1 XOR m15

```

Every emitted vector then passes through `normal_transform16_i32()` and the four-way fanout:

```

q + shuffle0(q)
q - shuffle0(q)
q + shuffle1(q)
q - shuffle1(q)

```

where the source uses:

```

shuffle0 = _MM_SHUFFLE(2,3,0,1)
shuffle1 = _MM_SHUFFLE(1,0,3,2)

```

## 60. Exact Normal-v9 Semantics

The active V1 transform is integer-only at the transformation boundary:

```

x = 16 x uint32 loaded into one ZMM
a = VPERMI2B / _mm512_permutex2var_epi8(LUT128, x)
p16 = VPMADDUBSW(a, x)
q = VPMADDWD(p16, vector-of-+1-int16)

```

VPMADDUBSW performs **unsigned-byte × signed-byte** multiplication; the sum of each adjacent pair is stored as a **signed saturating int16**. The subsequent VPMADDWD performs **signed-int16 × signed-int16** multiplication and adds each pair; each coefficient in the second operand is +1.

Thus, this does not specify an “all-ones” bit pattern such as 0xFFFF. It specifies an **int16 coefficient vector whose numerical value is +1 in every lane**.

On an x86 little-endian system, each uint32 enters the LUT lookup byte sequence from its low byte in memory order.

The resulting q is an integer-valued internal Normal-domain representation. The downstream floating-point path applies:

```
scale = 0.0009317057574691516 * 1.0007781.
```

For the R2 path, at the pair-formation boundary the components are:

$$x = q_0 s / \sqrt{2}, \quad y = q_1 s / \sqrt{2}$$

followed by  $R^2 = x^2 + y^2$ .

The implementation-level squared coefficient is:

$$scale2 = (s / \sqrt{2})^2.$$

## 61. Exact PaperView Control Variables

The current V1 public descriptor family contains 32 views and the following fixed offset list:

```

0,333,254,611,214,871,300,561,739,892,377,139,89,406,718,996,
353,202,491,528,322,276,730,847,41,385,936,839,238,427,960,762

```

The execution mapping is:

```

x0 = epoch & 1023
x1 = BR10(x0)
shift = x1 >> 6

```

```

phase = (epoch >> 10) & 63
vid = (view_id + phase + (phase >> 3)) & 31
base = (A64_OFFSET[vid] + shift) & 1023

```

For each output block `block_id`, with `out0 = block_id * 16` and `j = 0..15`, the exact logical-to-starter row mapping is:

```
src = (out0 + j + base) & 1023
```

The selected lane permutation is `UFR_PAPER_A64_INDEX[UFR_PAPER_A64_LANE[vid]]`. If `tail_mask[src] != 0`, the corresponding `UFR_PAPER_TAIL_PERM[UFR_PAPER_A64_TAIL[vid]]` is used instead.

The complete 64-entry lane-permutation family and tail-permutation family are included in Appendix D. Those arrays are implementation data, not a dynamic runtime algorithm.

## 62. Exact Final-Cache Rule

Generic path:

$$r' = P_{465}(r) = 465r \bmod 1024.$$

For the R2 path, the pair bank is constructed as:

$$r_0 = P_{555}(2p + \pi), \quad r_1 = P_{555}(2p + \pi + 1).$$

The runtime consumer selects the parity bank and pair offset within that bank and then performs sequential access.

Because 465 and 555 are each coprime with 1024, both row permutations are bijections.

The central design point is that the permutation cost is paid during cache construction, allowing the downstream MC consumer to use sequential reads.

## 63. Scientific Interpretation Boundary

The V1 source contains cryptographic primitives and an LPS-based preparation step, but the project does not claim that this composition constitutes a CSPRNG or a formal entropy extractor. The appropriate paper description is an engineering Monte Carlo PRNG/data-path architecture with strong deterministic conditioning and diffusion stages.

Likewise, measured stream correlations are empirical observations, not a theorem of pairwise independence.

# Part XXVII - Academic Evidence Map

## 64. Evidence Layers

The evidence for V1 should be understood as a layered structure rather than as one pass/fail result.

Layer A: implementation identity  
frozen source / SHA / exact implementation ledger

Layer B: internal numerical evidence  
Normal statistics / pair geometry / tail / inter-stream / convergence

Layer C: engineering validation  
self verification / benchmark reproduction / anti-reuse audit

Layer D: external statistical qualification  
PractRand / TestU01 / other named suites

Layer E: external competitive comparison  
same-host / same-build / same-workload baselines

The current book contains substantial evidence in Layers A-C. Layers D and E remain external tasks and are not silently replaced by the internal evidence.

## 65. What Is Already Strong Enough for the Paper

For the paper, the following can be presented as documented V1 results within the measured scope:

- exact V1 source identity and implementation mapping;
- the Back / Front architecture and one-core-complete execution model;
- the VHV / 3-parent / 16-state construction;
- integer Normal-v9 and its measured marginal statistics;
- PaperView / cultivation and the explicit qualification of sample-pool reuse;
- the measured MC pair/radial statistics and the 3-core worker-scaling experiment;
- deterministic streams, CRN, checkpoint/replay, and reproducibility controls.

These are not claims of universal statistical perfection or superiority. They are claims about the documented V1 design and the measurements actually performed.

## 66. What Remains Genuinely External

The following remain independent external qualification tasks:

PractRand named-suite execution  
TestU01 named-suite execution  
same-host / same-build competitor comparison  
broader literature review  
independent customer-domain numerical error analysis

A paper may state clearly that these are pending rather than filling the gaps with assumptions.

---

## Part XXVIII - Related Work: Defensible Positioning

### 67. Random123 / Counter-Based Generators

Random123 is relevant because its counter-based design directly addresses reproducible parallel streams and stateless/vectorizable transformations.

The useful comparison point for V1 is therefore not “both generate random numbers,” but how a deterministic stream architecture is coupled to the downstream MC data path.

## 68. PCG

PCG is a representative family combining a compact state-transition mechanism with a permutation-oriented output function.

It provides a useful baseline for discussing the separation between state evolution and output transformation, but it does not by itself define V1's Back/Front or cultivation architecture.

## 69. SFMT and AVX-512 LFSR113

SFMT and AVX-512 vectorized LFSR113 are relevant reference points for SIMD-oriented pseudorandom generation.

They demonstrate that wide-vector PRNG execution itself is an established direction. V1's candidate contribution is at the MC-system level, not the mere use of wide vectors.

## 70. MRG32k3a / RngStreams

MRG32k3a and RngStreams are relevant for reproducible stream management, stream/substream partitioning, and simulation methodology.

They are particularly useful comparators for discussing the reproducibility/control-plane layer separately from the raw generator core.

## 71. Ziggurat

Ziggurat is a major reference for fast Normal generation and provides a natural comparison target for the V1 integer LUT-based Normal stage.

The comparison must be made at the Normal-generation boundary, not by comparing its raw generator or runtime overhead with V1's full MC engine.

## 72. Intel oneMKL

Intel oneMKL is relevant for both Normal-generation performance methodology and established random/distribution infrastructure.

Its published performance data also make explicit that CPE depends on architecture, underlying BRNG, transformation method, vector length, and distribution parameters. This reinforces the need to keep measurement scope explicit.

---

# Part XXIX - Recommended Paper Claim Set

## 73. Strong Claims That Are Supported

Within the current V1 evidence scope, the paper can safely state that:

1. V1 is a documented AVX-512-oriented Monte Carlo execution architecture with a separated Back seed-preparation layer and a lightweight Front hot path.
2. The V1 implementation has a frozen source identity and an implementation-fidelity ledger sufficient for bit-for-bit reproduction at the documented boundaries.
3. The tested MC acceleration options preserved the benchmarked sample count and did not show estimator-breaking behavior in the measured Finance/Pharma scope.
4. The current audit explicitly distinguishes logical sample throughput from fresh/independent sample production.

5. Independent-worker scaling was measured on the dedicated three-core experiment under the stated AMD environment.
6. The current evidence supports further external statistical qualification and same-host comparison, but does not claim those tasks are already complete.

## 74. Claims That Require Qualification Before Use

The following statements should not be made without additional evidence or careful qualification:

"cryptographically secure"  
 "proven independent"  
 "IID by construction"  
 "passes PractRand" unless the suite was actually run  
 "passes TestU01" unless the suite was actually run  
 "faster than xoshiro" without same-work comparison  
 "faster than oneMKL" without same-work comparison  
 "46.602 G independent samples/s"  
 "8.948 T/s on a 192-core CPU"

In particular, the 192-core number is an idealized envelope, not a forecast or measurement.

---

## Part XXX - Reader's Execution Path

### 75. How a Third Party Should Approach the Project

A reader encountering V1 for the first time should use the following order:

1. README / scope and license
2. build / verify
3. representative example
4. architecture overview
5. mathematical specification
6. exact implementation ledger
7. benchmark results
8. anti-reuse and interpretation boundaries
9. known limitations / pending external qualification

This order minimizes the risk of interpreting an isolated performance figure or a development-history experiment as the current V1 definition.

## Final Status

V1 is the frozen reference implementation for the current paper / POC cycle.

The code identity, numerical path, measurement boundaries, and remaining external qualification tasks are recorded explicitly.

V2 remains research history and must not silently replace V1 in published measurements.

---

## Part XXX - Treatment of Publicly Published Benchmarks

### 76. Separate Same-Condition Comparisons from Reference Values

Public benchmark values should be used first as a reference landscape, not as automatic evidence of relative superiority.

A same-condition comparison requires matching at least:

CPU / hardware  
compiler / build flags  
ISA  
sample definition  
precision  
output target  
workload  
measurement boundary  
timing method  
stream / seed policy

Only after these are aligned should a direct speed ratio be reported.

### 77. Public Speed References Confirmed at This Stage

The following public raw-RNG references are retained for context. Their reported metrics are kept in the original benchmark scope.

Reference	Reported metric	Target	Conditions	Treatment here
xoshiro256+	0.61 ns / 64 bits	raw RNG emission	i7-12700KF, GCC 12.2.1	published raw-RNG reference
xoshiro256++	0.75 ns / 64 bits	raw RNG emission	same published shootout	published raw-RNG reference
SFMT19937	0.93 ns / 64 bits	raw RNG emission	same published shootout	published raw-RNG reference
PCG64-DXSM	1.11 ns / 64 bits	raw RNG emission	same published shootout	published raw-RNG reference
MT19937-64	1.36 ns / 64 bits	raw RNG emission	same published shootout	published raw-RNG reference

The published shootout itself explains that its speed metric is the time required to emit 64 random bits. Therefore, when these are converted to G outputs/s and shown beside V1, **the difference in measurement scope must be explicit.**

The purpose of this chapter is not to create a direct “winner” table. It is to show the existing raw-PRNG emission scale against which V1’s end-to-end MC throughput can be contextualized.

#### Philox / Threefry / MCKL

MCKL 0.1.0 reports RNG performance on an Intel Core i7-4960HQ in cycles/byte, including batch paths such as Philox4x32 at 0.63 cpB, Threefry4x32 at 0.98 cpB, MKL Philox4x32x10 at 0.77 cpB, and

MKL SFMT19937 at 0.22 cpB. These are historical 2019 measurements and are not same-condition measurements with V1's current AVX-512 MC pipeline.

### AVX-512 Vectorized MT/SFMT

The public VMT19937 repository reports up to 5,742 M/s for an AVX-512 vectorial 32-bit MT19937 path, up to 2,865 M/s for 64-bit MT19937, and up to 10,943 M/s for SFMT19937. These are useful as public AVX-512 raw-RNG references but are not treated as same-host V1 comparisons.

## 78. Public Normal-Generation Reference Landscape

Normal generation is shown in a separate reference table.

A 2026 public PhiloxRNG.jl benchmark reports on an AMD Ryzen 7 9800X3D that Philox4x32 + fast Box-Muller reaches 0.898 ns/value for CPU randn! Float32 and 2.009 ns/value for Float64. This is a Julia implementation and has a different architecture from V1's integer LUT Normal-v9, so it is a contextual reference only.

A public Go Ziggurat benchmark reports on an AMD Ryzen 9 5900X 10.58 ns/op for Normal / Ziggurat + xoroshiro128+ and 3.868 ns/op for Ziggurat (Symmetric) + xoroshiro128+. These measurements include Go runtime/library implementation effects and the selected PRNG, so they are not direct V1 comparison values.

MCKL also provides older public vectorized Normal(0,1) references on an Intel Core i7-4960HQ: VMF 17.6 cpE, VML 18.4 cpE, and MKL 15.4 cpE.

## 79. How to Compare V1 - Separate Raw RNG Speed from End-to-End MC Speed

### 79A. Treatment of Cycles/Sample

Cycles/Sample or CPE can be useful normalized metrics. However, they should be treated as direct comparison metrics only when **the same sample definition, timing boundary, precision, and workload are used**.

For V1, the current formal record does not preserve the effective cycle count required to convert the 46.602 G logical MC samples/s result into a reproducible CPE value. Therefore, this book does not reverse-calculate 0.064 cycles/sample by assuming a 3.0 GHz clock.

Intel oneMKL also uses CPE as a performance metric, but its own materials state that performance depends on hardware architecture, underlying BRNG, transformation method, vector length, and distribution parameters. CPE is therefore not a magic unit that eliminates environment differences; it is a normalization aid when measurement definitions are aligned.

This book keeps three measurement levels separate:

- A. RNG-core / raw output
- B. Normal-generation throughput
- C. End-to-end MC logical throughput

Cycles/sample and speedup should be compared directly only within a matched level and matched workload.

V1 representative values are:

FL0AT32 reference	= 22.962 Gsamples/s
R2 reference	= 43.514 Gsamples/s
strict one-core peak	= 46.602 Gsamples/s

The **46.602 Gsamples/s figure is not a raw random-output generation rate**. The strict one-core complete benchmark measures the engine/MC path on one CPU core:

```

Back A / Back B
 ↓
Front recurrence
 ↓
Normal v9
 ↓
4-way fanout
 ↓
PaperView / final cultivation
 ↓
R2_FLOAT32 MC consumer
 ↓
logical MC samples

```

The timed peak does not cross a generation boundary and therefore does not stress seed-exchange frequency, but it does measure the one-core-complete configuration in which Front + Normal + PaperView + Final-Cultivation + MC + Back A/B are maintained on the same core.

By contrast, the public xoshiro / PCG / MT / SFMT values above are defined as raw 64-bit random-output emission. Thus, even when both sets of numbers are displayed in G/s, **the workload semantics are different.**

This distinction should be stated explicitly:

```

Existing published values
 = raw PRNG output emission reference
 = "how fast raw random output is emitted"

```

```

UltraFastRng512 V1
 = end-to-end MC logical throughput
 = "how fast the RNG -> Normal -> fanout -> cultivation/layout -> MC path runs"

```

This supports a contextual performance-scale comparison, but not a direct claim such as "V1 is X times faster than xoshiro." Such a claim requires same-host / same-build / same-sample-count / same-precision / same-MC-work testing.

The paper should therefore retain the same three layers:

- A. RNG-core / raw-output throughput
- B. Normal-generation throughput
- C. End-to-end MC throughput

Published raw-PRNG figures should be presented as A-level contextual references, while V1's 22.962 / 43.514 / 46.602 Gsamples/s values should be labeled according to their actual benchmark boundaries.

## Part XXXI - Three-Core Scaling Experiment

### 80. From One-Core-Complete to Three Cores

V1's baseline architecture is one-core-complete execution. In a separate experiment, additional independent MC workers were placed on additional CPU cores to measure aggregate throughput scaling.

The experiment used the same AMD EPYC 9V74 / 3-vCPU environment with `-O3 -std=c11 -pthread -march=znver4`.

## FLOAT32

1-core median	12.7485	Gsamples/s
3-core clean median	38.656	Gsamples/s
3-core best	39.319	Gsamples/s
scaling	3.032x	

Adding independent workers produced nearly threefold aggregate throughput on the measured clean subset.

## R2\_FLOAT32

1-core median	12.8395	Gsamples/s
3-core clean	36.959	Gsamples/s
3-core best	38.740	Gsamples/s
scaling	2.878x	

R2 also scaled strongly, although not as close to threefold as FLOAT32.

This is not a claim that moving Back to other CPUs is what produced the speedup.

The result should be described as **parallel scaling from adding independent workers**.

## 81. Distinction from Back Placement Experiments

Separately, a single engine was tested with Front on CPU0 and Back A/B moved to CPU1/CPU2:

strict one-core complete median	= 11.763084	Gsamples/s
split-Back median	= 11.696659	Gsamples/s

The one-core-complete condition was approximately 0.568% higher in median throughput.

Therefore:

one-core complete	
+---->	independent workers × 3 -> near-linear aggregate scaling

Back delegation within one engine	
+---->	not the demonstrated source of the scaling benefit

## 82. Wording for the Paper

A safe formulation is:

V1 is executable in a one-core-complete configuration. When independent MC workers were replicated onto multiple cores, the measured environment produced approximately 3.03× three-core throughput for FLOAT32 and approximately 2.88× for R2\_FLOAT32.

This is an engineering benchmark in the specified VM environment, not a general NUMA-scaling theorem.

---

## Part XXXII - Current Comparison Strategy

### 83. Recommended Paper Figures

Performance comparisons should be divided into three separate figures:

Figure A: V1 internal performance  
46.6 peak / 43.5 R2 / 23.0 FLOAT32

Figure B: Published reference landscape  
xoshiro / PCG / MT / SFMT / Philox / Threefry / Ziggurat  
original units + CPU/compiler metadata

Figure C: Same-host direct benchmark  
V1 vs selected baseline(s)  
same CPU / same compiler / same sample count / same precision / same MC work

This structure allows the paper's context section to be completed before the same-condition baseline measurements are available, while keeping the eventual direct-comparison figure separate.

## 84. Current Conclusion

There is value in reporting public benchmarks as contextual references. Their role is to show **the scope, metric, and hardware conditions under which comparison targets were reported**, not to determine a direct speed ranking for V1.

For example, the public xoshiro shootout reports the time to emit 64 random bits under an Intel Core i7-12700KF + GCC 12.2.1 environment, while Intel oneMKL Gaussian tables report CPE with explicit BRNG and transformation distinctions. These are different measurement boundaries from V1's end-to-end MC metric. [Vigna/Blackman published RNG shootout] [Intel oneMKL performance documentation]

The comparison table should therefore function as a **scope map**, not as a direct ranking. Only values obtained under genuinely matched conditions should be promoted into the paper-grade direct benchmark table.

---

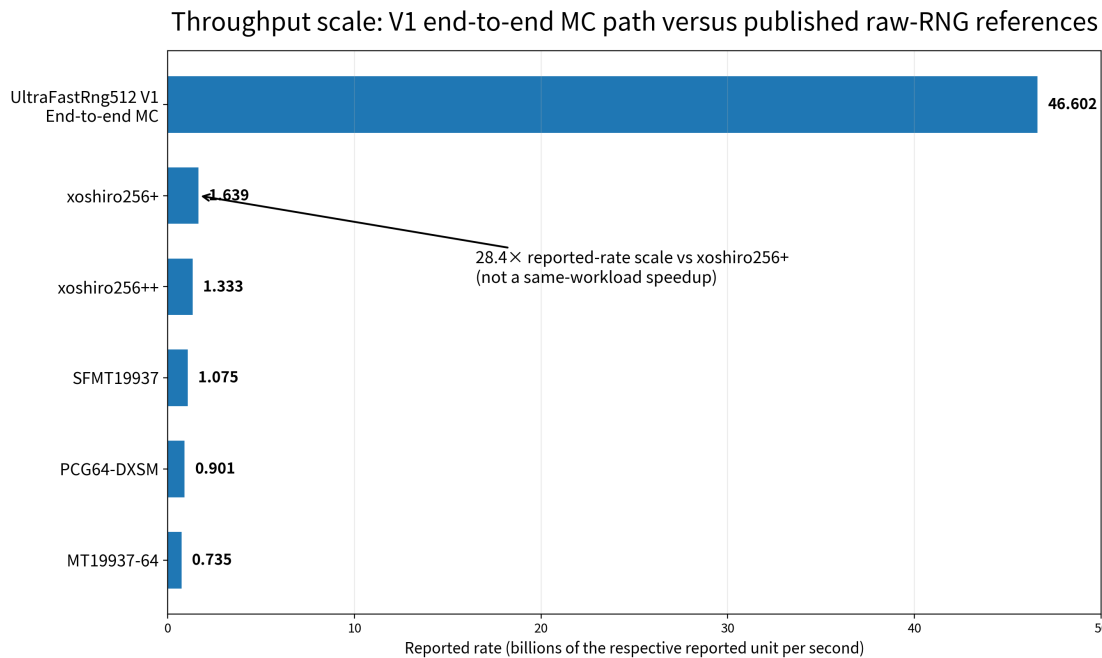
# Part XXXIII - Comparison Figures for Paper / Deconstruction Book

## 85. Figure Strategy

V1's performance, quality, and reference values should use common display units where possible without ever conflating the underlying measurement boundaries.

Figure 00 Scope and measurement-scope overview  
Figure 00A Reported throughput scale  
Figure 00B Measurement-environment context  
Figure 01 Scope-aware V1 throughput scale  
Figure 02 Published RNG reference landscape  
Figure 03 Published Normal-generation reference landscape  
Figure 04 V1 Normal tail: theory vs measurement  
Figure 05 Raw Front-state 1024-position p-value histogram  
Figure 06 Raw Front-state 1024-position p-value heatmap  
Figure 07 Raw Front-state battery overview  
Figure 08 One-core -> three-core independent-worker scaling

## 86. V1 Measured Throughput Figure



Published references: 64-bit raw-output rates converted from ns/output. V1: logical MC samples/s. Different units and environments; use for scale/context, not as a same-workload benchmark.

*Figure 01. Scope-aware throughput scale. V1 is shown as end-to-end logical MC throughput; published references are raw RNG output rates converted from their reported ns/64-bit-output figures. The common axis is a visual scale aid, not a same-workload speed comparison.*

01\_V1\_throughput . png represents the V1 internal comparison where the same logical-sample unit can be used:

46.602 Gsamples/s    strict one-core complete peak  
43.514 Gsamples/s    R2\_FLOAT32 reference median  
22.962 Gsamples/s    FLOAT32 reference median

46.602 is the long fixed-generation peak condition, while the latter two are five-run reference medians. The 46.602 headline is logical MC throughput under FIXED generation and includes starter reuse and cultivation amortization. Absolute throughput depends on CPU and VM environment and is not a portable CPU constant.

## 86A. Measurement Environment and Repeatability

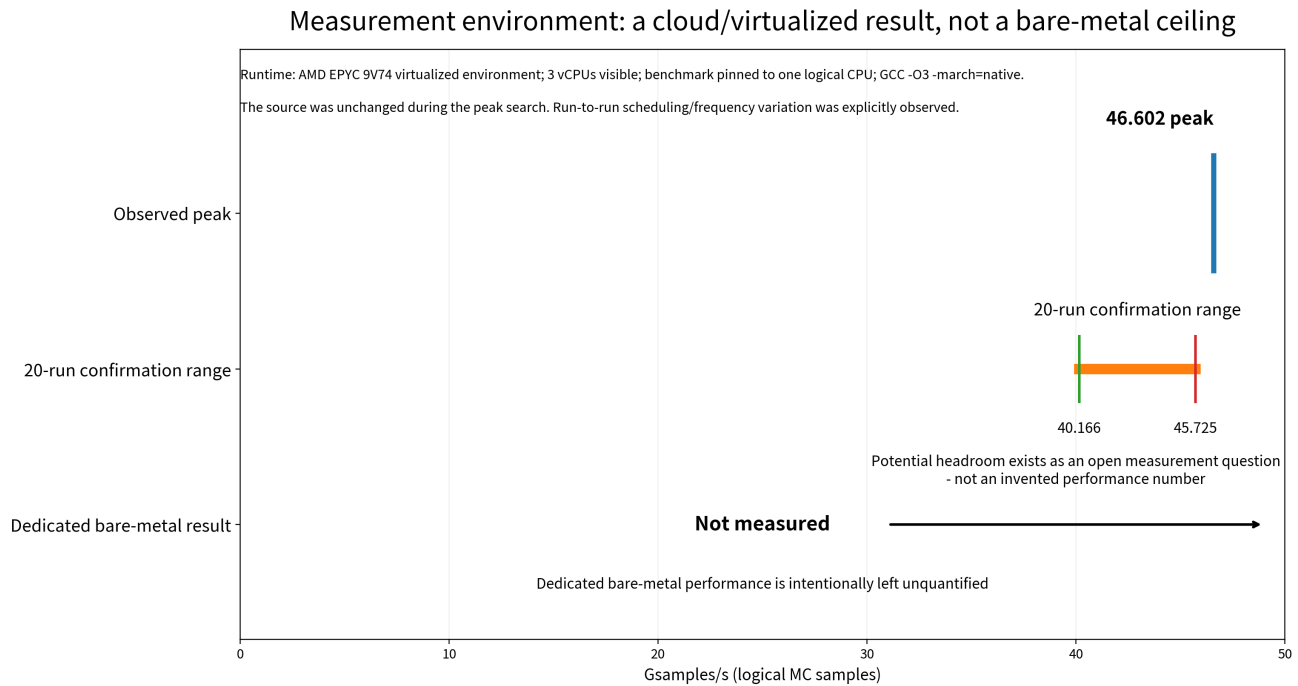
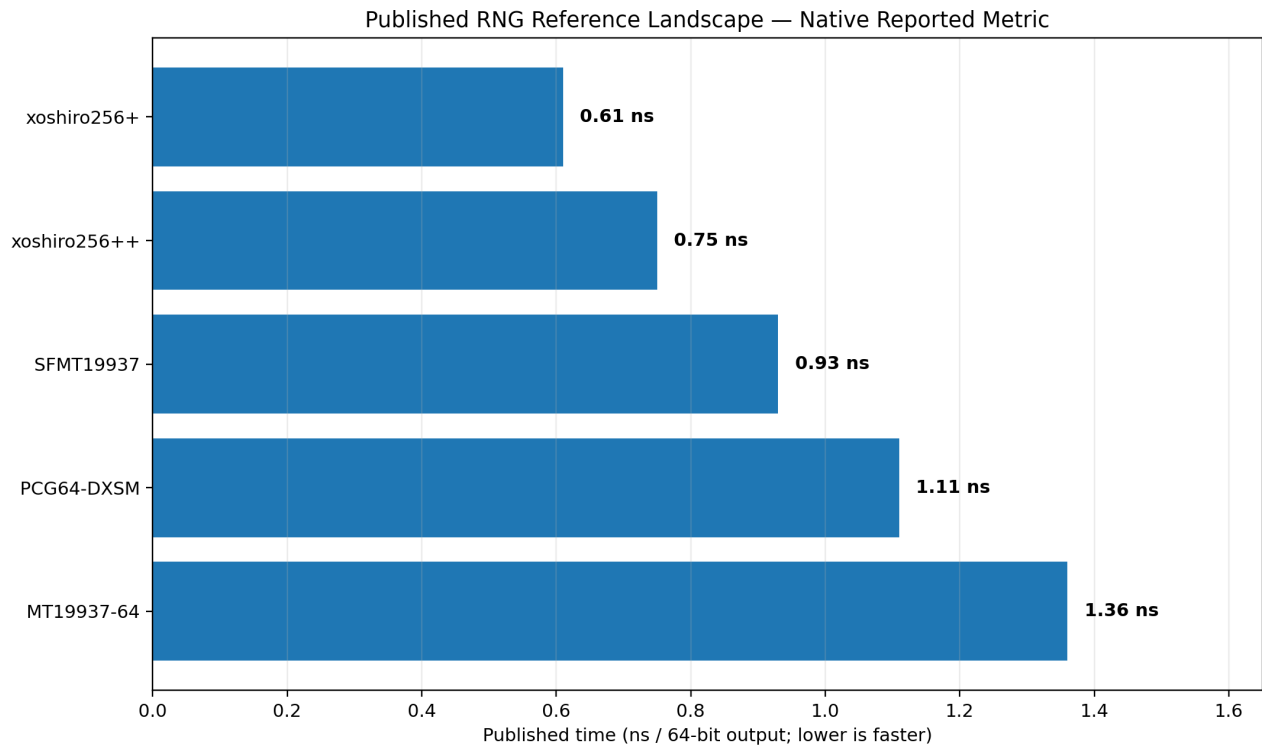


Figure 01A. Peak/repeatability context. The 46.602 Gsamples/s value is an observed peak in a virtualized environment; a dedicated bare-metal result is not yet measured.

CPU	= AMD EPYC 9V74 80-Core Processor
vCPUs visible	= 3
Benchmark placement	= exactly one logical CPU
Compiler family	= GCC
Build flags	= -O3 -std=c11 -pthread -march=native
Exact GCC version	= not preserved in current benchmark record
ISA	= AVX-512F/BW/DQ/VL, GFNI, VAES, VNNI and related extensions
20-run confirmation range	= 40.166 - 45.725 Gsamples/s
Observed peak	= 46.602 Gsamples/s

These measurements do not represent a dedicated bare-metal ceiling. Because no dedicated bare-metal upper-bound result has been measured, this book does not estimate an additional throughput value.

## 87. Published RNG Reference Landscape



Reference-only chart. Values come from a public PRNG shootout and are not same-host comparisons to V1.

*Figure 02. Published RNG reference landscape. Metrics remain in their original benchmark definitions; they are contextual references, not same-host end-to-end comparisons.*

02\_published\_rng\_reference.png presents representative published **raw RNG emission** results. Public 64-bit output values can be converted into G outputs/s using the conversion table above, but this does not make them the same workload as V1's end-to-end MC throughput.

The Vigna/Blackman public shootout reports, for example, xoshiro256+ at 0.61 ns/64-bit output, xoshiro256++ at 0.75, PCG64-DXSM at 1.11, MT19937-64 at 1.36, and SFMT19937 at 0.93. These are not same-host comparisons with V1. [Vigna/Blackman published RNG shootout]

MCKL also publishes batch cycles/byte values for Philox4x32, Threefry4x32, MKL Philox, and MKL SFMT. These use different metrics and environments and are therefore not mixed into a direct V1 comparison. [MCKL published benchmark documentation]

### 87A. How Readers Should Read the Comparison First

The comparison numbers should be read as follows:

Published PRNG 1.639 G 64-bit outputs/s

↓

raw random output generation rate

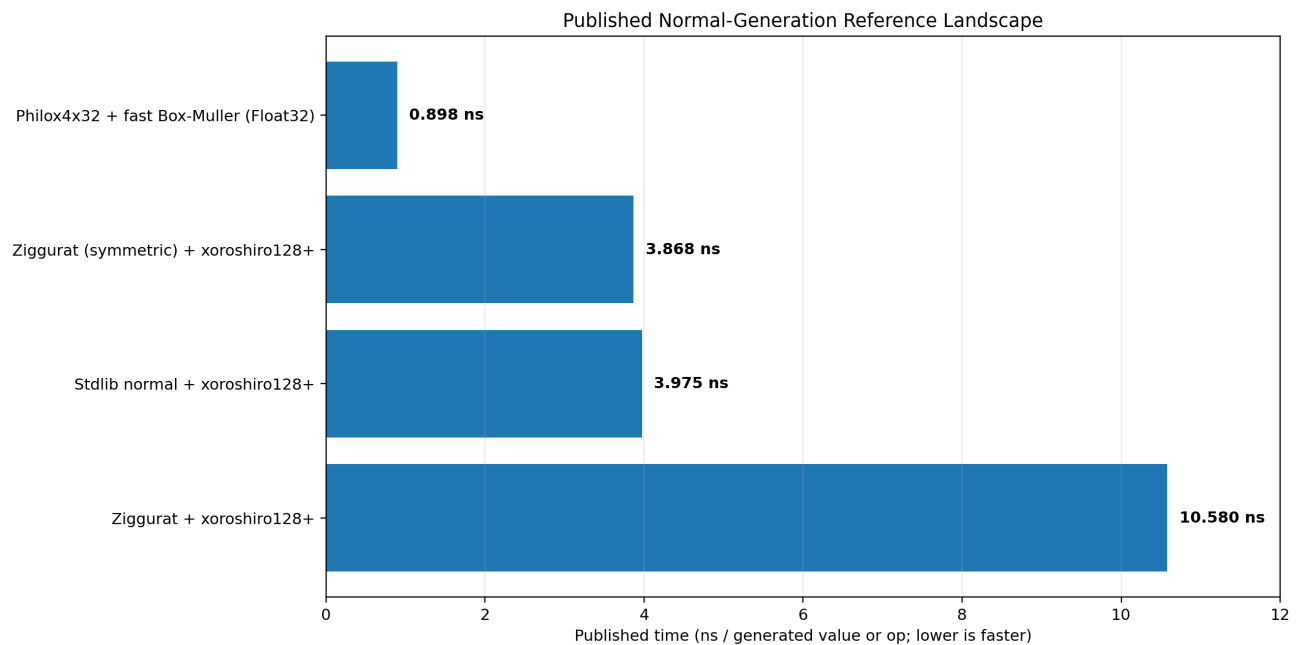
V1 46.602 G logical samples/s

↓

Front + Normal + PaperView + Final-Cultivation + MC + Back

Both can be expressed in a  $10^9/\text{s}$ -style display, which helps visualize performance scale. However, the "1 output" in the former and "1 logical MC sample" in the latter do not represent the same amount of work. This difference must be explained in the main text rather than hidden in a footnote.

## 88. Published Normal Reference Landscape



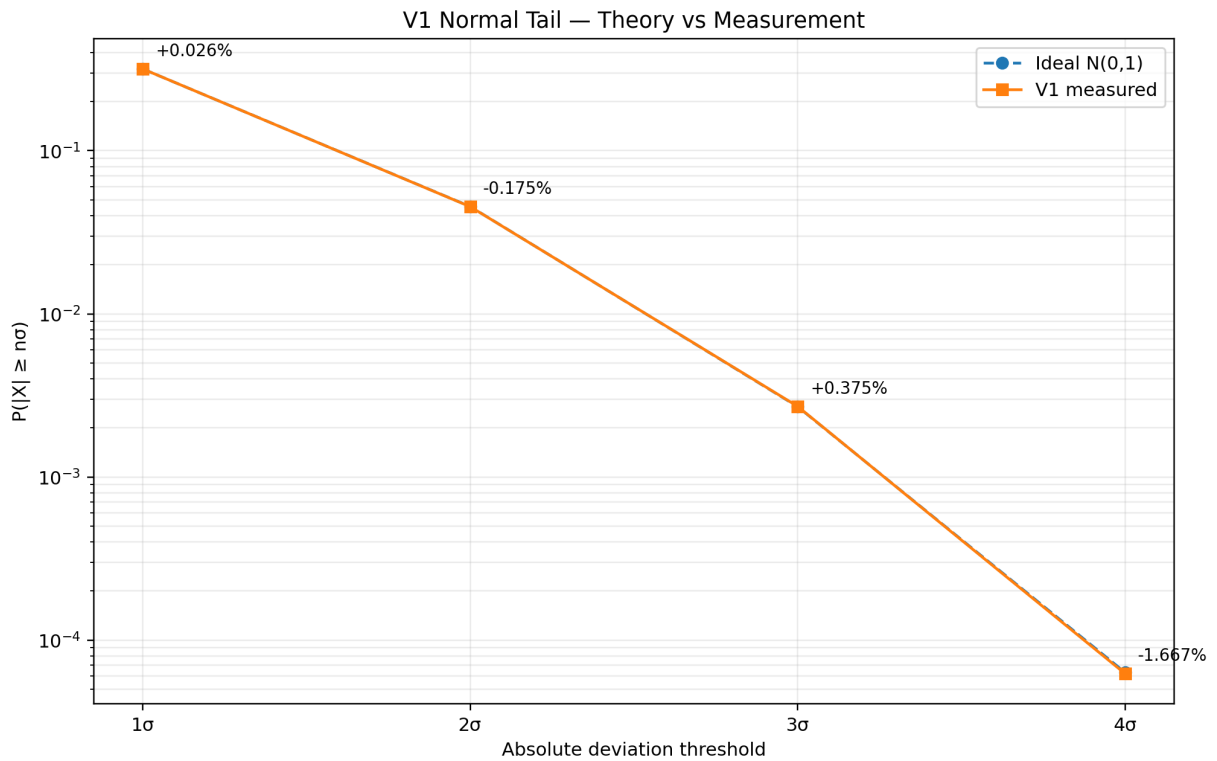
Reference-only. Sources, hardware, and exact operation definitions differ; not a direct V1 comparison.

*Figure 03. Published Normal-generation reference landscape.*

`03_published_normal_reference.png` presents published Normal-generation reference values using their native metrics.

A public AMD Ryzen 9 5900X benchmark for Ziggurat + xoroshiro128+ reports 10.58 ns/op for Normal and 3.868 ns/op for symmetric Ziggurat. These are not direct comparisons with V1 Normal-v9. [published Ziggurat benchmark]

## 89. Normal Tail Figure



V1 aggregate: 134,217,728 x/y pairs.  $4\sigma$  relative deviation = -1.667%;  $4\sigma$  event counts are finite-sample sensitive.

Figure 04. V1 aggregate Normal tail versus the ideal standard-normal reference.

04\_V1\_normal\_tail.png compares the aggregate V1 Normal tail against the ideal standard-normal tail.

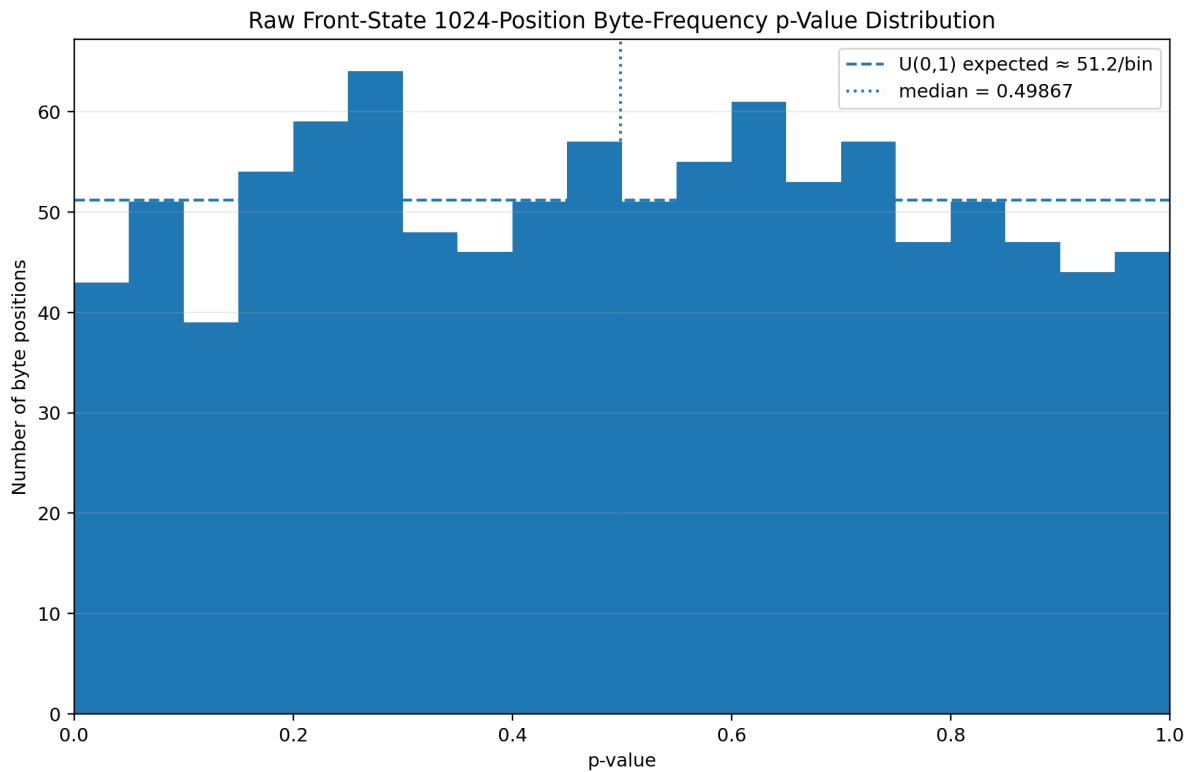
Values used:

$1\sigma$	V1	0.317391440	ideal	0.317310508
$2\sigma$	V1	0.0454207063	ideal	0.045500264
$3\sigma$	V1	0.00270991027	ideal	0.002699796
$4\sigma$	V1	0.0000622868538	ideal	0.0000633425

$4\sigma$  relative deviation = -1.667%.

The x-axis is linear and the y-axis logarithmic so that the low-probability region near  $4\sigma$  remains visible.

## 90. Raw Front-State p-Value Histogram



Exact 512 MiB stream. min p = 0.0016696; p<0.01: 8/1024; p<0.001: 0/1024.

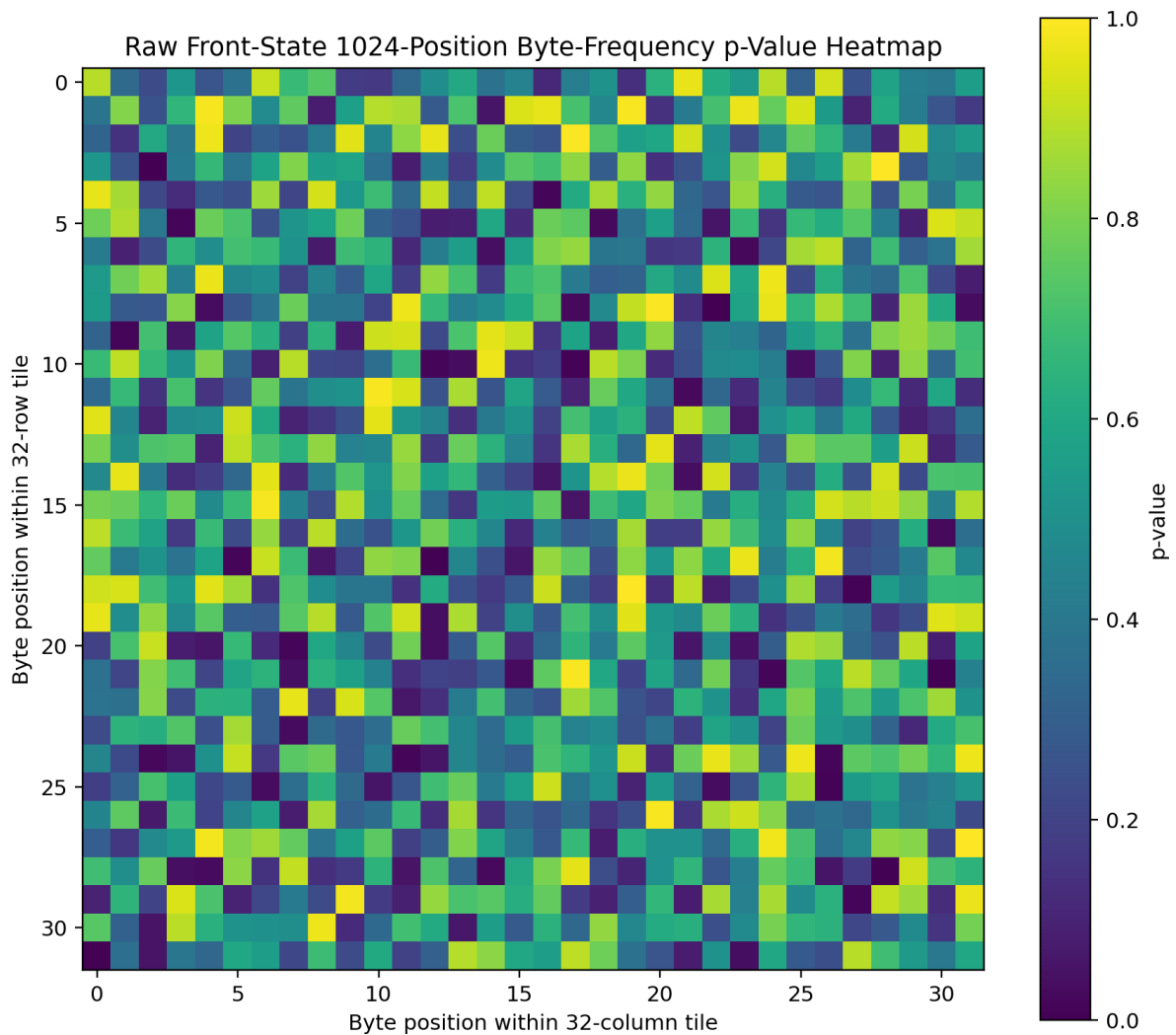
Figure 05. Empirical p-value histogram from the fixed raw Front-state stream.

05\_1024\_position\_pvalue\_histogram.png is derived from a fixed 512 MiB raw Front-state stream in which 1024 byte positions were evaluated separately.

minimum p	= 0.0016695553
median p	= 0.4986725282
p < 0.01	= 8 / 1024
p < 0.001	= 0 / 1024

The individual p-values were recomputed from the original stream and stored without changing the original 512 MiB stream SHA.

## 91. Raw Front-State p-Value Heatmap



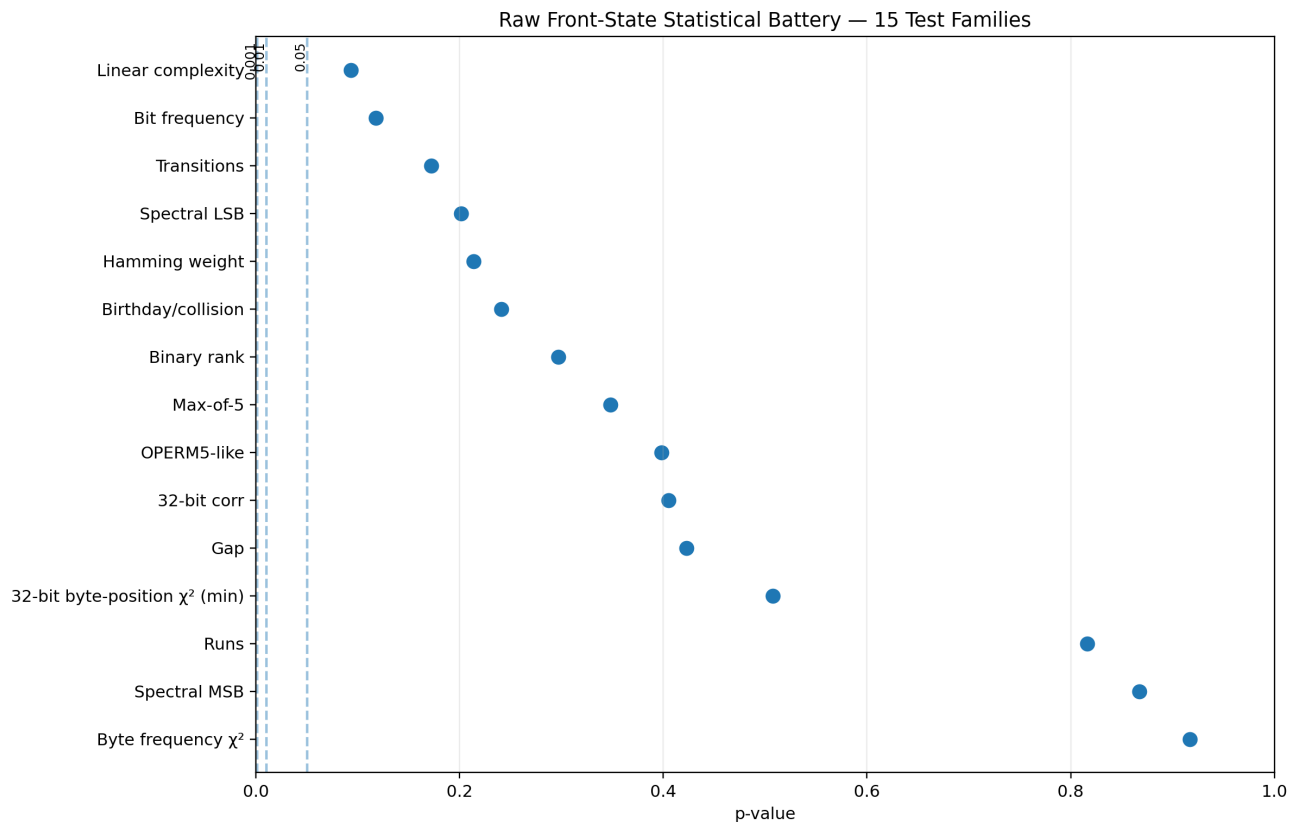
Each cell is one of the 1024 exact byte positions in the 1024-byte resident state. Median = 0.49867; min = 0.0016696.

*Figure 06. Same 1024 empirical p-values rendered as a 32×32 heatmap; no new test is introduced.*

`06_1024_position_pvalue_heatmap.png` simply rearranges the same 1024 measured p-values into a 32×32 display and does not introduce a new statistical test.

The histogram and heatmap are therefore two visualizations of the same data.

## 92. Battery Overview



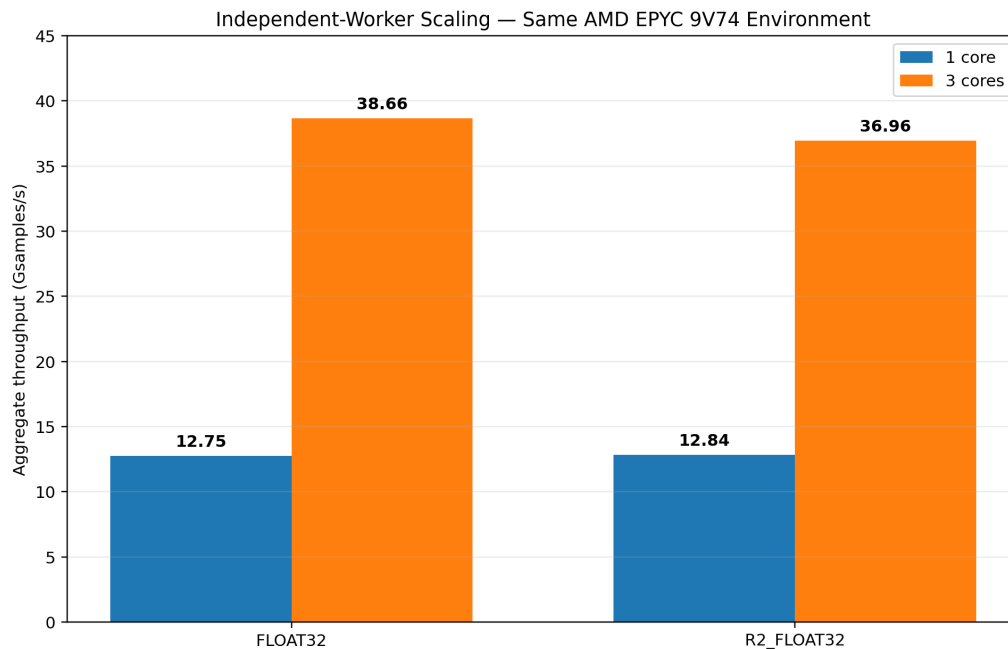
512 MiB raw Front-state stream. The chart is an empirical screen;  $p > 0.05$  is not itself a certification.

*Figure 07. Statistical battery overview.*

`07_statistical_battery_overview.png` summarizes the principal surrogate test families, including byte/bit frequency, transitions, runs, word correlation, collision, permutation, gap, max-of-5, binary rank, linear complexity, and spectral tests.

This is not a simplistic “ $p > 0.05$  means pass” chart. It is an overview for identifying extreme outliers or coverage imbalance across the tested domains.

## 93. Three-Core Scaling



FLOAT32 clean scaling = 3.032x; R2 clean scaling = 2.878x. Independent MC workers were used; this is not a claim that moving Back alone off-core accelerates one engine.

Figure 08. One-core to three-core independent-worker scaling.

08\_three\_core\_scaling.png shows independent MC-worker scaling in the same AMD EPYC 9V74 environment.

FLOAT32	12.7485	->	38.656	Gsamples/s	3.032x
R2	12.8395	->	36.959	Gsamples/s	2.878x

This figure does not claim that “moving Back to another CPU makes V1 faster.” It shows aggregate throughput scaling from adding independent workers.

## 94. Same-Condition Comparison Remains the Paper-Grade Target

Published values provide a comparison map. Paper-grade performance claims should be established separately through same-host direct benchmarks.

Required matching conditions:

- same CPU
- same compiler / flags
- same ISA
- same sample count
- same precision
- same output target
- same timing boundary
- same MC work
- same stream policy / seed policy
- same validation of generated sample count

### 94A. Minimal External Reference Table - First Align the Scope

Reference	Reported metric	Target	Conditions	Treatment here
xoshiro256+	0.61 ns / 64 bits	raw RNG emission	i7-12700KF, GCC 12.2.1	published raw-RNG reference
oneMKL Gaussian + MT19937	4.29 CPE/element (single, Box-Muller)	Gaussian generation	Xeon Platinum 8280L, vector length 1000	published Normal reference
oneMKL Gaussian + MT19937	2.44 CPE/element (single, ICDF)	Gaussian generation	Xeon Platinum 8280L, vector length 1000	published Normal reference
UltraFastRng512 V1	46.602 G logical samples/s	end-to-end MC path	AMD EPYC 9V74 VM, 1 logical CPU	measured value from this work

This table intentionally does not turn different units into one performance ranking. Its purpose is to make the difference in **scope—raw RNG / Normal generation / end-to-end MC—visible in a single place**.

The oneMKL CPE values come from Intel public performance data and include older Xeon generations; the xoshiro value likewise uses a different CPU/compiler environment. Neither is sufficient for a direct V1 comparison, and neither should be used alone to derive a V1 superiority claim.

#### 94B. Table to Publish After Same-Host Measurement

RNG core:				
V1 core	baseline core	cycles/output	same host	
Normal:				
V1 Normal v9	oneMKL / Ziggurat / ...	cycles/sample	same host	
End-to-end MC:				
V1 full path	baseline MC pipeline	logical MC/s	same work	

Until a baseline in the corresponding row is measured, leave the cell blank rather than filling it with a published number. **A blank comparison cell is not a defect; it is an accurate indication that the measurement has not yet been performed.**

#### 95. Figure Package

The complete comparison-figure package is fixed as:

UltraFastRng512\_V1\_3CORE\_REVISIED\_PACKAGE\_20260929\_FINAL.zip

The figures do not change the V1 algorithm. They are part of the paper/deconstruction presentation layer.

## Part XXXIV - Three-Core Benchmark Baseline Clarification

#### 96. Why the Three-Core Experiment Uses 12.7485 Gsamples/s as Its One-Worker Reference

The 12.7485 Gsamples/s value must not be interpreted as a regression from the approximately 23 Gsamples/s V1 reference measured on the Intel Xeon 8573C environment.

The two measurements answer different questions and were obtained on different CPU microarchitectures and compiler targets.

Reference throughput benchmark  
Intel Xeon Platinum 8573C  
-march=icelake-server  
V1 FLOAT32 reference median  $\approx$  22.962 Gsamples/s

Dedicated three-core scaling benchmark  
AMD EPYC 9V74 / Zen 4  
-march=znver4  
1-worker reference median = 12.7485 Gsamples/s  
3-worker clean full-active median = 38.656 Gsamples/s

The dedicated three-core experiment answers one question: does aggregate MC throughput increase approximately in proportion to the number of independent CPU cores, providing measured evidence for worker-level scaling? It was not a peak-speed search and does not replace the separately observed one-core-complete peak.

## 97. CPU Microarchitecture Explanation

AMD's official 4th Generation EPYC architecture documentation states that Zen 4 uses a 512-bit AVX register file with 256-bit internal data paths, with 512-bit AVX operations represented as two 256-bit vectors executed in sequential clock cycles. AMD describes the EPYC 9004 implementation as a "double-pumping" 256-bit datapath. [AMD EPYC architecture documentation]

This matters to V1 because the workload makes heavy use of 512-bit AVX-512 integer/vector operations, including GFNI and integer multiply-add style operations in the Normal path.

The appropriate interpretation is therefore:

Intel Ice Lake-class execution model  
-> native 512-bit AVX-512 datapath resources are available

AMD Zen 4 EPYC 9004 execution model  
-> 512-bit AVX register width  
-> 256-bit internal datapaths  
-> 512-bit operations are executed as two sequential 256-bit pieces

This provides a plausible microarchitectural explanation for why the same 512-bit-heavy V1 workload can have substantially lower per-core throughput on the AMD Zen 4 environment than on the earlier Intel Ice Lake environment.

The benchmark alone does not prove that this mechanism explains the entire 44.5% throughput difference. Exact instruction-level throughput also depends on instruction mix, scheduler behavior, cache state, frequency, compiler code generation, VM scheduling, and other platform conditions.

## 98. The Apparent "Drop" Is Therefore Not a V1 Algorithm Regression

The relevant comparison for the three-core experiment is not:

23.0 -> 12.75 -> therefore V1 became slower

but:

Different CPU / compiler target

```

|
+--> AMD Zen 4 one-worker reference = 12.7485
 |
 +--> 3 independent workers
 |
 +--> 38.656 clean aggregate
 = 3.032x scaling
```

The 12.7485 Gsamples/s figure is the local baseline of the dedicated scaling experiment.

## 99. VM Scheduling and Raw-Run Variability

The three-worker test also exposed runs with strong VM scheduling interference:

20.348  
14.709  
14.918 Gsamples/s

The raw 10-run median was therefore 37.016 Gsamples/s, while the six full-active runs were:

38.393  
38.956  
35.668  
39.319  
38.364  
38.918

with a clean full-active median of 38.656 Gsamples/s.

The clean subset is used specifically to assess whether all independent workers can remain near their single-worker throughput. It is not a cherry-picked “best six” for a speed claim; excluded runs show visible external VM scheduling interference and remain in the raw record.

## 100. Recommended Paper Wording

The one-core and three-core measurements were performed as separate experiments under different CPU environments. The peak/reference measurements used an Intel Xeon Platinum 8573C target (`-march=icelake-server`), whereas the dedicated parallel-scaling experiment used an AMD EPYC 9V74 (Zen 4) environment with `-march=znver4`. Accordingly, the 12.7485 Gsamples/s one-worker value in the scaling experiment is a same-environment baseline, not a replacement for the higher-throughput Intel reference. AMD documents Zen 4 as using a 512-bit AVX register file backed by 256-bit internal datapaths, with 512-bit operations executed as two sequential 256-bit pieces; this provides a plausible architectural explanation for the lower per-core throughput of this 512-bit-heavy workload, although the benchmark does not isolate this factor from all other platform effects. In the same AMD environment, three independent workers reached 38.656 Gsamples/s on the clean full-active subset, corresponding to 3.032× scaling.

## 101. Forward-Looking Scaling Statement - Use the Measured Peak as the Envelope

The observed 3.032× three-core scaling demonstrates that independent workers can scale close to linearly in the tested AMD environment. The three-core result is evidence for **worker-level scalability**, not the source of the 192-core peak number.

For the forward-looking 192-core figure, the appropriate starting point is the separately measured **V1 one-core-complete peak of 46.602 Gsamples/s**. Under the idealized assumption that the same peak is sustained independently by 192 CPU workers:

46.602 Gsamples/s × 192  
= 8,947.584 Gsamples/s  
= 8.947584 T logical samples/s

This is a **conditional peak envelope**, not a benchmark result. Large-system NUMA effects, memory hierarchy, frequency behavior, scheduling, topology, and other system-level effects can reduce actual throughput.

A paper may therefore state:

The measured one-core peak of 46.602 Gsamples/s, combined with the observed near-linear three-core independent-worker scaling, motivates investigation of high-core-count CPU systems. If a 192-core deployment could preserve the measured one-core peak under ideal linear scaling, the resulting aggregate envelope would be approximately 8.95 T logical samples/s. This value is a thought experiment rather than a measurement or prediction. It does not establish parity with, or superiority to, any multi-GPU system.

## 102. Latency / Energy / TCO Hypothesis - Explicitly Unmeasured

V1's CPU-centric execution model may be attractive for workloads where low host-device transfer overhead, resident-memory execution, or tight control-plane integration matter.

A future comparison should measure at least:

throughput  
 latency / time-to-first-result  
 energy per logical sample  
 energy-to-solution  
 memory traffic  
 host-device transfer volume  
 software deployment complexity

Only these measurements can establish whether V1 has a practical latency or energy-efficiency advantage relative to a particular GPU implementation. At the present stage, such an advantage remains a research hypothesis, not an established result.

---

## Part XXXV - Mathematical Core / Full Formalization

### 103. Purpose and Position of This Chapter

The chapters above explain V1 in the order of design philosophy, implementation structure, quality results, and performance results. A third party who wants to trace the actual computation, however, needs more than explanatory pseudocode: the state space, recurrence equations, vector operations, and Normal-domain mapping need to be presented as a single mathematical skeleton.

This chapter therefore does not merely summarize a separate mathematical specification. It expands the mathematical description until a reader can trace V1's numerical data path using the Deconstruction Book itself.

The full formalization does not need to be reproduced in the paper. The paper should extract the minimal core while this book retains the complete equations, constants, boundary conditions, and interpretation limits.

The following four regions remain distinct:

- (1) Front / Back state transition
- (2) Normal-domain mapping and MC equations
- (3) AVX-512 vector algebra / lane mapping
- (4) State-space size, period bounds, and what is NOT proved

### 104. Core Mathematical Objects

Represent each resident state as:

$$m_j \in (\mathbb{Z}/2^{64}\mathbb{Z})^8, \quad j = 0, 1, \dots, 15.$$

One state therefore consists of eight 64-bit words, or 64 bytes.

The complete Front resident state is:

$$S = (m_0, m_1, \dots, m_{15}),$$

with total width:

$$16 \times 64 \text{ bytes} = 1024 \text{ bytes} = 8192 \text{ bits}.$$

Let the parent triple be:

$$(a, b, c) \in ((\mathbb{Z}/2^{64}\mathbb{Z})^8)^3$$

and let generation be  $g$ .

V1 prepares parent lineages A/B/C on the Back side and selects their order by generation:

$$\pi(g) = \left\lfloor \frac{g}{2} \right\rfloor \bmod 6.$$

The six orderings are:

- 0: A B C
- 1: A C B
- 2: B A C
- 3: B C A
- 4: C A B
- 5: C B A

This six-way permutation provides generation-varying relationships among three same-profile lineages; it does not increase the number of parents.

## 105. Front Parent Recurrence - Formal State Transition

The operation mode selected by generation is:

$$\mu(g) = \left\lfloor \frac{g}{2} \right\rfloor \bmod 3.$$

All arithmetic is evaluated modulo  $2^{64}$  per word, and  $\text{ROTL}_r$  denotes a 64-bit left rotation.

### Mode 0

$$\begin{aligned} p_0 &= a \oplus \text{ROTL}_{17}(b), \\ p_1 &= b + \text{ROTL}_{29}(c), \\ p_2 &= c + \text{ROTL}_{41}(a). \end{aligned}$$

### Mode 1

$$\begin{aligned} p_0 &= a + \text{ROTL}_{17}(b), \\ p_1 &= b \oplus \text{ROTL}_{29}(c), \\ p_2 &= c + \text{ROTL}_{41}(a). \end{aligned}$$

## Mode 2

$$\begin{aligned}p_0 &= a + \text{ROTL}_{17}(b), \\p_1 &= b + \text{ROTL}_{29}(c), \\p_2 &= c \oplus \text{ROTL}_{41}(a).\end{aligned}$$

The recurrence may be summarized as:

$$(p_0, p_1, p_2) = F_{\mu(g)}(a, b, c).$$

The resulting parent vectors feed both output generation and auxiliary-state update. The details of state storage and generation bookkeeping belong to the control layer; the mathematical core is the three modes above.

---

### 106. 13-Phase Auxiliary State Transition

The 13 auxiliary states are not all updated on every micro-step. Instead, one state is updated at a time according to a 13-phase schedule.

For phase  $\phi \in \{0, 1, \dots, 12\}$ , the updated state is  $m_{3+\phi}$ .

The complete update equations are:

$$\begin{aligned}m'_3 &= (m_3 + \text{ROTL}_7(p_0)) \oplus \text{ROTL}_{13}(m_3), \\m'_4 &= (m_4 + \text{ROTL}_{13}(p_0)) \oplus \text{ROTL}_{17}(m_4), \\m'_5 &= (m_5 + \text{ROTL}_{17}(p_0)) \oplus \text{ROTL}_{21}(m_5), \\m'_6 &= (m_6 + \text{ROTL}_{23}(p_0)) \oplus \text{ROTL}_{29}(m_6), \\m'_7 &= (m_7 + \text{ROTL}_{29}(p_1)) \oplus \text{ROTL}_{33}(m_7), \\m'_8 &= (m_8 + \text{ROTL}_{31}(p_1)) \oplus \text{ROTL}_{37}(m_8), \\m'_9 &= (m_9 + \text{ROTL}_{37}(p_1)) \oplus \text{ROTL}_{41}(m_9), \\m'_{10} &= (m_{10} + \text{ROTL}_{42}(p_1)) \oplus \text{ROTL}_{47}(m_{10}), \\m'_{11} &= (m_{11} + \text{ROTL}_{11}(p_2)) \oplus \text{ROTL}_{17}(m_{11}), \\m'_{12} &= (m_{12} + \text{ROTL}_{19}(p_2)) \oplus \text{ROTL}_{23}(m_{12}), \\m'_{13} &= (m_{13} + \text{ROTL}_{27}(p_2)) \oplus \text{ROTL}_{31}(m_{13}), \\m'_{14} &= (m_{14} + \text{ROTL}_{37}(p_2)) \oplus \text{ROTL}_{41}(m_{14}), \\m'_{15} &= (m_{15} + \text{ROTL}_{43}(p_2)) \oplus \text{ROTL}_{47}(m_{15}).\end{aligned}$$

All additions are modulo  $2^{64}$ .

If the phase update operator is denoted by  $U_\phi$ , one micro-step can be represented conceptually as:

$$S_{g,\phi+1} = U_\phi(S_{g,\phi}, F_{\mu(g)}(a, b, c)).$$

This notation does not mean that the 13 states are updated simultaneously. It represents the intentional **temporal distribution of auxiliary-state updates** across 13 phases.

## 107. 16-Vector Emission Map

Let the 16 vectors emitted at one micro-step be  $e_0, \dots, e_{15}$ . The exact order is:

$$\begin{aligned} e_0 &= p_0, \\ e_1 &= p_1, \\ e_2 &= p_2, \\ e_3 &= p_0 \oplus m_3, \\ e_4 &= p_1 \oplus m_7, \\ e_5 &= p_2 \oplus \text{ROTL}_1(m_{11}), \\ e_6 &= p_0 \oplus m_4, \\ e_7 &= p_1 \oplus m_8, \\ e_8 &= p_2 \oplus m_{12}, \\ e_9 &= p_0 \oplus m_5, \\ e_{10} &= p_1 \oplus m_9, \\ e_{11} &= p_2 \oplus m_{13}, \\ e_{12} &= p_0 \oplus m_6, \\ e_{13} &= p_1 \oplus m_{10}, \\ e_{14} &= p_2 \oplus m_{14}, \\ e_{15} &= p_1 \oplus m_{15}. \end{aligned}$$

Each  $e_i$  is a 64-byte vector and can be represented on AVX-512 as eight 64-bit lanes in one ZMM register.

Therefore one micro-step emits 16 raw vectors.

After the Normal stage and four-way fanout, the logical Normal-domain sample count is:

$$16 \times 16 \times 4 = 1024$$

samples per micro-step.

Here “1024 samples” means the **logical sample count obtained by multiplying emitted-vector count, 32-bit lane count, and fanout factor**. It is not a mathematical guarantee of 1024 independent random variables.

---

## 108. Normal-v9 - Mapping from Raw Integer State to the Normal Domain

V1’s Normal stage is not an analytical inverse-CDF implementation of a continuous Gaussian. It is an approximate mapping based on an integer LUT and integer SIMD dot-product-style operations.

Let the input be:

$$\mathbf{x} = (x_0, \dots, x_{15}) \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}.$$

Let the 128-byte lookup table be  $L$ . Abstracting the AVX-512 byte-permute lookup gives:

$$\mathbf{a} = L(\mathbf{x}).$$

Represent VPMADDUBSW as:

$$\mathbf{p}_{16} = D_{ub}(\mathbf{a}, \mathbf{x})$$

and VPMADDWD with a +1-int16 coefficient vector as:

$$\mathbf{q} = D_{16}(\mathbf{p}_{16}, \mathbf{1}).$$

The implementation treats the resulting lanes as an int32 quantized representation and applies fixed scale  $s$  downstream:

$$z_i = sq_i.$$

The mapping is therefore defined as:

$$Z(\mathbf{x}) = s D_{16}(D_{ub}(L(\mathbf{x}), \mathbf{x}), \mathbf{1}).$$

### 108.1 Statistical Meaning

This equation does not mean that V1's output is mathematically identical to a continuous exact  $N(0, 1)$  distribution.

It is a finite-precision discrete mapping:

```
uint32 bit pattern
 ↓
128-byte LUT / byte permutation
 ↓
integer SIMD dot-product-style reduction
 ↓
quantized q
 ↓
fixed scale
 ↓
Normal-domain z
```

Therefore, Normal quality should be reported through measured mean / variance / skew / kurtosis / tail / pair geometry / radial statistics rather than by claiming exact equality with the ideal continuous distribution.

---

## 109. Four-Way Fanout - Algebraic Representation

Let the 16 int32 vectors be represented by:

$$\mathbf{q} \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}.$$

The two active hot-path shuffle permutations are  $P_0$  and  $P_1$ :

$$\mathbf{y}_0 = P_0\mathbf{q}, \quad \mathbf{y}_1 = P_1\mathbf{q}.$$

The four resulting series are:

$$\mathbf{f}_0 = \mathbf{q} + \mathbf{y}_0, \quad \mathbf{f}_1 = \mathbf{q} - \mathbf{y}_0, \quad \mathbf{f}_2 = \mathbf{q} + \mathbf{y}_1, \quad \mathbf{f}_3 = \mathbf{q} - \mathbf{y}_1.$$

The current shuffle definitions are:

P0 = \_MM\_SHUFFLE(2, 3, 0, 1)  
P1 = \_MM\_SHUFFLE(1, 0, 3, 2)

This makes clear that the four series are linear combinations of the same underlying **q**. Therefore, they must not be treated as four independent standard-normal variable sets by construction.

This distinction is essential to the pair-geometry audit: **fanout amplifies supply/layout rather than mathematically creating independence.**

The implementation audit additionally found that the four PaperView views can reference the same Normal starter value pool through different deterministic ordering and lane permutations. Therefore, 4 views = 4 disjoint Normal pools is not a valid interpretation.

---

## 110. AVX-512 - V1 as 512-bit Vector Algebra

Describing V1 only by instruction names makes it difficult to see which quantities are processed in parallel. The mathematical view is therefore lane-wise.

### 110.1 Front Arithmetic

For a 64-bit state vector:

$$\mathbf{u} = (u_0, \dots, u_7) \in (\mathbb{Z}/2^{64}\mathbb{Z})^8,$$

XOR, ADD, and ROTL are applied independently per lane:

$$(\mathbf{u} \oplus \mathbf{v})_k = u_k \oplus v_k,$$

$$(\mathbf{u} + \mathbf{v})_k = (u_k + v_k) \bmod 2^{64},$$

$$(\text{ROTL}_r(\mathbf{u}))_k = \text{ROTL}_r(u_k).$$

The parent recurrence is therefore a vector function:

$$\mathbf{p} = F_\mu(\mathbf{a}, \mathbf{b}, \mathbf{c}).$$

This is not a description of executing the scalar equation eight times sequentially. It is the same algebraic operation applied simultaneously across eight lanes.

### 110.2 Normal Arithmetic

The Normal stage uses one ZMM containing  $16 \times 32$ -bit lanes.

Therefore:

$$\mathbf{x}, \mathbf{q}, \mathbf{y}_0, \mathbf{y}_1 \in (\mathbb{Z}/2^{32}\mathbb{Z})^{16}$$

and lookup, shuffle, integer multiply-add, and addition/subtraction are applied across 16 lanes.

Thus V1's "16-lane parallelism" does not mean that every stage processes 16 64-bit random values. Front uses 64-bit lanes, while Normal uses 32-bit lanes.

### 110.3 Memory-Side Vectorization Boundary

AVX-512 expresses computation in 512-bit units, but PaperView / final-cache uses a separate logical data structure based on 16-lane rows.

The boundary is therefore:

ZMM / 512-bit instruction domain

↓

Normal 16 × uint32 domain

↓

4-way logical fanout

↓

1024-row PaperView

↓

MC cache / sequential consumer

The number 1024 is a logical generation count derived from vector count, lane count, and fanout. It does not mean that “512-bit automatically produces 1024 independent samples.”

---

## 111. PaperView / Final Cultivation - Mathematical Meaning

PaperView is not a new random transformation. It is deterministic re-indexing of already-generated Normal-domain data.

For the standard view:

$$\text{shift} = \left\lfloor \frac{BR10(\text{epoch} \ \& \ 1023)}{64} \right\rfloor,$$

$$\text{phase} = (\text{epoch} \gg 10) \ \& \ 63,$$

$$\text{vid} = (\text{view\_id} + \text{phase} + \lfloor \text{phase} / 8 \rfloor) \ \& \ 31,$$

$$\text{base} = (A64\_OFFSET[\text{vid}] + \text{shift}) \ \& \ 1023.$$

The generic final cache is:

$$P_{465}(r) = 465r \bmod 1024.$$

Because:

$$\gcd(465, 1024) = 1,$$

this is a bijection of the 1024 rows.

For R2:

$$P_{555}(r) = 555r \bmod 1024,$$

and because:

$$\gcd(555, 1024) = 1,$$

this is also a row permutation.

The two have different purposes:

465 = generic final-cache row permutation

555 = R2 pair-geometry permutation

The difference is directly relevant to pairwise quality and was confirmed empirically by the V1 sign/dep2 root-cause experiment.

## 112. Theoretical R2 Distribution - From the Normal Domain to the MC Quantity

Let a pair be  $(x, y)$  and define:

$$R^2 = x^2 + y^2.$$

If  $x$  and  $y$  are ideally independent standard Normal variables:

$$x, y \stackrel{ind}{\sim} N(0, 1),$$

then:

$$R^2 \sim \chi_2^2.$$

Thus:

$$E[R^2] = 2,$$

$$\text{Var}(R^2) = 4,$$

and:

$$P(R^2 \leq 1) = 1 - e^{-1/2} \approx 0.3934693402873666.$$

The V1 metrics

$E[R^2]$

$\text{Var}(R^2)$

$P(R^2 \leq 1)$

sign

corr(x, y)

dep2

are used to evaluate implementation and pair-geometry behavior against these ideal references.

A key point is that good marginal Normal quality does not guarantee good pair geometry. V1 experiments showed substantial sign/dep2 bias from the combination of same-q-derived fanout pairs and cache-row pairing, with large improvement after the stride correction.

MC quality should therefore be considered at three levels:

marginal quality + pair geometry + consumer mapping.

---

### 113. State-Space Size and Period - What the 8192-bit State Implies

Considering only the resident state, V1 has at most:

$$2^{8192}$$

possible state configurations.

If one instead fixes all generation/phase control parameters and treats the Front recurrence as a time-invariant deterministic map:

$$S_{t+1} = F(S_t),$$

then any trajectory has period bounded by:

$$\boxed{\text{period} \leq 2^{8192}}.$$

This is **not** a claim that V1's measured period is  $2^{8192}$ .

Moreover, current V1 changes parent permutation and ADD/XOR mode according to generation. To represent the complete stream as one time-invariant map, the generation schedule must be incorporated into an extended state.

The safe statements are therefore:

1. resident numeric state alone has at most  $2^{8192}$  configurations
2. a fixed-transition interpretation gives period  $\leq 2^{8192}$
3. the full V1 stream period has not been formally established

A finite state space plus a 512 MiB test cannot prove the absence of periodicity beyond the tested window. The 512 MiB test is finite-window empirical qualification, not a period theorem.

---

### 114. Separate Theoretical Limits from Empirical Results

The state-space definition does not by itself prove:

uniformity is guaranteed  
independence is guaranteed  
period is maximal

The current V1 evidence has a two-layer structure:

Mathematical layer

- > exact state transition
- > exact finite-state representation
- > exact permutation / fanout / R2 equations
- > finite state-space bound

Empirical layer

- > Normal marginal statistics
- > pair / radial statistics

- > raw Front-state surrogate battery
- > multi-seed / inter-stream measurements
- > convergence measurements

The mathematical specification tells us **what was computed**. Statistical testing tells us **what empirical properties that computation exhibited**.

These two layers are complementary rather than substitutes for each other.

---

Thus the book has a two-layer structure.

In other words, the mathematical specification establishes **what was computed**, while the statistical testing tells us **what empirical properties that computation exhibited**.

These are complementary, not interchangeable, forms of evidence.

\* \* \*

#### ## 115. Back-Side Seed Construction - Initial Conditions for the Front State

Looking only at the Front state transition is not sufficient for a complete mathematical description of the system.

The entropy-conditioning stage produces a 256-bit conditioned value:

```

$$
C=SHA256(\text{"UFRNG-ENTROPY-CONDITIONER-V2"}\parallel R\parallel \text{NodeID}\parallel \text{RunID}\parallel \text{SeedBank})
$$

```

The LPS/PSL2(F29) stage uses  $p=5$ ,  $q=29$ , four lanes, and six generators, producing eight star values.

For lane  $\ell$ , with matrix state  $X_i$ :

```

$$
X_{i+1}=\text{operatorname{canon}}(X_iG[s_i]\bmod 29)
$$

```

The four endpoints are combined into a domain-separated extraction payload and hashed as:

```

$$
\text{RootSeed}=\text{SHA512}(\text{payload}).
$$

```

The Seed Bank is derived by:

```

$$
\text{Bank}_b=\text{SHA512}(\text{"UFRNG-SEED-BANK-V1"}\parallel \text{Root}\parallel \text{BE64}(b)).
$$

```

CHACHA20-AAA-A1 then creates the Back-side profile.

The architectural meaning of this boundary is that the **short Front recurrence** starts from strong entropy conditioning.

SHA/LPS/AAA processing is not brought into the Front hot loop. This is an important performance-design constraint.

\* \* \*

## ## 116. VHV Front16 Initialization

The VHV expansion initializes the 16 states directly from the profiled seed:

\$\$

$V1_i = \text{SHA512}(\text{"UFRNG-FRONT16-SHA512-V1"}) \parallel \text{ProfiledSeed} \parallel \text{BE64}(\text{bank}) \parallel$

\$\$

The 16 values are then combined into:

\$\$

$H = \text{SHA512}(\text{"UFRNG-FRONT16-VHV-HASH-V1"}) \parallel V1_0 \parallel \dots \parallel V1_{15} \parallel$

\$\$

The final state is:

\$\$

$V2_i = \text{SHA512}(\text{"UFRNG-FRONT16-VHV-COMPACT-V1"}) \parallel V1_i \parallel H \parallel \text{BE32}(i).$

\$\$

Thus the 16 64-byte resident states handed to the Front are not merely copies of a single parent v state material.

This structure should be understood as the Back-side diffusion layer that supplies high-quality initial conditions to the downstream 16-state recurrence.

\* \* \*

## ## 117. Minimum Mathematical Skeleton to Carry from the Specification into the Paper

The paper does not need the full content of this chapter. The minimum mathematical skeleton requi

```text

A. state representation

$S = (m_0, \dots, m_{15}), 16 \times 64\text{-byte}$

B. parent recurrence

$(p_0, p_1, p_2) = F_{\text{mode}}(a, b, c)$

C. 13-phase auxiliary update

$m_j' = U_j(m_j, p_k)$

D. emission map

$e_0, \dots, e_{15}$

E. Normal mapping

$z = \text{scale} \times D16(\text{Dub}(\text{LUT}(x), x), 1)$

F. fanout

$q \pm P_0(q), q \pm P_1(q)$

G. MC mapping

$R^2 = x^2 + y^2$

H. cultivation
 $P465(r)=465r \bmod 1024$ / $P555(r)=555r \bmod 1024$

I. state-space statement
 resident configuration count $\leq 2^{8192}$
 but no formal full-stream period claim

This is what should be compressed into the paper, while the present book retains the constants, lane semantics, boundary conditions, and relationship to measured results.

118. Mathematical-Core Audit Result

With this extension, the Deconstruction Book explicitly covers the following four areas:

| Core item | Location in this book | Status |
|----------------------------------|-----------------------|--|
| Front/Back state transition | §§104–107, 115–116 | Fully formalized |
| Normal / MC distribution mapping | §§108–109, 112 | Implementation mapping + theoretical reference |
| AVX-512 vector algebra | §110 | Lane / vector formalization |
| State-space / period bound | §§113–114 | Bound and unproven points explicitly stated |

These sections are not a shortened paper presentation. They are retained as a reference base so that a reviewer who wants to go beyond the concise paper description can trace the implementation completely.

Part XXXVI - Bit-for-Bit Reproducibility and Exact Implementation Semantics

119. Purpose of This Chapter

The mathematical sections above define V1’s design and numerical data path. This chapter goes one step further and fixes the byte/bit semantics required for a third party to reproduce the same seed, Front initial state, Normal output, and PaperView placement from the same input, metadata, and generation.

Here, “reproducible” does not mean merely obtaining statistically similar random values. It means obtaining the **same byte sequence from the same canonical input**.

120. Binary Encoding Rules

Integer fields used by V1 seed construction are serialized in big-endian order where explicitly specified.

$BE32(x) = x \gg 24, x \gg 16, x \gg 8, x$
 $BE64(x) = x \gg 56, \dots, x \gg 8, x$

ChaCha20 internal 32-bit words use the standard ChaCha little-endian load/store semantics.

Thus the boundary is:

Seed / domain payload integer metadata
-> explicit BE encoding

ChaCha internal state words
-> explicit LE 32-bit encoding

Domain strings do not include a NUL terminator in hash input; the source feeds `sizeof(domain)-1` bytes.

121. Light Cleanup - Exact Behavior

Raw input must be processed in 64-byte blocks.

Each block is handled in order:

1. If all 64 bytes are zero, discard the block.
2. If it is byte-for-byte identical to the previously **accepted cleaned block**, discard it.
3. Otherwise, accept it unchanged.

The comparison is therefore against the previously accepted cleaned block, not against the immediately preceding raw block.

Cleanup is deterministic preprocessing before conditioning; it is not an entropy-rate proof.

122. SHA-256 Entropy Conditioning - Exact Payload

Let the cleaned byte string be *R*. The exact payload order is:

"UFRNG-ENTROPY-CONDITIONER-V2"

R

NodeID 16 bytes

RunID 16 bytes

BE64(Sequence) 8 bytes

Therefore:

$$C = SHA256(\text{domain} \parallel R \parallel \text{NodeID} \parallel \text{RunID} \parallel BE64(\text{Sequence})).$$

The output is 32 bytes.

NodeID, RunID, and Sequence are domain-separation / reproducibility identifiers; they are not counted as physical entropy by themselves.

123. LPS Selector - Exact Bit Consumption

For each 64-byte SHA-512 digest block, the LPS selector consumes bytes from low-order bits first.

From each byte `buf[i]`, it extracts 3-bit candidates in this order:

`shift = 0, 3, 6`

`x = (buf[i] >> shift) & 7`

`x = 0..5 -> accept`

`x = 6 or 7 -> reject`

Only accepted values are appended to the symbols array, in order, until enough symbols have been collected.

If one digest block does not provide enough accepted symbols, the counter is incremented and the next SHA-512 input is generated.

The exact selector input is:

```
"UFRNG-LPS-X4-SELECTOR-V2"
seed
BE64(counter)
```

The counter starts at zero.

The selector is therefore not modulo 6. It is a **rejection sampler that consumes 3-bit values directly and rejects values 6 and 7.**

124. LPS Lane Seed Derivation - Exact Domain Separation

For lane e11, a 4-byte big-endian lane identifier is used.

```
start_seed = SHA512(
    "UFRNG-LPS-X4-START-V2" || conditioned32 || BE32(e11)
)
```

```
walk_seed = SHA512(
    "UFRNG-LPS-X4-WALK-V2" || conditioned32 || BE32(e11)
)
```

Each lane produces 8 symbols from the start selector and 16 symbols from the walk selector.

The matrix state begins at:

```
X0 = [[1,0],[0,1]]
```

and for each symbol:

$$X_{i+1} = \text{canon}(X_i G_{s_i} \bmod 29).$$

125. canon(X) - Exact Representative Rule

For matrix A, compute:

$$-A \pmod{29}.$$

The comparison order is:

```
(a00, a01, a10, a11)
```

in lexicographic order.

The lexicographically smaller representative is selected at the first differing field; if all four fields are equal, the A side is retained.

This rule implements the sign identification needed for the PSL2(F29) representative. The word "canon" alone is therefore insufficient; the ordering of all four matrix entries is fixed.

126. RootSeed Extraction - Exact Byte Layout

The four lane endpoints $E_0 \dots E_3$ are appended to the following payload in order:

```
"UFRNG-LPS-X4-EXTRACT-V2"  
BE32(5)           // p  
BE32(29)          // q  
BE32(16)          // walk steps  
BE32(4)           // lane count
```

```
for lane = 0..3:  
    BE32(lane)  
    BE16(E.a00)  
    BE16(E.a01)  
    BE16(E.a10)  
    BE16(E.a11)
```

Matrix entries are encoded as 16-bit big-endian values.

The RootSeed is the SHA-512 digest of the entire payload:

$$RootSeed = SHA512(payload).$$

This produces the 64-byte root.

127. Seed Bank and Profile - Exact Sequence

The bank seed is:

$$Bank_b = SHA512("UFRNG-SEED-BANK-V1" \parallel Root \parallel BE64(b)).$$

For the CHACHA20-AAA-A1 profile:

```
K = SHA256("UFRNG-PROFILE-CHACHA20-KEY-V1" \parallel Bank_b)  
M = SHA256("UFRNG-PROFILE-CHACHA20-NONCE-V1" \parallel Bank_b \parallel BE64(Sequence))  
Nonce = M[0:12]
```

ChaCha block counters for the profile output are generated in this order:

```
block 0  
block 1
```

All 64 bytes of each block are generated, but the public profile selects:

```
block0[0:32]  
block1[0:32]
```

These are concatenated into 64 bytes.

Immediately after the column half of double-round index 9, the AAA mutation is applied once to word 8:

$$x_8 \leftarrow x_8 + 0x3B54CDA5 \pmod{2^{32}}.$$

The diagonal half then executes normally.

AAA profile identifier: CHACHA20-AAA-A1.

128. VHV - Exact Front16 Seed Materialization

For each $i=0 \dots 15$:

$$V1_i = SHA512("UFRNG-FRONT16-SHA512-V1" \parallel ProfiledSeed \parallel BE64(bank) \parallel BE64(generation) \parallel BE64(i)).$$

The 16 values are concatenated in their current C row-major / lane-major order, $V1[0]$ through $V1[15]$.

Then:

$$H = SHA512("UFRNG-FRONT16-VHV-HASH-V1" \parallel V1_0 \parallel \dots \parallel V1_{15} \parallel BE64(bank) \parallel BE64(generation)).$$

Each final lane becomes:

$$V2_i = SHA512("UFRNG-FRONT16-VHV-COMPACT-V1" \parallel V1_i \parallel H \parallel BE32(i)).$$

This is the canonical initial material for the 16×64 -byte Front resident states.

129. Front Recurrence - Closed State Transition

Let the parent states at micro-step t be:

$$(a_t, b_t, c_t) = (m_0^{(t)}, m_1^{(t)}, m_2^{(t)}).$$

Let the mode be:

$$\mu_t = \left\lfloor \frac{g}{2} \right\rfloor \bmod 3.$$

First compute:

$$(p_0, p_1, p_2) = F_{\mu_t}(a_t, b_t, c_t).$$

Then perform the 13-phase auxiliary update. The next parent state is explicitly:

$$(a_{t+1}, b_{t+1}, c_{t+1}) = (p_0, p_1, p_2)$$

The auxiliary phase advances as:

$$\phi_{t+1} = (\phi_t + 1) \bmod 13.$$

Thus, for fixed generation g and phase parameter, the complete Front can be represented as:

$$S_{t+1} = \mathcal{F}_{g, \phi_t}(S_t)$$

as a **parameterized deterministic state transition**.

Because g may be updated externally at a generation handoff boundary, the system should not be described as one autonomous time-invariant closed system unless that generation schedule is incorporated into an extended state.

This does not mean that all 13 auxiliary states are updated at every step. Parent states update every step; one auxiliary state is updated at a time, distributed across the 13 phases.

130. Exact Resident-State Map

Using the selected A/B/C lineage according to $\text{pi}(g)$, load the Front resident states into the following fixed positions:

```

m0 <- selected A[0]
m1 <- selected B[0]
m2 <- selected C[0]
m3 <- selected A[2]
m4 <- selected B[2]
m5 <- selected C[2]
m6 <- selected A[3]
m7 <- selected B[3]
m8 <- selected C[3]
m9 <- selected A[4]
m10 <- selected B[4]
m11 <- selected C[4]
m12 <- selected A[5]
m13 <- selected B[5]
m14 <- selected C[5]
m15 <- A[1] XOR B[1] XOR C[1]

```

m_{15} is fixed as the XOR of the A/B/C lineage values from the source implementation; it is not an abstract placeholder replaced solely by the selected parent labels.

131. Normal-v9 - Signedness and Saturation Semantics

The input is $16 \times \text{uint32}$ on a ZMM register.

```

x    = 16 x uint32
L(x) = VPERMI2B / VPERMUTEX2VAR_EPI8 lookup result
p16  = VPMADDUBSW(L(x), x)
q    = VPMADDWD(p16, +1-int16 vector)

```

The signedness of the instruction operands is essential:

- the multiplication at the VPMADDUBSW boundary uses **unsigned-byte $\times$ signed-byte** semantics;
- the sum for each adjacent pair is stored as a **signed saturating int16**;
- VPMADDWD performs signed-int16 $\times$ signed-int16 multiplication and reduces adjacent pairs into int32 sums;
- the second operand contains **the numerical value +1 in every int16 lane**, not the all-ones bit pattern $0xFFFF$.

Therefore, the Normal mapping must be understood as a **finite-precision integer mapping including byte signedness and 16-bit saturation**, not simply as an abstract LUT/vector dot product.

The final Normal value is:

$$z = sq, \quad s = 0.0009317057574691516 \times 1.0007781.$$

132. Exact Logical-Sample Accounting

The meaning of sample in V1 is fixed by layer:

| Layer | 1 unit | Role |
|----------------|---|------------------------------------|
| Front state | 64-bit lane | resident arithmetic unit |
| Front vector | $8 \times 64\text{-bit} = 64\text{ bytes}$ | 1 ZMM state vector |
| Normal input | $16 \times 32\text{-bit} = 512\text{ bits}$ | 1 Normal-transform input |
| Normal output | $16 \times \text{int32 } q$ | 1 q-vector |
| Fanout | 4 q-vectors | 4 logical output vectors |
| One micro-step | $16\text{ emitted vectors} \times 16\text{ lanes} \times 4\text{ fanout}$ | 1024 logical Normal samples |
| PaperView | 1024×16 | 16,384 logical samples |
| Default chunk | 4 PaperViews | 65,536 logical samples |
| R2 pair | 2 Normal samples | 1 (x, y) pair |
| R2 event | 1 pair | one $R^2 = x^2 + y^2$ evaluation |

V1 Gsamples/s benchmark values are, in general, based on this logical-sample accounting.

The four fanout series are derived from the same q, so **logical sample count is not synonymous with a statistically independent sample count**. The deep audit additionally established that the four PaperView views re-index the same starter value pool.

133. PaperView - Exact Materialization Boundary

PaperView is not a random transform.

The input is an already-generated Normal-domain starter, and the output is deterministic rematerialization.

For a standard view:

$$x_0 = \text{epoch} \& 1023, \quad x_1 = BR10(x_0), \quad \text{shift} = x_1 \gg 6,$$

$$\text{phase} = (\text{epoch} \gg 10) \& 63, \quad \text{vid} = (\text{view_id} + \text{phase} + (\text{phase} \gg 3)) \& 31,$$

$$\text{base} = (A64_OFFSET[\text{vid}] + \text{shift}) \& 1023.$$

For block b and row j:

$$\text{src} = (16b + j + \text{base}) \& 1023.$$

When the src row tail flag is zero, the $A64_INDEX[A64_LANE[\text{vid}]]$ mapping is used; when it is one, the corresponding tail permutation is used.

134. Final-Cache and R2-Cache - Exact Consumer Contract

Generic final cache:

$$P_{465}(r) = 465r \bmod 1024.$$

R2 pair cache:

$$r_0 = P_{555}(2p + \pi), \quad r_1 = P_{555}(2p + \pi + 1), \quad \pi \in \{0, 1\}.$$

Thus 555 is not applied to the logical pair index itself; it is applied to the logical row numbers that form the pair.

The R2 consumer computes $\text{mapped_base} = P_{\{555\}}(\text{base})$, then uses $\text{parity} = \text{mapped_base} \& 1$ and $\text{p0} = \text{mapped_base} \gg 1$ to read pairs sequentially from the corresponding parity bank.

The mapping is paid for during cache construction, leaving the MC consumer with sequential row/pair access.

The representative R2 hot loop maps a materialized pair to:

$$x = q_0 s / \sqrt{2}, \quad y = q_1 s / \sqrt{2}, \quad R^2 = x^2 + y^2.$$

and evaluates

$$h = \mathbf{1}[R^2 \leq 1].$$

A representative payoff is

$$f = h(1 - R^2).$$

Part XXXVII - Back / Front Seed-Exchange State Machine

135. Slot States

The two persistent Back handoff slots in V1 have four states:

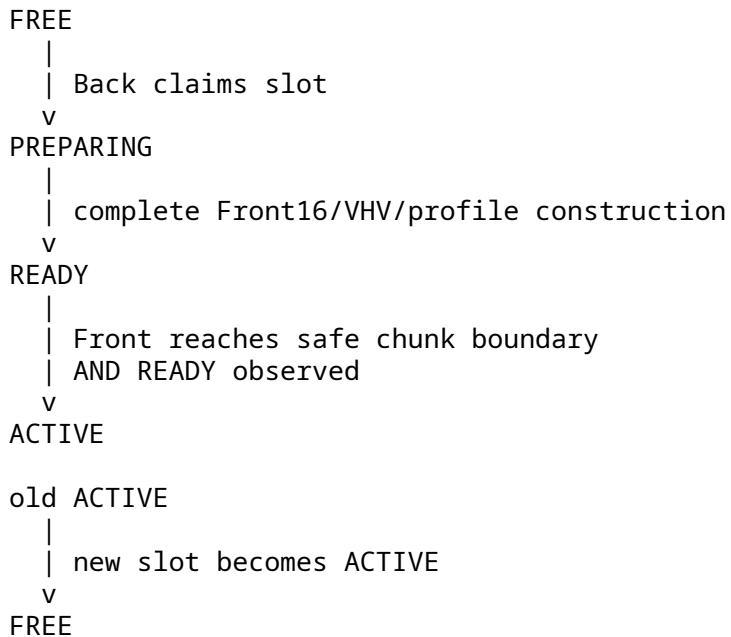
FREE
PREPARING
READY
ACTIVE

The active slot holds the completed Front16 state set currently consumed by Front.

The alternate slot is the location where the Back worker prepares the next generation.

136. Preparation and Handoff State Machine

The logical transition is:



Front does not spin or block waiting for Back to finish.

If the alternate slot is not READY, Front continues with the current generation.

137. Exact Switch Condition

A generation switch is attempted at the safe point after a complete MC chunk has been consumed.

Let the current slot be `cur` and the alternate slot be `nxt`. Front first observes:

`nxt.state == READY`

using acquire semantics.

It then commits:

`READY -> ACTIVE`

with CAS.

After success:

```
load(nxt Front16 state)
cur.state = FREE
active ^= 1
switches++
generation_id = nxt.generation
chunks_in_generation = 0
```

This ordering prevents Front from reading an incomplete seed and returns the previous active slot to FREE so that Back can reuse it.

138. Requested Generation Interval vs. Actual Exchange Interval

Under the current default, the frequency at which the generation policy makes a switch eligible and the frequency at which a switch actually occurs are not the same.

```
policy eligibility
    !=
actual READY handoff
```

In measurement, even a run of 1,073,741,824 logical samples produced only about 3–4 successful switches.

This is not abnormal. Because Back runs on the same core as Front and is kept as a low-priority / scalar-only quality-preparation process, the Back seed-preparation throughput determines the actual handoff frequency.

V1 does not aim to create a new seed every chunk. Its design goal is to **exchange a READY seed at a safe point without stalling Front**.

Part XXXVIII - Benchmark Reproduction Protocol

139. Fixed Items for Reproducing Benchmark Results

When comparing V1 throughput values, the following should be recorded together, in light of the anti-reuse audit:

```
source SHA
benchmark harness SHA
CPU model
visible vCPU count
CPU affinity / pinning
compiler
compiler flags
ISA features
transport mode
cultivation mode
final profile
chunk size
views/chunk
generation policy
generation chunks
warm-up count
timed chunk count
logical sample count
timer
run count
outlier handling rule
```

If these conditions are not aligned, an absolute Gsamples/s difference should not be interpreted directly as a V1 code difference.

140. Current V1 Peak Reproduction Record

The currently recorded peak was measured under:

CPU: AMD EPYC 9V74 80-Core Processor
runtime: 3 vCPUs visible
pinning: exactly one logical CPU
compiler: gcc -O3 -std=c11 -pthread -march=native
profile: R2_FLOAT32
cultivation: DIRECT
fixed generation: 1,048,576 chunks
timed chunks: 131,072
logical samples: 8,589,934,592
with:

46.602 Gsamples/s

This peak is an aggregate throughput result from an 8.59-billion-logical-sample timed run, not a single-second microbenchmark.

141. Boundary for Comparison with Published Raw-RNG References

Published ns/64-bit PRNG values measure raw random-output generation. V1's representative throughput is logical MC sample throughput.

Therefore, this book may place the values on a common G/s visual scale, but the column header must state the **measurement boundary**.

Published raw RNG
-> 64-bit random output / second

UltraFastRng512 V1
-> logical MC samples / second

This comparison is useful for performance-scale orientation, but it does not automatically imply a same-workload speed ratio.

Part XXXIX - Domain Adapter Mathematical Reference

142. Finance Common Log-Domain Path

A representative Finance q256 path processes 4 independent path blocks × 64 time steps.

The increment is formed from q as:

$$\Delta \log S_t = q_t s \sigma \sqrt{dt} + (r - \frac{1}{2}\sigma^2)dt$$

A prefix sum is formed within each 16-step block, and the final prefix value is carried into the next block.

Thus path log-price is:

$$\log S_t = \log S_0 + \sum_{j=1}^t \Delta \log S_j.$$

For a Geometric Asian payoff, the mean log-price over 64 time points is:

$$\bar{L} = \frac{1}{64} \sum_{t=1}^{64} \log S_t$$

and:

$$S_{geo} = S_0 \exp(\bar{L})$$

followed by the call payoff:

$$\max(S_{geo} - K, 0).$$

For a lookback call, the path maximum is tracked in log-domain; for a lookback put, the path minimum is tracked. The path returns to spot-domain with a single exponential evaluation at the end.

143. Pharma Two-Compartment Path

For the standard linear two-compartment IV-bolus model:

$$k_{10} = CL/V_C, \quad k_{12} = Q/V_C, \quad k_{21} = Q/V_P.$$

Define:

$$\Sigma = k_{10} + k_{12} + k_{21}.$$

The macro rates are:

$$\alpha = \frac{\Sigma + \sqrt{\Sigma^2 - 4k_{10}k_{21}}}{2},$$

$$\beta = \frac{\Sigma - \sqrt{\Sigma^2 - 4k_{10}k_{21}}}{2}.$$

Under the IV-bolus condition, concentration is:

$$C(t) = Ae^{-\alpha t} + Be^{-\beta t}.$$

The coefficients are:

$$A = \frac{D/V_C (\alpha - k_{21})}{\alpha - \beta},$$

$$B = \frac{D/V_C (k_{21} - \beta)}{\alpha - \beta}.$$

For fixed dt:

$$r_\alpha = e^{-\alpha dt}, \quad r_\beta = e^{-\beta dt}$$

and the time loop becomes:

$$p_\alpha \leftarrow p_\alpha r_\alpha, \quad p_\beta \leftarrow p_\beta r_\beta,$$

$$C_t = Ap_\alpha + Bp_\beta.$$

These equations are reference equations for understanding V1's manual/domain-specific two-compartment acceleration; they are not part of the generic RNG core.

144. Piecewise-Constant Infusion / Segment Transition

Represent the linear time-invariant compartment system as:

$$x_{n+1} = Mx_n + Gu_n.$$

Within a segment where $u_n = u$ is constant, this becomes:

$$x_{n+1} = Mx_n + Gu.$$

V1's piecewise path prepares, once per segment:

```
M
G
G*u_segment
```

and reuses the same transition quantities for the time steps within that segment.

Only segment boundaries apply events such as boluses.

This should not be interpreted as a "faster general-purpose ODE solver." It is an optimization that **pre-computes repeated work for a domain with fixed linear transitions**.

Part XL - Benchmark and Reproducibility Checklist

145. Third-Party Reproduction Checklist

A third party reproducing V1 results should preserve at least:

```
[Identity]
source SHA
math-spec version
book version
```

```
[Seed]
raw input bytes
NodeID
RunID
Sequence
```

bank index
profile ID

[Front]
generation
parent permutation index
mode
phase
16 initial states or their canonical source seed

[PaperView]
epoch
view_id
shift
phase
vid
base
profile / transport

[MC]
precision
pairing rule
chunk count
sample count
MC domain

[Timing]
CPU
pinning
compiler
flags
warmup
timed region
timer
all raw runs

Keeping this checklist makes it possible to binary-search the point at which a result changed.

146. Reproduction Granularity

V1 reproducibility should not rely on a single golden vector. It should be verified at the following levels:

Level 0: seed hash
Level 1: RootSeed
Level 2: Bank seed
Level 3: Profile seed
Level 4: Front16 VHV set
Level 5: one Front micro-step output
Level 6: Normal q-vector
Level 7: PaperView row
Level 8: final cache row
Level 9: MC result / checksum

These staged golden references allow the divergence point to be localized even when only the final MC result differs.

Part XLI - Exactness Audit Revision Record

147. V2.5 / V2.6 Correction Set

This revision changes documentation only; the frozen V1 source algorithm is not changed. The purpose is to make the documentation reproduce the already-frozen implementation rather than redesign any algorithm. The authoritative implementation is the V1 source with SHA-256:

40a6b4311b4cb0e4348f08990b5380ff7607444a92534e17e7014b997af393ba

The following documentation-level discrepancies from v2.4 have been corrected:

| Item | v2.4 issue | v2.5 resolution | v2.6 resolution |
|---------------------------|--|---|---|
| Normal LUT | Appendix D contained an older LUT | Replaced by frozen V1 source LUT | — |
| VPMADDWD coefficient | “+1-int16 vector” was ambiguous/misleading | Defined as +1 int16 in every lane | — |
| R2 stride | 555 × logical_pair_row oversimplified the mapping | Defined exact $P555(2p+\pi) / P555(2p+\pi+1)$ mapping | — |
| R2 scaling | $R^2=x^2+y^2$ omitted the implementation boundary | Defined $x=q0 \ s/\sqrt{2}$, $y=q1 \ s/\sqrt{2}$, $scale2=(s/\sqrt{2})^2$ | — |
| Tail rule | Only “tail flag” was named | Exact threshold and strict comparisons added | — |
| PaperView shift | shift=BR10(epoch) did not match frozen source | Defined shift=BR10(epoch & 1023) >> 6 | — |
| PaperView row addressing | row mapping was implicit | Exact src=(16b+j+base)&1023 added | — |
| Physical starter size | 64 KiB logical view could be confused with retained storage | 32 KiB int16 starter + 2 KiB tail mask distinguished | — |
| LPS wording | “independent lanes” could imply theorem-level independence | Changed to domain-separated lanes | — |
| Front transition | “closed” was stronger than parameterized form | Defined as parameterized deterministic transition | — |
| Optimization / anti-reuse | logical sample count could be read as fresh/independent sample count | — | Added deep PaperView/FIXED reuse audit, CHUNK-vs-FIXED companion throughput, and explicit logical-vs-fresh sample terminology |

No benchmark result is changed by this documentation revision; the v2.6 revision adds qualification to

the interpretation of existing throughput numbers.

148. Authority Rule

When prose, mathematical shorthand, and implementation differ, this book uses the **frozen V1 source** as the bit-for-bit authority. Mathematical shorthand is retained only where it is exactly equivalent to the source semantics.

For statistical interpretation, source fidelity and statistical independence remain separate claims: exact reproducibility does not imply IID sampling, and domain separation does not constitute a proof of independence.

Appendix D - Complete Static Implementation Tables

This appendix exists so that the Deconstruction Book itself is sufficient to reproduce V1's static mapping without opening a separate mathematical specification.

The following tables are copied from the **frozen V1 source** (SHA-256: 40a6b4311b4cb0e4348f08990b5380ff760744) and are the authoritative static implementation data for bit-for-bit reconstruction.

16.1 Normal 128-byte LUT

```
static const uint8_t normal_mag128[128] __attribute__((aligned(64))) = {
0,4,6,4,5,5,2,3,2,5,1,6,11,4,3,3,1,6,6,6,6,14,0,13,15,7,2,7,5,11,9,3,
10,8,4,11,2,8,2,4,5,9,9,6,10,8,10,8,11,7,11,11,10,11,3,9,2,12,11,4,7,8,5,9,
9,9,5,8,7,4,11,12,2,9,3,11,10,11,11,7,11,8,10,8,10,6,9,9,5,4,2,8,2,11,4,8,
10,3,9,11,5,7,2,7,15,13,0,14,6,6,6,6,1,3,3,4,11,6,1,5,2,3,2,5,5,4,6,4
}
```

16.2 PaperView view descriptors

```
static const uint16_t UFR_PAPER_A64_OFFSET[32] = {0, 333, 254, 611, 214, 871, 300, 561, 739, 892, 377,  
static const uint8_t UFR_PAPER_A64_LANE[32] = {0, 58, 49, 52, 53, 9, 47, 54, 56, 27, 61, 43, 22, 15, 61, 56  
static const uint8_t UFR_PAPER_A64_TAIL[32] = {0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2
```

16.2a Exact Tail Flag Rule

For a retained `int16` starter row whose widened values are `q_i`, `V1` computes:

$$T_q = \text{trunc}_{int32} \left(\frac{3.8}{s/\sqrt{2}} \right),$$

with the canonical V1 scale $s = 0.0009317057574691516 \times 1.0007781$, and sets `tail_mask[row]` if and only if at least one of its 16 lanes satisfies:

$$q_i > T_q \quad \text{or} \quad q_i < -T_q.$$

The comparison is strict ($>$ / $<$), not inclusive.

16.3 A64 64×16 Lane-Index Family

```
static const uint32_t UFR_PAPER_A64_INDEX[64][16] = {
{9,14,3,8,13,2,7,12,1,6,11,0,5,10,15,4},
{14,1,4,7,10,13,0,3,6,9,12,15,2,5,8,11},
{11,8,5,2,15,12,9,6,3,0,13,10,7,4,1,14},
{4,7,10,13,0,3,6,9,12,15,2,5,8,11,14,1},
{9,6,3,0,13,10,7,4,1,14,11,8,5,2,15,12},
{6,3,0,13,10,7,4,1,14,11,8,5,2,15,12,9},
{12,15,2,5,8,11,14,1,4,7,10,13,0,3,6,9},
{1,4,7,10,13,0,3,6,9,12,15,2,5,8,11,14},
{1,14,11,8,5,2,15,12,9,6,3,0,13,10,7,4},
{5,2,15,12,9,6,3,0,13,10,7,4,1,14,11,8},
{10,7,4,1,14,11,8,5,2,15,12,9,6,3,0,13},
{3,0,13,10,7,4,1,14,11,8,5,2,15,12,9,6},
{5,8,11,14,1,4,7,10,13,0,3,6,9,12,15,2},
{10,13,0,3,6,9,12,15,2,5,8,11,14,1,4,7},
{2,5,8,11,14,1,4,7,10,13,0,3,6,9,12,15},
{6,9,12,15,2,5,8,11,14,1,4,7,10,13,0,3},
{9,12,15,2,5,8,11,14,1,4,7,10,13,0,3,6},
{2,15,12,9,6,3,0,13,10,7,4,1,14,11,8,5},
{13,0,3,6,9,12,15,2,5,8,11,14,1,4,7,10},
{7,4,1,14,11,8,5,2,15,12,9,6,3,0,13,10},
{0,3,6,9,12,15,2,5,8,11,14,1,4,7,10,13},
{8,11,14,1,4,7,10,13,0,3,6,9,12,15,2,5},
{4,15,10,5,0,11,6,1,12,7,2,13,8,3,14,9},
{13,10,7,4,1,14,11,8,5,2,15,12,9,6,3,0},
{7,12,1,6,11,0,5,10,15,4,9,14,3,8,13,2},
{1,6,11,0,5,10,15,4,9,14,3,8,13,2,7,12},
{14,11,8,5,2,15,12,9,6,3,0,13,10,7,4,1},
{15,12,9,6,3,0,13,10,7,4,1,14,11,8,5,2},
{12,7,2,13,8,3,14,9,4,15,10,5,0,11,6,1},
{2,7,12,1,6,11,0,5,10,15,4,9,14,3,8,13},
{0,11,6,1,12,7,2,13,8,3,14,9,4,15,10,5},
{15,10,5,0,11,6,1,12,7,2,13,8,3,14,9,4},
{10,15,4,9,14,3,8,13,2,7,12,1,6,11,0,5},
{5,10,15,4,9,14,3,8,13,2,7,12,1,6,11,0},
{4,9,14,3,8,13,2,7,12,1,6,11,0,5,10,15},
{1,12,7,2,13,8,3,14,9,4,15,10,5,0,11,6},
{8,3,14,9,4,15,10,5,0,11,6,1,12,7,2,13},
{11,0,5,10,15,4,9,14,3,8,13,2,7,12,1,6},
{12,1,6,11,0,5,10,15,4,9,14,3,8,13,2,7},
{14,9,4,15,10,5,0,11,6,1,12,7,2,13,8,3},
{0,5,10,15,4,9,14,3,8,13,2,7,12,1,6,11},
{2,13,8,3,14,9,4,15,10,5,0,11,6,1,12,7},
{7,2,13,8,3,14,9,4,15,10,5,0,11,6,1,12},
{3,8,13,2,7,12,1,6,11,0,5,10,15,4,9,14},
{11,6,1,12,7,2,13,8,3,14,9,4,15,10,5,0},
{6,1,12,7,2,13,8,3,14,9,4,15,10,5,0,11},
{13,8,3,14,9,4,15,10,5,0,11,6,1,12,7,2},
{6,11,0,5,10,15,4,9,14,3,8,13,2,7,12,1},
{9,4,15,10,5,0,11,6,1,12,7,2,13,8,3,14},
{13,2,7,12,1,6,11,0,5,10,15,4,9,14,3,8},
{10,5,0,11,6,1,12,7,2,13,8,3,14,9,4,15},
{5,0,11,6,1,12,7,2,13,8,3,14,9,4,15,10},
{15,4,9,14,3,8,13,2,7,12,1,6,11,0,5,10},
```

```
{8,13,2,7,12,1,6,11,0,5,10,15,4,9,14,3},
{14,3,8,13,2,7,12,1,6,11,0,5,10,15,4,9},
{3,14,9,4,15,10,5,0,11,6,1,12,7,2,13,8},
{15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,0},
{3,2,1,0,15,14,13,12,11,10,9,8,7,6,5,4},
{11,10,9,8,7,6,5,4,3,2,1,0,15,14,13,12},
{4,5,6,7,8,9,10,11,12,13,14,15,0,1,2,3},
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15},
{8,9,10,11,12,13,14,15,0,1,2,3,4,5,6,7},
{12,13,14,15,0,1,2,3,4,5,6,7,8,9,10,11},
{7,6,5,4,3,2,1,0,15,14,13,12,11,10,9,8},
}
```

16.4 Tail Permutation Family

```
static const uint32_t UFR_PAPER_TAIL_PERM[4][16] = {
{8,0,9,1,10,2,11,3,12,4,13,5,14,6,15,7},
{0,8,1,9,2,10,3,11,4,12,5,13,6,14,7,15},
{15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,0},
{4,5,6,7,0,1,2,3,12,13,14,15,8,9,10,11}
}
```

Appendix E - Final Implementation-Fidelity Notes

E.1 V1 Canonical Normal Scale

0.0009317057574691516f * 1.0007781f

E.2 V1 Canonical Source Identity

Frozen submission SHA-256:

40a6b4311b4cb0e4348f08990b5380ff7607444a92534e17e7014b997af393ba

Measured benchmark snapshot SHA-256:

cd8dae646a9865f9867095df5da4903a6e03ce3f806a9dbbce135c8ec2a4ee8b

E.3 Scope of This Complete Specification

The book now defines the V1 numeric path at four levels:

1. Mathematical meaning
2. Exact implementation semantics
3. Reproducibility / control semantics
4. Benchmark measurement boundary

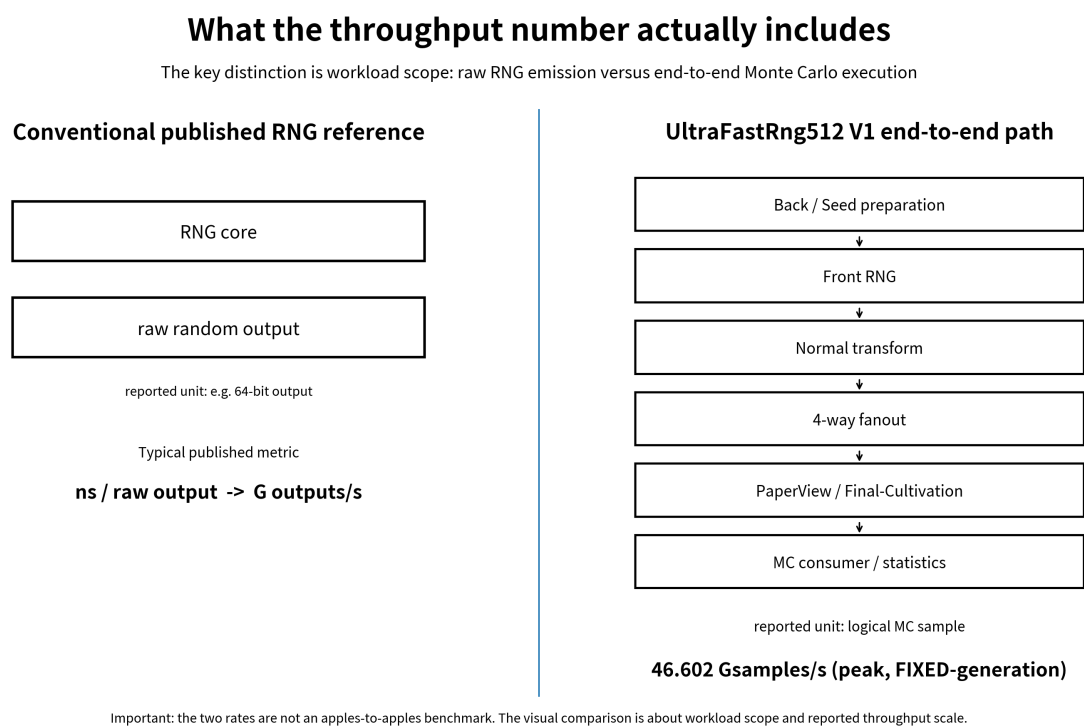
The external validation boundary remains unchanged: formal named statistical suites and fair same-host competitor measurements are empirical qualification tasks, not mathematical consequences of the implementation.

Appendix E - Embedded Comparison Figures

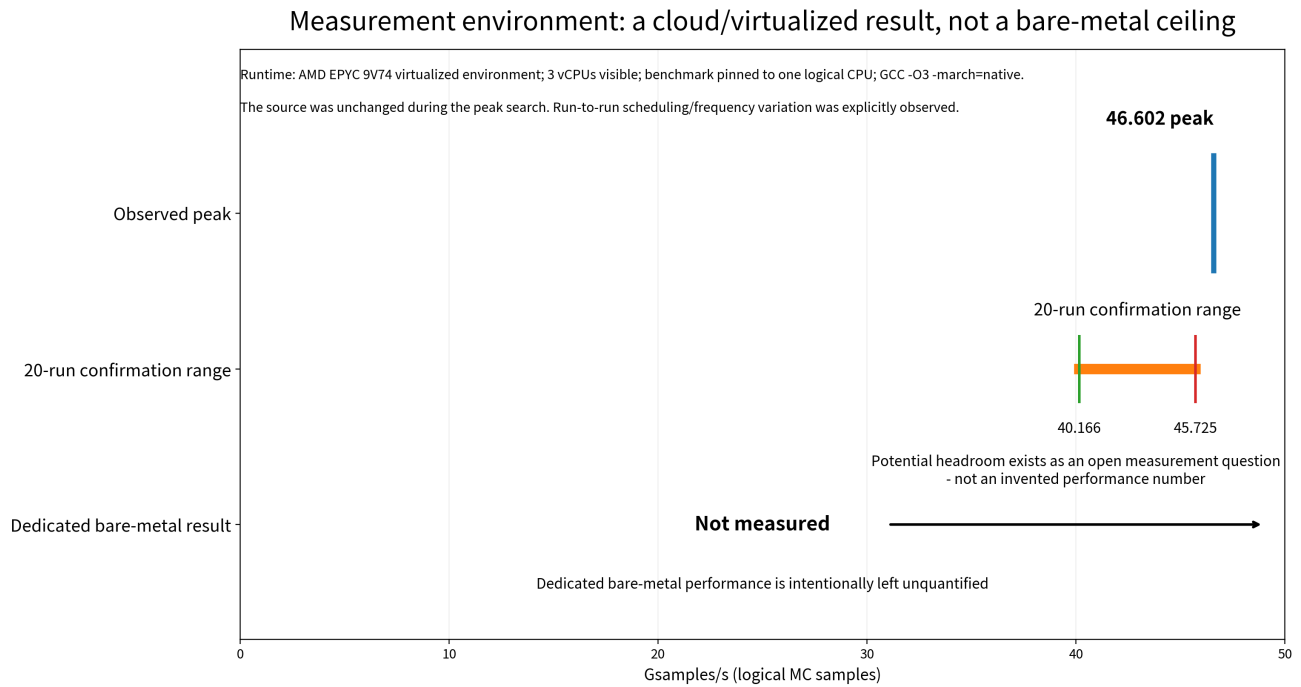
This appendix embeds the complete comparison-figure package used by the V1 Deconstruction Book. The figures are included in the PDF so that the principal visual evidence does not depend on external files. This

release adds three scope-aware visual aids before the original comparison set.

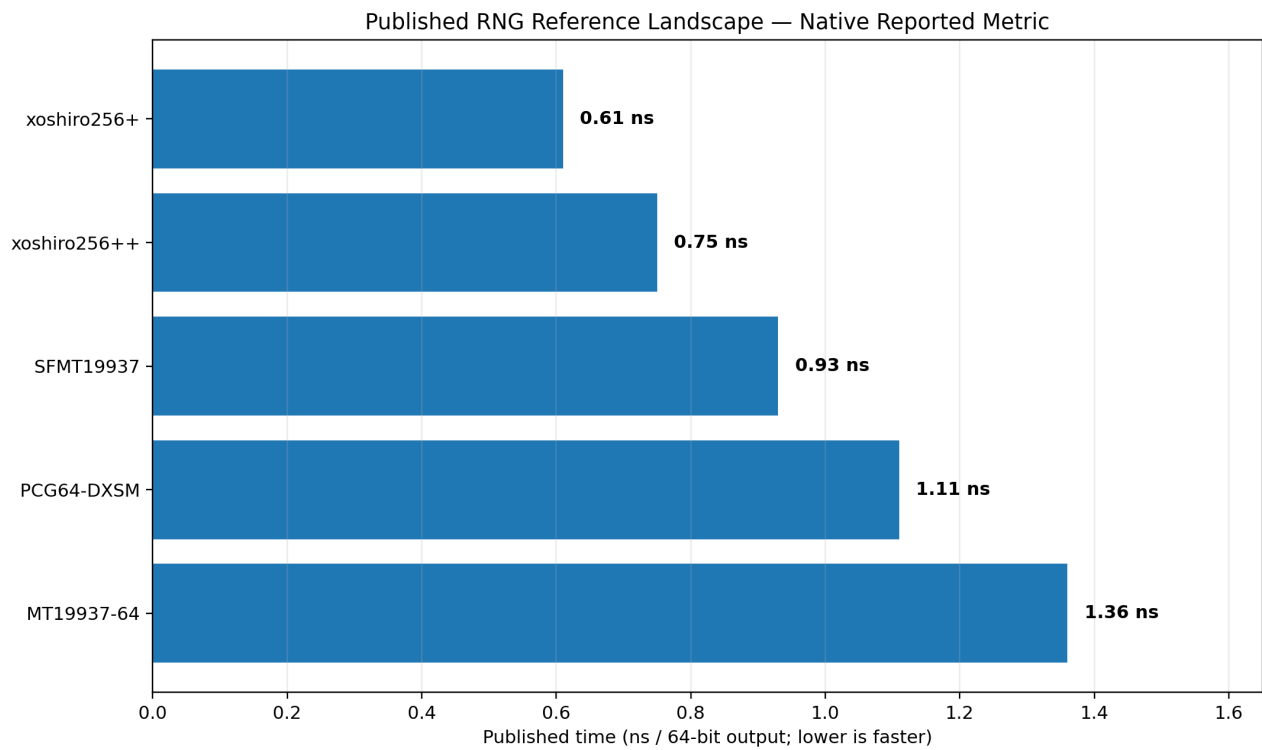
E.1 Scope and Measurement-Scope Overview



E.3 Measurement-Environment Context

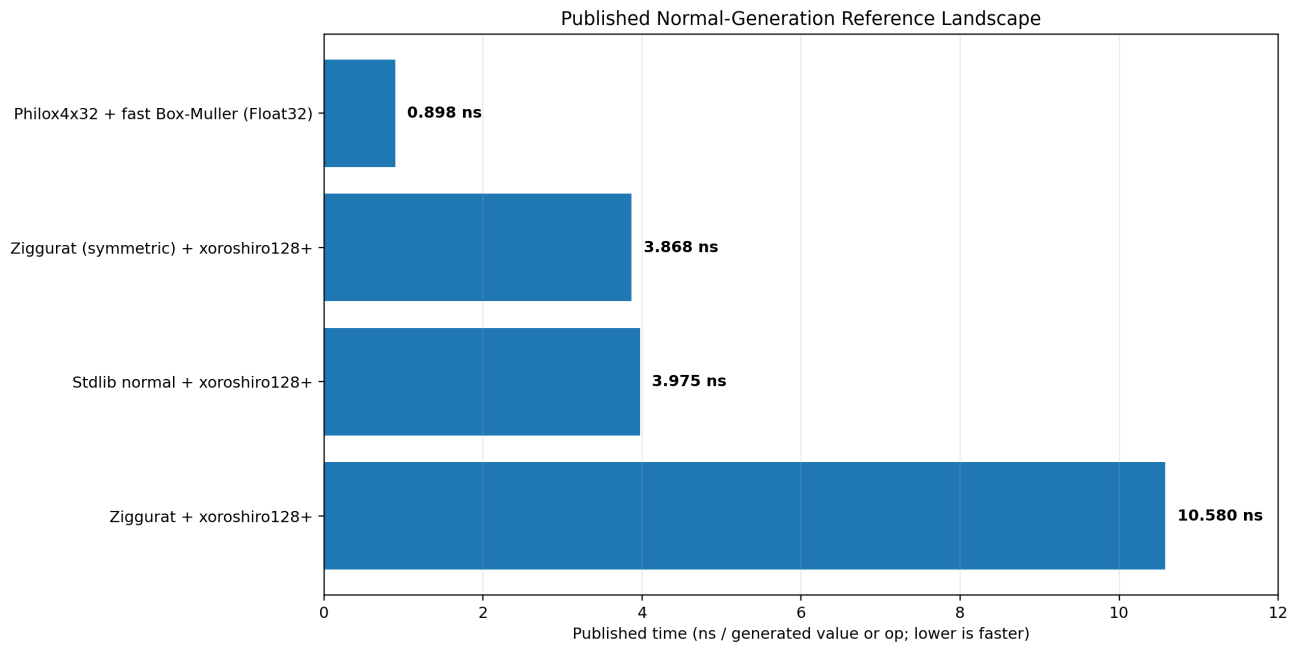


E.4 Published RNG Reference Landscape



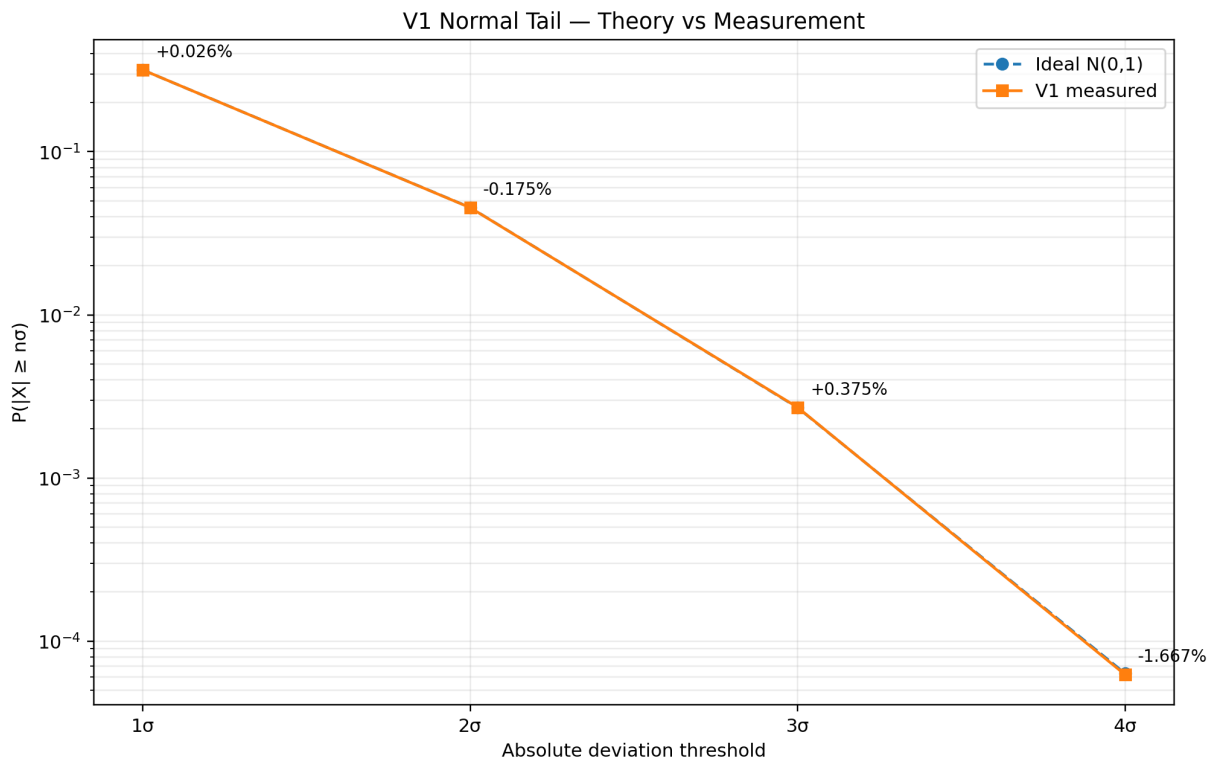
Reference-only chart. Values come from a public PRNG shootout and are not same-host comparisons to V1.

E.5 Published Normal-Generation Reference Landscape



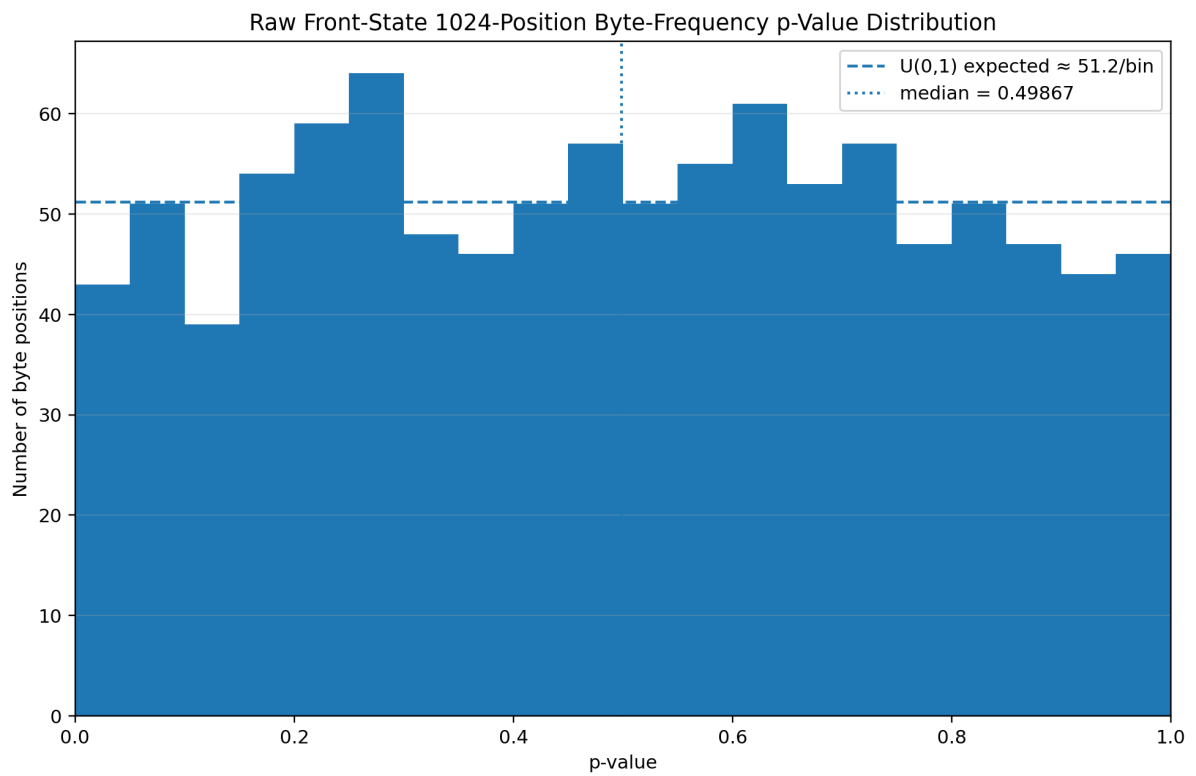
Reference-only. Sources, hardware, and exact operation definitions differ; not a direct V1 comparison.

E.6 V1 Normal Tail



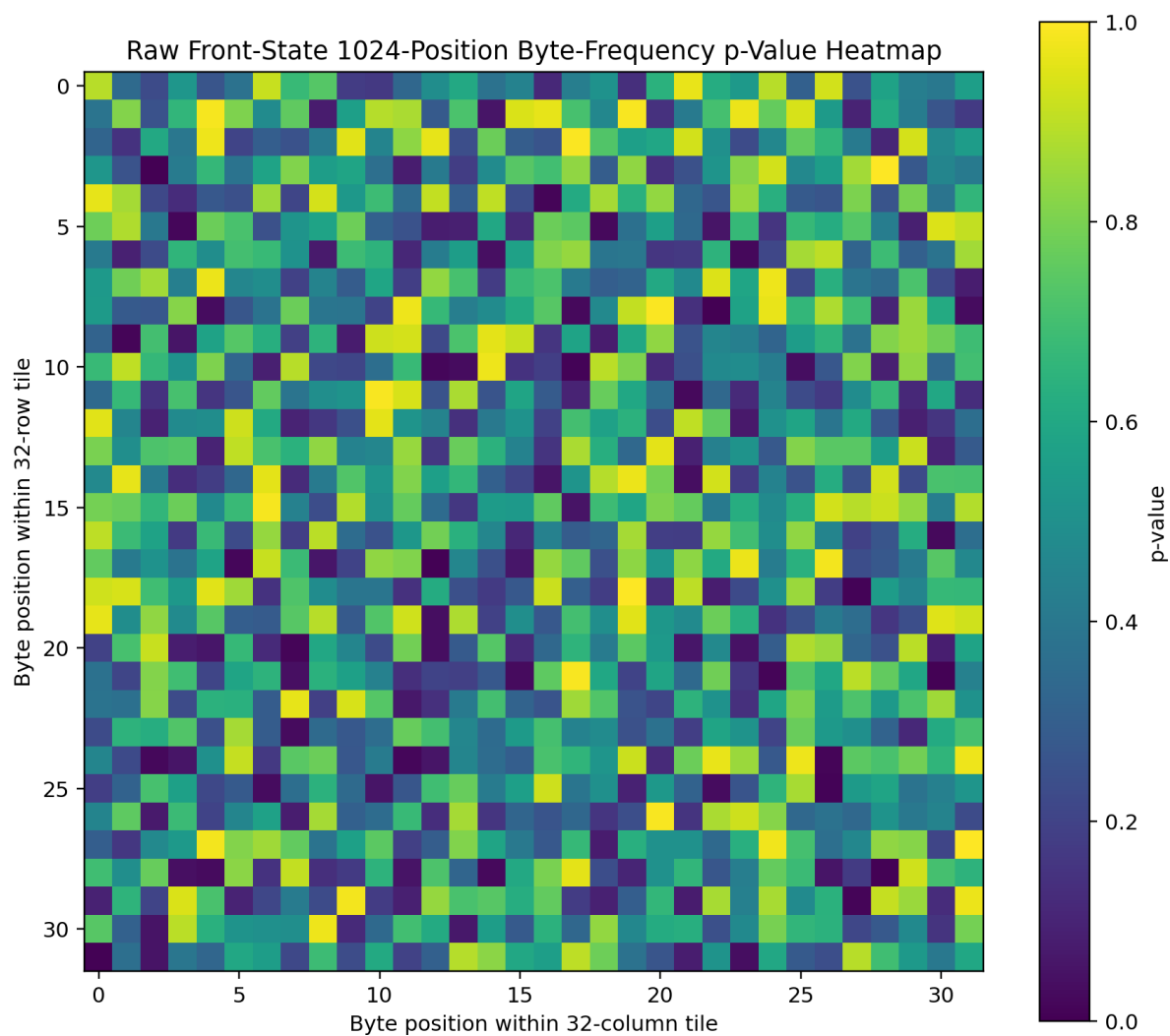
V1 aggregate: 134,217,728 x/y pairs. 4σ relative deviation = -1.667%; 4σ event counts are finite-sample sensitive.

E.7 1024-Position p-Value Histogram



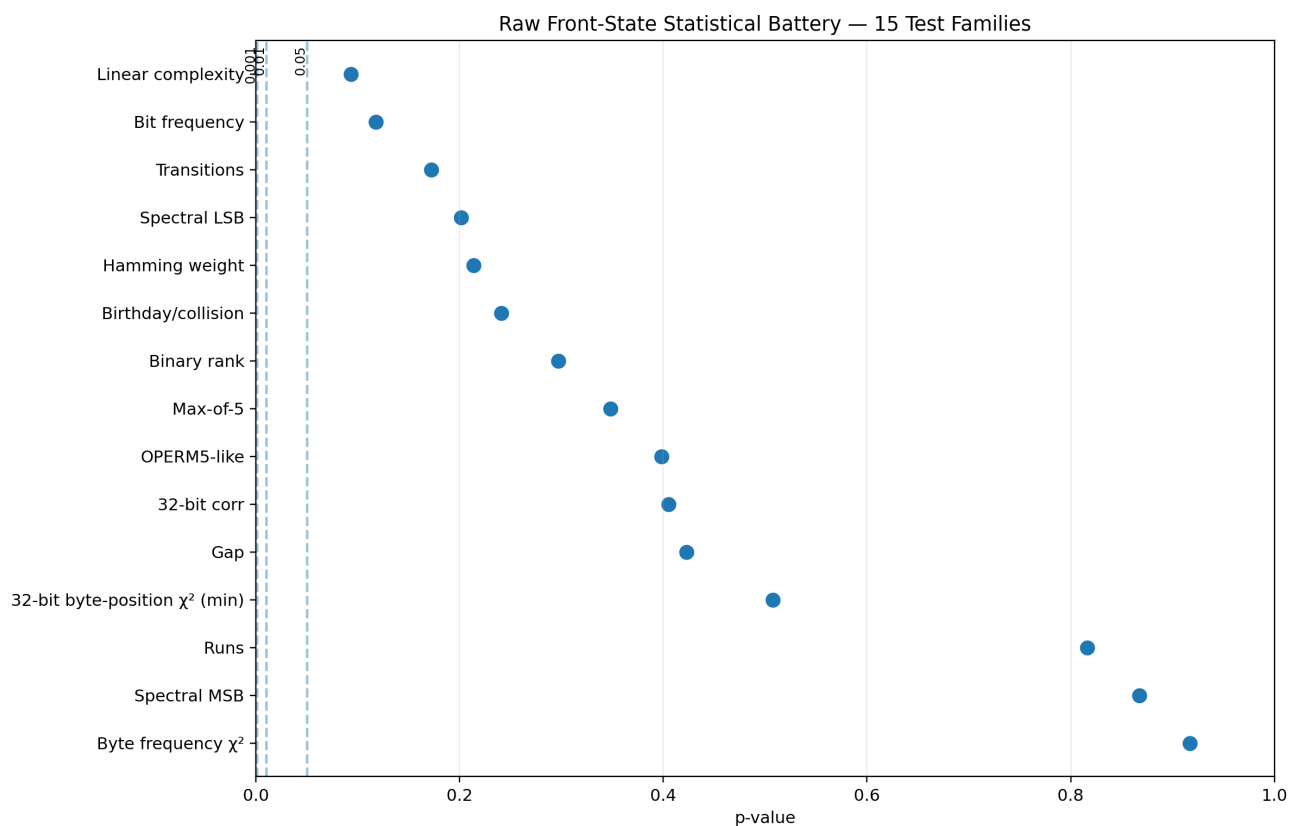
Exact 512 MiB stream. min p = 0.0016696; $p < 0.01$: 8/1024; $p < 0.001$: 0/1024.

E.8 1024-Position p-Value Heatmap



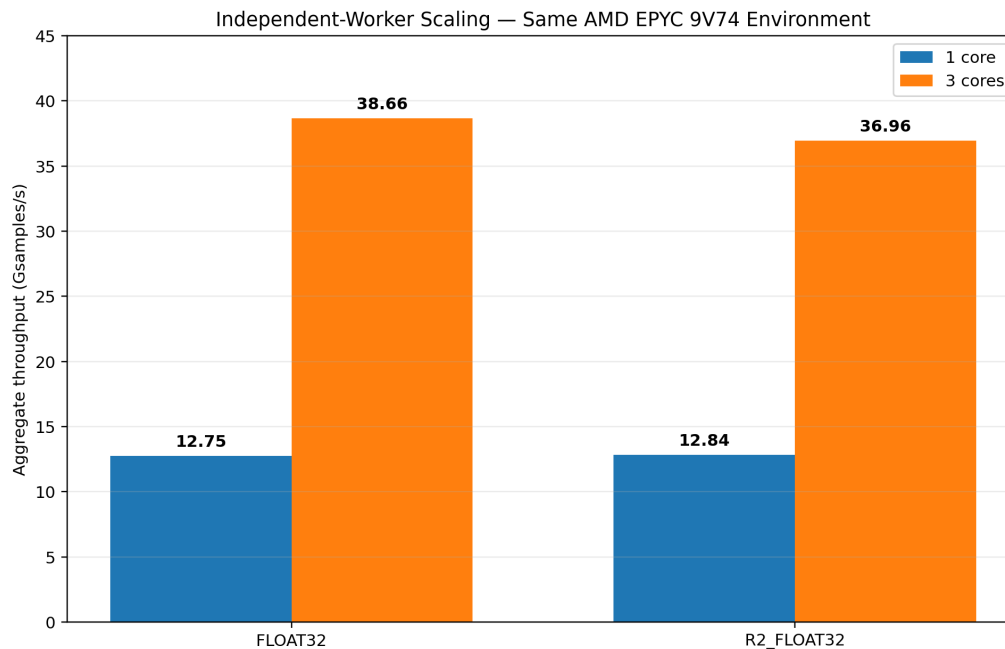
Each cell is one of the 1024 exact byte positions in the 1024-byte resident state. Median = 0.49867; min = 0.0016696.

E.9 Statistical Battery Overview



512 MiB raw Front-state stream. The chart is an empirical screen; $p > 0.05$ is not itself a certification.

E.10 Three-Core Scaling



FLOAT32 clean scaling = 3.032x; R2 clean scaling = 2.878x. Independent MC workers were used; this is not a claim that moving Back alone off-core accelerates one engine.

E.11 Supplemental Data Package

The accompanying figure package also contains:

README_FIGURES_v1.1.md
comparison_data.csv
position_pvalues_exact_1024.csv

These tabular files are source data for the embedded visual figures. The PDF embeds the rendered figures; the original CSV files remain part of the companion research archive.

Appendix F - External Source Notes Used by the Manuscript

The following public sources are used as methodological or contextual references. Their reported metrics are retained with the scope and hardware conditions stated by the original source.

F.1 PractRand

PractRand / Practically Random is a statistical test suite for PRNG data, with command-line tooling and large-stream testing capability. The current public project page lists release 0.96 dated 2025-12-26.

[PractRand documentation](#) / [project archive](#)

F.2 TestU01

L'Ecuyer and Simard, *TestU01: A C Library for Empirical Testing of Random Number Generators*, ACM Transactions on Mathematical Software 33(4), 2007, DOI 10.1145/1268776.1268777. TestU01 provides empirical statistical tests and predefined batteries for uniform random number generators.

F.3 Intel oneMKL Vector Statistics

Intel's oneMKL Vector Statistics documentation defines CPE (Clocks Per Element) as a normalized performance metric and explicitly notes dependencies on hardware architecture, BRNG, transformation method, vector length, and distribution parameters.

Gaussian / Box-Muller performance table: [Gaussian / Box-Muller table](#)

Gaussian / ICDF performance table: [Gaussian / ICDF table](#)

oneMKL performance methodology: [oneMKL performance methodology](#)

F.4 xoshiro Published PRNG Shootout

Vigna / Blackman, public PRNG shootout. The page states that speed is the time required to emit 64 random bits, and records the test platform as 12th Gen Intel Core i7-12700KF @ 3.60 GHz using GCC 12.2.1. The published table reports xoshiro256+ at 0.61 ns/64 bits.

[PRNG shootout](#)

F.5 MCKL Benchmark Documentation

MCKL 0.1.0 provides public batch RNG benchmarks for Philox, Threefry and MKL-backed engines. These values are retained as historical/contextual references because the hardware, metric and implementation boundary differ from V1.

[MCKL 0.1.0 benchmark PDF](#)

F.6 VMT19937 Public SIMD Benchmark

The public VMT19937 repository reports AVX-512 vectorial throughput figures for MT19937, MT19937-64 and SFMT19937. These are useful as an AVX-512 raw-RNG reference, but they are not treated as same-host comparisons with V1.

[VMT19937 repository](#)

F.7 Citation Policy Used in This Book

- Public values are labeled by their original scope and unit.
- A published number is not converted into a claimed V1 speedup unless the workload definition is actually the same.
- Unknown compiler versions or frequencies are not inferred.
- Formal named-suite results are only marked complete when an actual test output is archived.

The manuscript contains contextual references to public processor and RNG benchmark documentation. In this PDF, tool-specific citation tokens have been converted into readable source labels. The principal public sources used for those statements include AMD EPYC 9004 architecture documentation, AMD processor documentation for EPYC 9005, Intel AVX-512/optimization documentation, published RNG shootout material by Vigna and Blackman, and MCKL benchmark documentation.

Appendix G - Documentation License and Intellectual Property Notice

Document license: CC BY-NC-ND 4.0

This full-version Deconstruction Book is licensed under the **Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0)** license.

Intellectual Property and Use Notice

This book is a copyrighted work describing the architecture, design concepts, implementation considerations, evaluation methods, and related technical results of UltraFastRng512 V1. Copyright and other applicable rights in the text, figures, tables, structure, editing, and other original expression remain with the author or other lawful rights holder.

Under CC BY-NC-ND 4.0, non-commercial viewing, quotation, and redistribution of the unmodified work are permitted subject to the license terms. Commercial use and public distribution of adapted or modified versions of this book are not included in that license.

Nothing in this notice is intended to state that copyright by itself creates an exclusive right over ideas, algorithms, mathematical concepts, methods, procedures, or other abstract technical concepts described in the text. Where separate patent rights, trade secrets, or other intellectual-property rights exist or are subsequently obtained, those rights are governed by their own legal basis and terms.

The UltraFastRng512 core engine software is licensed separately under **AGPL-3.0-only and a separate Commercial License**. Permission to read or reference this book does not grant any additional license to the software, source code, or other protected project assets.

Commercial use, product integration, SaaS/cloud deployment, derivative products, or other uses of the Software or documentation may require separate permission depending on the applicable asset and license. Commercial inquiries should be directed to the rights holder.

Component License Separation

| Component | License |
|------------------------------------|------------------------------------|
| Core engine code | AGPL-3.0-only + Commercial License |
| Benchmark data / measured results | CC0 1.0 or CC BY 4.0, as marked |
| Sample code / usage examples | MIT License or 0BSD, as marked |
| Third-party / comparison material | Original license retained |
| Mathematical specification | CC BY 4.0 |
| Deconstruction Book - full version | CC BY-NC-ND 4.0 |

Important clarification: CC BY-NC-ND 4.0 applies to this documentation as a copyrighted work. It does not, by itself, create a patent right over ideas, algorithms, mathematical concepts, methods, or procedures described in the text. Any separately existing or subsequently obtained intellectual-property rights are governed by their own legal basis and terms.

Reference: [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International](#)

Rights Holder / Public Project Identity: Watney-0717

Official Communications & Commercial Inquiries: WatneyWatneyWatney0717@proton.me

Distribution note: This appendix forms an integral part of the complete English edition. The source V1 content is translated into English while preserving the mathematical definitions, source identities, benchmark values, code and static implementation data. This appendix establishes the documentation license and intellectual-property use boundary for this English edition.